

CAMPUS QUEUE

Project, code, database and viva master guide

Detailed technical reference

Prepared for project demonstration, code review and academic viva.

8 October 2026

Verified source, practical SQL and honest implementation boundaries. Credentials are deliberately excluded.

Contents

How to use this guide	6
1. Problem, objective and scope	7
2. Live system and deployment topology	8
3. Technology stack and responsibilities	9
4. Repository orientation	10
5. Service, counter and staff: three different concepts	11
6. Student journey and confirmation boundary	12
7. Operating hours and prebooking	13
8. Token identity, position and order	14
9. Ticket state machine	15
10. Staff workflow and telephone action	16
11. Missed turns and fairness	17
12. Closing risk and next-day carryover	18
13. Dynamic average processing time	19
14. ETA calculation and its limits	20
15. Transactions and concurrent booking	21
16. Database model and relationships	22
17. SQL practice: read and explain	23
18. SQL practice: safe inserts, updates and deletes	24
19. Authentication and validation	25
20. Security boundaries and limitations	26
21. Code walkthrough: HTTP to database	27

22. How to explain any random line	28
23. Local versus cloud database viewing	29
24. Cloud database setup	30
25. Render deployment step by step	31
26. Netlify deployment step by step	32
27. Deployment incident log: what went wrong and why	33
28. Hosting costs and demonstration reliability	34
29. Testing evidence and what remains	35
30. Known limitations to acknowledge confidently	36
31. Viva question bank: architecture and frontend	37
32. Viva question bank: Java, HTTP and deployment	38
33. Viva question bank: SQL and queue logic	39
34. Submission checklist and study allocation	40
35. File inventory and responsibility map	41
36. Numbered source and guided line explanations	50
QueueSystemFrontend/src/App.jsx	51
QueueSystemFrontend/src/main.jsx	64
QueueSystemFrontend/src/queue-layout.js	65
QueueSystemFrontend/src/student-api.js	66
frontend/src/App.jsx	68
frontend/src/components/ActivityFeed.jsx	70
frontend/src/components/QueueTable.jsx	71
frontend/src/components/StatCard.jsx	72
frontend/src/components/StudentPhone.jsx	73

frontend/src/main.jsx	74
frontend/src/pages/StaffDashboard.jsx	75
frontend/src/pages/StaffLogin.jsx	80
frontend/src/services/api.js	82
scripts/netlify-redirects.mjs	84
src/main/java/com/queue/controller/AnalyticsServlet.java	85
src/main/java/com/queue/controller/BookingServlet.java	87
src/main/java/com/queue/controller/QueueServlet.java	93
src/main/java/com/queue/controller/ServiceServlet.java	104
src/main/java/com/queue/controller/StaffServlet.java	106
src/main/java/com/queue/controller/StudentPortalServlet.java	116
src/main/java/com/queue/controller/StudentServlet.java	120
src/main/java/com/queue/dao/CounterDAO.java	124
src/main/java/com/queue/dao/QueueDAO.java	128
src/main/java/com/queue/dao/ServiceDAO.java	135
src/main/java/com/queue/dao/StaffDAO.java	137
src/main/java/com/queue/dao/StudentDAO.java	140
src/main/java/com/queue/filter/StudentOwnershipFilter.java	142
src/main/java/com/queue/model/Booking.java	144
src/main/java/com/queue/model/Counter.java	146
src/main/java/com/queue/model/Queue.java	147
src/main/java/com/queue/model/Service.java	149
src/main/java/com/queue/model/Staff.java	150
src/main/java/com/queue/model/Student.java	152

src/main/java/com/queue/service/AnalyticsService.java	154
src/main/java/com/queue/service/BookingService.java	159
src/main/java/com/queue/service/ClosingTimeListener.java	167
src/main/java/com/queue/service/NoShowService.java	168
src/main/java/com/queue/service/ReservationService.java	172
src/main/java/com/queue/service/ServiceHours.java	178
src/main/java/com/queue/service/ServiceService.java	180
src/main/java/com/queue/service/StaffService.java	181
src/main/java/com/queue/service/StudentService.java	184
src/main/java/com/queue/service/WaitingTimeService.java	185
src/main/java/com/queue/util/DBConnection.java	188
src/main/java/com/queue/util/DatabaseSetup.java	189
src/main/java/com/queue/util/Json.java	190
src/main/java/com/queue/util/Passwords.java	191
src/main/java/com/queue/util/StudentValidation.java	192
src/main/java/com/queue/util/TransactionLocks.java	193
Dockerfile	194
37. HTML, CSS and SQL source practice	195
Academic preparation edition 8 October 2026	

How to use this guide

Read the architecture, queue rules and database chapters first. Then read the walkthrough for the module you own. Practise the SQL exercises on a scratch database. Use the numbered source appendix to rehearse explaining unfamiliar lines. This is a reference manual, not a script to memorize. The separate presentation runbook provides the demonstration order.

This edition describes the integrated repository on main. Local test fixtures and credentials are excluded. Screens and source may evolve; the line numbers in the appendix refer to the snapshot used to generate this document. Regenerate it after substantial changes.

1. Problem, objective and scope

Campus offices handle general inquiries, fee payments and document verification. A physical line requires students to stand at a counter and gives little visibility into their turn. Campus Queue lets a student reserve a place in a service queue, inspect the projected position and estimated time, and track the ticket while staff process the queue.

The core contribution is coordination: the student interface, staff interface and Java backend share one database. Staff actions change ticket states, and the student interface polls the backend to display the change. The application does not process bank payments, verify documents automatically, send SMS, or conduct phone calls itself.

Three services are configured: General Inquiries, Fee Payment and Document Verification. Their initial processing estimates are 10, 15 and 20 minutes. These are defaults until real completed-service samples exist; they are not appointment slots.

Explain the project in one sentence: "Our application manages same-day campus service queues with online reservation, service-specific staff access, measured processing estimates and a controlled missed-turn policy."

2. Live system and deployment topology

Student website: <https://campusque.netlify.app/> Staff website: <https://campusquestaff.netlify.app/> Backend service: <https://queuesystem-1.onrender.com> Public catalogue check: <https://queuesystem-1.onrender.com/api/services> Repository: <https://github.com/Apeksha2203/QueueSystem>

The browser downloads React HTML, CSS and JavaScript from Netlify. Requests to /api on that same Netlify origin are rewritten to the Render backend. Tomcat hosts the Java WAR on Render and uses JDBC to access MySQL on Railway. Workbench on a PC is another client of that same cloud MySQL server.

Student browser -> student Netlify /api proxy -> Render Tomcat -> Railway MySQL. Staff browser -> staff Netlify /api proxy -> the same Render Tomcat -> the same Railway MySQL. Workbench -> Railway public TCP proxy -> the same Railway MySQL.

GitHub stores code, not running database rows. A push triggers deployment if automatic deployment is enabled. Updating Java source does not copy local MySQL data into Railway. Export/import is a separate deliberate operation.

3. Technology stack and responsibilities

HTML supplies document structure. CSS controls layout, colour, spacing, typography, responsive rules and animation. JavaScript handles browser behaviour. JSX describes React component trees and is compiled into JavaScript by Vite; the browser does not execute JSX directly.

React manages components and state. The student app uses HashRouter, producing routes such as `/#/bookings`. The staff app uses BrowserRouter and needs a static-host fallback for routes such as `/dashboard`. Vite is a development server and build tool; its local proxy does not configure production routing.

Java 17 implements request handling and business rules. Jakarta Servlet 6 provides HTTP request/response/session APIs. Tomcat 10.1 is the servlet container. Maven resolves dependencies, compiles classes and packages a WAR. MySQL Connector/J is the JDBC driver. JDBC exposes Connection, PreparedStatement and ResultSet.

JSP files remain in `src/main/webapp` from earlier teammate modules. They are not the deployed React dashboards. Be honest if asked: "The current primary dashboards use React; JSP is retained as legacy code and is processed by Tomcat when requested." Do not describe this project as Spring Boot or claim it runs an executable Spring JAR.

4. Repository orientation

QueueSystemFrontend/ contains the student app, package files, public assets and Netlify config. frontend/ contains the staff app. src/main/java/com/queue/controller contains HTTP adapters. service contains business rules. dao contains database access. model contains Java data objects. util contains configuration, hashing, JSON, validation and locking helpers. filter contains legacy route ownership enforcement.

database/schema.sql creates campus_queue and the base tables. migrations/001_daily_reservations.sql adds reservation metadata, event history and counter state. migrations/002_processing_duration.sql adds measured-processing fields. seed_services.sql inserts the service catalogue without duplicate names.

scripts/ contains local startup/configuration, staff seed generation, deployment redirect generation and test tools. scripts/tests/ contains standalone validation programs. These are not automatically all executed by mvn package. Dockerfile builds the backend image. .dockerignore limits the build context. .gitignore prevents local credentials and generated runtime artifacts from being committed.

TEAM-CHANGELOG.md records implemented changes. BRANCH-INTEGRATION.md explains integration of teammate branches. CLOUD-DEPLOYMENT.md documents hosting. Earlier plans/reviews and legacy tests can describe superseded behaviour; use the current code and the explicit limitations chapter for final claims.

5. Service, counter and staff: three different concepts

A service is the work requested: for example Fee Payment. A counter is a physical or logical desk that serves one service. A staff member is a person/account assigned to a service and counter. Multiple counters can serve one service queue.

All three demonstration accounts have staff privileges, with different service assignments. They are not three independent global security roles. The backend reads `staff_id` from the authenticated session, resolves the account's service and counter, and scopes queue operations accordingly. Frontend hiding is not the authorization boundary.

There is no implemented administrator portal or administrator login in this snapshot. Assignments are provisioned in database setup scripts. An admin portal with audited assignment management is a future improvement, not a feature to demonstrate as completed.

The identifiers stored in `staff.service_id` and `staff.counter_id` must agree with `counters.service_id`. `ReservationService` validates the counter-service match before processing a ticket. Keeping both references is convenient for current queries but creates redundant data that must remain consistent.

6. Student journey and confirmation boundary

The public landing page explains the workflow. The student creates an account using name, email, Indian mobile number and password, then signs in. The overview shows available services and any active ticket. Selecting a service card opens the booking flow with that service selected; browsing does not create a ticket.

The booking preview shows projected position, people ahead, estimated service clock time and operating hours. Only an explicit Reserve my place submission creates queue and reservation rows. A preview can become stale when another student books first, so the backend recalculates eligibility and ordering in the reservation transaction.

The queue page shows token, service, state, position and guidance. A waiting ticket for today can be cancelled by its owner. Bookings lists active/upcoming and past records. Profile/theme preferences should not be presented as proof of server-side notification delivery.

7. Operating hours and prebooking

The campus timezone is Asia/Kolkata. Service is open from 10:00 inclusive to 13:00 exclusive, and from 14:00 inclusive to 15:00 exclusive. At 13:00 the lunch break begins; at 15:00 the service window closes. Registration of a new queue reservation is for today's date only.

A student may reserve at 07:30 for today's queue even though staff have not opened the desk. This establishes queue order, not a right to immediate service. Pre-opening estimates start from 10:00 and assume the scheduled counters will be staffed. At lunch the next service opening is 14:00. The system does not currently implement holidays or weekends; "next day" means the next calendar day.

Local testing previously used an ignored `test-clock.local.properties` file. It has been disabled for the final schedule. `ServiceHours` reads real campus time when no override is present. The Docker image does not copy local configuration or test-clock files. Do not change the Render web PORT to change operating hours.

8. Token identity, position and order

A token is a stable label within a service. ReservationService computes the next token as $\text{MAX}(\text{token_number})+1$ for that service under a service lock. Tokens do not reset each day in the current implementation. The displayed number is zero-padded, such as 001. Some older staff components prefix a token with A; this is presentation, not a separate queue identity.

queue_id is the database primary key and identifies the record. token_number is a service-local display label. queue_order in queue_reservations determines FIFO order for a visit date. Position is derived from currently active tickets and can change when others complete, cancel, miss a call or when counter assignments change.

Example: token 017 can move from position 10 to position 1 without its token changing. A carried ticket also preserves its token. Do not confuse token 017 with a guarantee that sixteen people are ahead now.

9. Ticket state machine

The normal path is WAITING -> CALLED -> SERVING -> COMPLETED. WAITING means booked but not assigned for service. CALLED means staff has summoned and assigned the ticket to a counter. SERVING means processing has started. COMPLETED means staff explicitly completed a started service.

WAITING -> CANCELLED is permitted for the owner and today's reservation. CALLED -> WAITING occurs after the first absence. CALLED -> NO_SHOW occurs after the second absence. A staff skip can produce SKIPPED. Closing rollover preserves WAITING and updates the reservation date/order; it must not falsely mark work COMPLETED.

The backend rejects invalid transitions, such as completing WAITING or marking a SERVING student absent. The UI buttons reflect state, but the service layer rechecks state and ownership in the transaction.

10. Staff workflow and telephone action

At the first queue of a morning session, staff explicitly summons the next student. CALLED exposes Start and Mark absent. Start records started_at and changes the state to SERVING. Complete records completion and processing duration. During the same open operating window, completion can automatically summon the next waiting student.

An explicit summon is required after lunch before the afternoon sequence. A SUMMON_MORNING or SUMMON_AFTERNOON queue event identifies the operating window. If a service crosses lunch, automatic continuation does not silently start the afternoon queue. Automatic advancement also respects counter availability.

The telephone icon uses a tel: link with the student's normalized contact number. It asks the browser/device's calling application to handle a call. A PC may prompt for an app; the website does not initiate a server-side telephone call or know whether the student answered. Calling the phone does not change ticket state.

11. Missed turns and fairness

Only a CALLED student can be marked absent. On the first miss, missed_turns increments, the ticket returns to WAITING, the assignment is cleared and the ticket is inserted behind up to ten waiting students. If fewer than ten are waiting, it goes behind those available students. This is not always literally position ten: with ten people ahead, the absent student becomes the eleventh waiting ticket.

On the second miss, the ticket becomes NO_SHOW and exits the active queue. This ends that ticket; it does not permanently ban the student's account. The student does not lose their token history. The service records event types MOVED_BACK_FIRST_MISS and REMOVED_SECOND_MISS.

An absent student is excluded from immediate automatic recall when no other student is waiting. A waiting ticket rolled to tomorrow due to closing receives no additional missed-turn penalty.

12. Closing risk and next-day carryover

The booking preview projects the time at which the student's turn would start. If that projected time is at or after 15:00, a new booking is rejected with guidance to return tomorrow. Existing waiting students can see a closing-risk warning as queue estimates change.

At closing, pending WAITING tickets are moved to the next day's 10 AM queue. Their tokens and missed counts are preserved. Carryovers are ordered ahead of new reservations already created for the destination date. They remain WAITING; completion requires an actual staff completion action.

ClosingTimeListener runs reconciliation periodically, and reads also invoke reconciliation. After an outage, older waiting dates can be moved to the current open day's queue or the next day if already closed. The operation locks a service and is idempotent: tickets already moved are not duplicated on the next run.

CALLED and SERVING tickets are not part of waiting rollover. Finishing a current service after closing is allowed where the current date/ownership rules still permit it. Cross-midnight called/serving handling needs further validation and should not be claimed as fully resolved.

13. Dynamic average processing time

Average processing time is based on actual processing, not the interval from booking to completion. A booking at 07:00, a service start at 10:00 and completion at 10:03 contribute approximately three processing minutes, not three hours and three minutes.

WaitingTimeService reads the latest twenty valid COMPLETED records for that service, ordered by completed_at and queue_id. It uses processing_seconds where present, otherwise it calculates the overlap of started_at/completed_at with scheduled service windows. Only positive durations are averaged. The lower bound is one second, preserving fast demonstration measurements.

If five positive samples total fifteen minutes, the average is three minutes. Adding a sixth sample of nine minutes produces twenty-four divided by six = four minutes. Once there are more than twenty samples, the oldest sample drops out. The average is per service, not pooled across unrelated services.

When there are no usable samples, the service's configured estimate is used. Seeing 10, 15 or 20 on a newly deployed empty database is therefore expected. The public /api/services endpoint returns configuration defaults; the authenticated student overview computes dynamic averageServiceTime separately.

14. ETA calculation and its limits

For a booking preview, processing backlog is approximately $\text{peopleAhead} * \text{measuredAverage} / \text{eligibleCounters}$. `ReservationService` rounds the required backlog up to whole minutes and projects through service openings and the lunch break. Helpers in `WaitingTimeService` also expose round-based text/integer estimates; distinguish them from the actual preview calculation.

Position ten ordinarily means nine people ahead. At three minutes per person and one counter, waiting before the student's service is about twenty-seven minutes. Ten people ahead means about thirty minutes. The student's own processing time is not part of time until their service starts.

Before opening, a second reservation with a three-minute average and one counter projects about 10:03. At 12:58 with five minutes of backlog, projection skips lunch and becomes 14:03. With two counters, the calculation assumes both can process at the measured average rate.

The student app polls about every eight seconds and also refreshes when focused. The staff app polls about every seven seconds. This is polling, not WebSockets or instantaneous synchronization.

Important limit: the current estimator updates completed-service samples after completion. It does not continuously predict the remaining duration of a particularly long active service, and it is not a machine-learning model. Parallel counter speed differences are approximated. Do not promise a guaranteed arrival or completion time.

15. Transactions and concurrent booking

Two browsers can confirm a reservation simultaneously. Without protection, both could read the same maximum token and insert conflicting logical order. TransactionLocks locks the service's parent row with `SELECT ... FOR UPDATE`. All participating reservation/operation transactions acquire the same service lock, serializing changes to that service.

`reserve` opens a connection, sets `READ_COMMITTED` isolation, disables auto-commit, acquires the service lock, recomputes preview, calculates token/order, inserts queue, inserts queue_reservations, adds `RESERVED` event, and commits. Any exception rolls back the entire transaction. This prevents an orphan queue record from a partial insert.

`operate` resolves the staff's service/counter, locks the service and counter, validates current assignment/state, applies the action and related event, optionally advances the next ticket, and commits. Different services have different parent-row locks and can proceed independently.

Cancellation is currently a conditional update rather than the same multi-step transaction. A conditional `WHERE` on owner, status and date provides basic state protection, but comprehensive database constraints and race testing remain valuable. `MAX+1` relies on all writers following the lock convention; manually bypassing the service layer can undermine it.

16. Database model and relationships

students: student_id primary key; name, unique email, normalized phone and password hash. services: service_id, name, description and initial average. counters: counter_id, name, service foreign key and active flag. staff: staff_id, unique email, hash and service/counter assignments.

queue: queue_id, student/service foreign keys, token, state and joined/called/started/completed timestamps. Migration 002 adds processing_seconds and processing_clock_offset_seconds. queue_reservations: one-to-one metadata keyed by queue_id, visit_date, queue_order, missed_turns and optional staff/counter assignment.

queue_events: append-only event records with event_id, queue_id, optional staff_id, event_type and occurred_at. counter_states: one row per counter containing AVAILABLE, PAUSED or CHECKED_OUT. bookings: older appointment-slot records; the current same-day reservation flow writes queue plus queue_reservations, not this legacy table.

Primary keys identify rows. Foreign keys preserve relationships. students.email and staff.email have unique constraints. NOT NULL rejects absent required values. DEFAULT supplies initial state or timestamps. The daily_order index on reservation date/order supports ordered queue retrieval but is not a unique constraint.

Deleting a queue row cascades to its reservation and events according to their foreign keys. Deleting a student who still has queue/bookings records may fail; understand child rows before deleting. Not every relationship uses ON DELETE CASCADE. staff_id in queue_events is currently not a declared foreign key.

17. SQL practice: read and explain

Use a scratch database or read-only queries for viva. Never experiment with unrestricted DELETE or UPDATE on teammates' live bookings. The following SELECTs are safe to run against the cloud schema.

```
USE campus_queue;
SHOW TABLES;
DESCRIBE students;
SHOW CREATE TABLE queue_reservations;
SELECT service_id, service_name, average_service_time FROM services;
SELECT queue_id, token_number, status FROM queue ORDER BY queue_id DESC LIMIT 10;
```

Explain USE as selecting the default schema; SHOW TABLES as listing objects; DESCRIBE as exposing columns/types/nullability/keys; SHOW CREATE TABLE as revealing exact constraints. WHERE filters rows, ORDER BY sorts, LIMIT bounds output, and selecting explicit columns avoids unnecessary sensitive fields such as password hashes.

```
SELECT q.queue_id, s.student_name, v.service_name,
       q.token_number, q.status, r.visit_date, r.queue_order
FROM queue q
JOIN students s ON s.student_id = q.student_id
JOIN services v ON v.service_id = q.service_id
JOIN queue_reservations r ON r.queue_id = q.queue_id
ORDER BY r.visit_date, r.queue_order, q.queue_id;
```

This INNER JOIN returns tickets with all required related records. Aliases shorten qualified names. Explain each ON clause as the relationship being matched, not a condition that invents a new relationship. A LEFT JOIN is appropriate when assignment is optional and unassigned tickets must remain visible.

```
SELECT v.service_name, q.status, COUNT(*) AS tickets
FROM queue q JOIN services v ON v.service_id = q.service_id
GROUP BY v.service_name, q.status
ORDER BY v.service_name, q.status;

SELECT service_id, AVG(processing_seconds) / 60 AS avg_minutes,
       COUNT(processing_seconds) AS measured_count
FROM queue WHERE status = 'COMPLETED' AND processing_seconds > 0
GROUP BY service_id HAVING COUNT(processing_seconds) >= 1;
```

COUNT(*) counts rows; COUNT(column) ignores NULL. AVG ignores NULL. WHERE runs before grouping; HAVING filters groups after aggregation. This illustrative AVG uses all matching completed records, whereas application ETA uses the most recent twenty valid samples.

18. SQL practice: safe inserts, updates and deletes

For live demonstration of CRUD, create a separate scratch schema. This shows DDL and DML without changing real queue records or storing a fake plaintext password in the application tables.

```
CREATE DATABASE IF NOT EXISTS campus_queue_viva;
USE campus_queue_viva;
CREATE TABLE IF NOT EXISTS demo_visits (
  id INT PRIMARY KEY AUTO_INCREMENT,
  student_name VARCHAR(100) NOT NULL,
  service_name VARCHAR(100) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'WAITING'
);
INSERT INTO demo_visits (student_name, service_name)
VALUES ('Viva Test Student', 'General Inquiries');
SET @demo_id = LAST_INSERT_ID();
SELECT * FROM demo_visits WHERE id = @demo_id;
UPDATE demo_visits SET status = 'CALLED' WHERE id = @demo_id;
SELECT * FROM demo_visits WHERE id = @demo_id;
DELETE FROM demo_visits WHERE id = @demo_id;
SELECT * FROM demo_visits WHERE id = @demo_id;
```

DDL creates or changes structures; DML manipulates row data; SELECT retrieves data. AUTO_INCREMENT provides a surrogate key. LAST_INSERT_ID() is connection-specific. A DELETE with a primary-key condition targets one row. Deleting a row and dropping a table are different operations.

```
START TRANSACTION;
INSERT INTO demo_visits (student_name, service_name)
VALUES ('Rollback Example', 'Fee Payment');
SELECT * FROM demo_visits WHERE id = LAST_INSERT_ID();
ROLLBACK;
SELECT * FROM demo_visits WHERE student_name = 'Rollback Example';
USE campus_queue;
```

Explain COMMIT as making a transaction permanent and ROLLBACK as discarding uncommitted changes. MySQL DDL can implicitly commit, so do not assume CREATE TABLE can be rolled back like an INSERT. If asked to demonstrate foreign-key rejection, use a scratch parent/child pair instead of modifying application assignments.

19. Authentication and validation

The student browser submits form-encoded data. StudentPortalServlet validates registration through StudentService and StudentValidation, hashes the password, inserts the student, and creates a session after successful login. Replacing the prior session reduces session fixation risk. studentId or staffId in the server session is trusted identity, not a client-supplied ID.

Passwords uses PBKDF2WithHmacSHA256 with 210000 iterations, a random sixteen-byte salt and a 256-bit derived key. The stored format includes algorithm, iteration count, salt and derived hash in Base64 components. A salt makes identical passwords produce different stored hashes. Base64 is encoding, not encryption. The password is verified by deriving the candidate hash and comparing bytes.

The project's user-requested password rule is six to 128 characters with at least one ASCII letter and number. It is deliberately simple for demonstration and is not a claim of strong production password policy. Registration accepts an Indian ten-digit number beginning 6-9, optionally +91, and normalizes it to +91 form. Email validation checks syntax and length; it does not verify ownership or restrict every account to an actual college domain.

The student cookie session expires after thirty minutes of inactivity. Frontend fetch includes credentials so the browser carries the session cookie. Logout invalidates the server session. Staff and student domains receive their own proxy-origin cookies; do not assume a login on one domain signs into the other automatically.

Legacy plaintext passwords can be migrated after a successful login in retained DAO paths. Cloud staff setup stores hashes from the beginning. Do not show stored hashes or database passwords in screenshots unnecessarily, and do not publish the real database credentials in this manual.

20. Security boundaries and limitations

PreparedStatement binds values using ? placeholders rather than concatenating user input into SQL. This prevents parameter values from being interpreted as SQL syntax. It does not secure arbitrary dynamically constructed table names or replace authorization checks.

Staff operations verify their assigned service, counter, student ticket, ownership and current state. Student cancellation matches the student session ID. A client that changes a serviceId cannot gain another staff account's service access merely by editing the frontend.

Student mutating requests require an X-Requested-With header in the current adapter. This is not a comprehensive CSRF design. The application does not yet claim rate limiting, password recovery, email/phone verification, MFA, comprehensive audit administration or fine-grained admin access. A publicly hosted demo with shared staff credentials should not be treated as hardened production infrastructure.

Some older endpoints retain manual JSON construction and DAO error-swallowing patterns. /api/services was corrected to return HTTP 503 on catalogue query failure rather than a misleading successful empty array. Safe console diagnostics expose exception type, SQLState and vendor code, not database credentials.

21. Code walkthrough: HTTP to database

For a reserve click, find BookingForm in student App.jsx. Locate studentRequest in student-api.js: it chooses GET or POST, includes credentials, uses URLSearchParams for the body, parses JSON and throws an error when HTTP or application success indicates failure.

Then open StudentPortalServlet. @WebServlet("/api/student/*") maps the route prefix. doGet/doPost dispatch to the handler. /bookings/create checks the date and invokes ReservationService.reserve with the authenticated student ID. The service locks, validates and writes. The servlet serializes the result to JSON; the frontend refreshes state to render the ticket.

For a staff completion, trace StaffDashboard.runAction -> api.completeService -> POST /api/queue/complete -> QueueServlet -> ReservationService.operate. The service checks ownership and SERVING state, calculates processing duration, updates COMPLETED, records an event, may summon next, then commits. QueueTable and ActivityFeed render the refreshed response.

For ETA, trace student overview/preview -> WaitingTimeService -> completed sample SQL -> ServiceHours processing overlap -> projection. This is distinct from AnalyticsService's dashboard statistics, some of which still use elapsed timestamps and can include pre-opening waiting.

22. How to explain any random line

Begin with the file's responsibility, then identify the enclosing function and input/output. Explain the statement in plain language, why it is needed, and what happens on failure. Follow the caller or callee if needed. Avoid reading punctuation aloud without meaning.

Example: `p.setInt(1, servid)` binds `servid` to the first SQL placeholder; it does not execute the query. `executeQuery` executes a `SELECT` and returns `ResultSet`. `while(r.next())` advances through records. `try-with-resources` automatically closes result sets, statements and connections even after an exception.

Example: `c.setAutoCommit(false)` groups subsequent statements into a transaction. `c.commit()` persists them together. `c.rollback()` undoes pending statements on that connection. `FOR UPDATE` holds a row lock until the transaction completes; it is not a Java synchronized block.

Example: `useState` creates component state and a setter; `useEffect` synchronizes with an external process such as polling and returns cleanup; `useMemo` caches a derived calculation by dependencies; `useCallback` preserves a function identity by dependencies. An interval must be cleared on unmount to avoid background work from abandoned screens.

Example: conditional JSX selects what to display based on state; it does not authorize database access. `map` renders a list, and a stable key helps React preserve identity. Optional chaining accesses a field only when the value is not nullish. A CSS grid declaration distributes columns; media queries change the rule at a viewport boundary.

The source appendix includes numbered current production files and a file role map. Match a judge's selected line to this sequence: declaration -> input -> validation -> work -> result/error -> next consumer.

23. Local versus cloud database viewing

MySQL is the database server. Workbench is a visual client, comparable to Supabase's table/SQL editor experience but provided separately. Expand `campus_queue` -> Tables and open/select table rows to view them; a result grid can refresh without typing a new query every time.

Create separate Workbench connections: local `127.0.0.1:3306` and Campus Queue Cloud using Railway's public host/port. They point to different servers. Opening cloud MySQL from your PC shows online data because the PC connects to the cloud; it does not create an offline synchronized copy.

For the viva, open Campus Queue Cloud and show tables, DESCRIBE, joins and the newly created queue record. If asked to show a truly local database, use a deliberate export/import snapshot. Automatic bidirectional offline synchronization is not implemented. An offline PC cannot view the current cloud database without network access.

24. Cloud database setup

On Railway create a project and add MySQL. Apply pending changes and wait until running. Enable a public TCP proxy in Networking. MYSQL_PUBLIC_URL has a form such as `mysql://USER:PASSWORD@PUBLIC_HOST:PUBLIC_PORT/railway`. The hostname is the part after `@` and before the host-port colon; the public port is the following number. The variable name itself is not a hostname.

In Workbench create a MySQL connection and enter the actual public host, public port, username and password separately. Test Connection. Do not use `mysql.railway.internal` from a PC or Render; that private address is for services inside Railway.

Run `schema.sql`, migration 001, migration 002 and `seed_services.sql` in order. The project creates `campus_queue` rather than using the default railway schema. Generate the ignored `.runtime/cloud-staff-seed.sql` with `scripts/create-cloud-staff-seed.py` and execute it on the cloud connection to create counters and staff assignments. That seed uses PBKDF2 hashes and preserves existing accounts.

There are no bank/API provider keys required for the core project. Database credentials are secrets. `.env.example` is a template; `.env` and local config files are ignored. Java reads environment variables and does not automatically load a `.env` file. VITE-prefixed environment values become browser-visible and must never contain database passwords.

25. Render deployment step by step

Create New -> Web Service and connect Apeksha2203/QueueSystem, main branch. The repository owner must grant the Render GitHub app access for a private repository; collaborator push permission alone is not sufficient. Select Docker runtime and leave Root Directory blank. Choose Free compute explicitly; do not accidentally leave a paid selection enabled.

Set PORT=8080 for Tomcat HTTP. Set QUEUE_DB_USER and QUEUE_DB_PASSWORD to database credentials. Set QUEUE_DB_URL to jdbc:mysql://PUBLIC_HOST:PUBLIC_PORT/campus_queue?connectionTimeZone=Asia/Kolkata. The MySQL public port and the Tomcat HTTP port are different numbers. Configure TLS according to the provider's supported setup; do not casually disable certificate verification to conceal a connection problem.

The Dockerfile uses Maven with Java 17 to build queue-system.war, then copies it into Tomcat as ROOT.war. It removes default webapps and disables the shutdown listener. Docker context includes pom.xml and src only; local runtime/configuration is not copied. Tomcat listens for HTTP on 8080.

After Deploy reports Live, open /api/services and confirm the three service records. A 404 at / is expected because there is no backend homepage. Live means the web process is up, not that database queries or login are necessarily correct. Check a real API and authentication flow.

26. Netlify deployment step by step

Create a project by importing the same GitHub repository and main branch. For student set base directory QueueSystemFrontend. For staff create a second project with base directory frontend. Each base contains netlify.toml. Build command: `npm run build && node ../scripts/netlify-redirects.mjs`. Publish directory: `dist`.

Set `QUEUE_BACKEND_URL=https://queuesystem-1.onrender.com` on both Netlify projects. Use the origin only, without `/api/services`. No database password belongs in Netlify frontend environment variables. The build script requires an HTTPS origin and generates `dist/_redirects` with the `/api` proxy first, then the SPA fallback.

An `/api/*` rewrite sends requests to the Render backend while the browser uses the Netlify origin. The final `/* /index.html` fallback allows staff BrowserRouter URLs to work on refresh. Student HashRouter handles the hash locally. The API rule must come first so API requests are not returned HTML instead of JSON.

To remove the bottom-right badge for all visitors: Project configuration -> General -> Powered by Netlify badge -> Configure -> Off -> Save. This does not require a redeploy. Merely hiding the badge as a visitor applies only to that browser, not everyone.

27. Deployment incident log: what went wrong and why

Incident 1: Workbench host was entered as `MYSQL_PUBLIC_URL` literally and port became NaN. Resolution: reveal the variable's value and split the real address into host, port, username and password. Lesson: a configuration variable name is not its value.

Incident 2: Render defaulted to a paid \$7/month compute option. Resolution: select \$0/month Free and verify the displayed selection before deploying. Lesson: budget and free-tier limits must be explained before provisioning.

Incident 3: root URL returned Tomcat 404. Resolution: check `/api/services`; the backend hosts APIs, not the React homepage. Lesson: a root-page 404 is not by itself proof of failed deployment.

Incident 4: logs showed Invalid shutdown command [`HEAD / HTTP/1.1`]. HTTP probes were reaching a shutdown listener. Resolution: disable Tomcat's shutdown port in the Docker image, leaving HTTP on 8080. Lesson: distinguish a management listener from the HTTP connector.

Incident 5: API returned `success:true` with an empty array while Workbench had three services. ServiceDAO swallowed failures. Resolution: throw a service-load exception and return HTTP 503 with a safe generic message. Console logging was then changed to stderr because context logging did not surface in the host console.

Incident 6: `CommunicationsException SQLState=08S01` showed a communications failure. Comparing settings found Render JDBC used database port 8080 while the working Workbench connection used public MySQL port 29081. Resolution: fix the JDBC database port while retaining `PORT=8080` for Tomcat. The API then returned all three services. This specific proxy port can change if infrastructure is recreated; always use the current provider value.

28. Hosting costs and demonstration reliability

Render Free web services spin down after fifteen minutes without inbound traffic; the first request after that can be slow. Free hours are shared at workspace level. A successful demo should begin by opening the backend and websites early, not waiting until a judge clicks Login. Do not use artificial traffic merely to evade provider limits.

Railway's trial grants five dollars of credit for up to thirty days; after that, its Free plan provides one dollar of monthly credit. A continuously running MySQL server is not guaranteed to fit that allowance. Netlify free plans also have usage limits. Check account billing/usage rather than promising perpetual zero-cost hosting. No paid upgrade is authorized by this manual.

References checked for this edition: <https://render.com/docs/free> <https://render.com/docs/docker>
<https://render.com/docs/web-services> <https://docs.railway.com/pricing/free-trial>
<https://docs.railway.com/databases/mysql> <https://docs.netlify.com/manage/routing/redirects/rewrites-proxies/>
<https://docs.netlify.com/manage/projects/powered-by-netlify-badge/>
<https://tomcat.apache.org/tomcat-10.1-doc/config/server.html>

29. Testing evidence and what remains

Local validation previously passed 63 daily reservation checks, analytics/processing-window checks, password policy checks and 29 reservation API checks. Frontend builds and Maven packaging passed. These are standalone scripts and manual checks; do not say Maven automatically executed all of them.

Cloud smoke checks during this documentation run confirmed both frontend homepages, backend catalogue and both /api proxies return successfully. All three staff accounts logged in with the expected service/counter IDs, and profile, queue and dashboard-summary requests preserved sessions. A browser registration for a labelled deployment test student reached the dashboard. Real-time prebooking preview returned 10:00 with `testClock=false`; a reservation received token 001 and was immediately cancelled successfully. The labelled test account and cancelled ticket remain as audit evidence. Full notes are in docs/CLOUD-VALIDATION.md.

No full cloud concurrency/absence/lunch/midnight suite has been run against production in this edition. It would require controlled fixtures and operating-window timing. Local tests exercise several of these rules using isolated fixed-time fixtures. Functional correctness and UI appearance are different verification dimensions.

30. Known limitations to acknowledge confidently

No administrator portal. No true offline/cloud bidirectional synchronization. No SMS/OTP/email verification or password recovery. Telephone is a device tel: link. Queue updates are periodic polling. ETA does not yet model a running job's remaining duration continuously. Initial values remain until completed positive samples exist.

Future carried WAITING tickets are visible, but cancellation currently requires `visit_date=today`; the UI should not promise future-ticket cancellation. Cross-midnight CALLED/SERVING handling needs refinement. Holidays/weekends are not excluded. Closure admission uses projected start, not guaranteed completion before closing. Multi-counter prediction assumes comparable rates.

Some dashboard analytics use elapsed timestamp differences and can include pre-opening wait; they are distinct from the scheduled-window ETA samples. Legacy slot-booking endpoints, JSP pages and tests remain and should be inventoried or retired before a hardened production release. Authentication error handling and manual JSON in older modules need further review. Database timezones must remain aligned with campus logic.

Present limitations as engineering scope: explain the existing boundary, why it matters and the next change needed. Do not invent features to fill a judge's question.

31. Viva question bank: architecture and frontend

Q: Why two frontends? A: Student and staff workflows differ, but both use the same backend and database. Separate deployments do not create separate queues.

Q: Why React? A: Component/state-based rendering suits polling queue data and reusable ticket/service UI. It is a design choice, not a requirement for the algorithm.

Q: What is JSX? A: Syntax describing element trees that a build tool compiles to JavaScript; it is not Java or JSP.

Q: What happens after clicking Reserve? A: A POST reaches the Java adapter, session identity is checked, the service transaction inserts related rows, and refreshed frontend state renders the ticket.

Q: Why useState? A: It stores values whose changes should render the UI, such as queue data or a busy flag.

Q: Why useEffect cleanup? A: It stops intervals/side effects after unmount or dependency changes.

Q: Why credentials:include? A: Session cookies must accompany requests through the proxy.

Q: What is HashRouter? A: It interprets the URL fragment locally; BrowserRouter uses actual paths that need a hosting fallback.

Q: Why not create a booking on service-card click? A: Exploration should show a preview; confirmation is the explicit write boundary.

Q: Is the UI real time? A: It refreshes by polling, not push-based WebSockets.

Q: Is the phone number verified? A: Syntax is validated and normalized; ownership is not verified.

Q: Can a frontend hidden button secure an action? A: No; the server must verify identity, assignment and state.

32. Viva question bank: Java, HTTP and deployment

Q: What is a servlet? A: A container-managed Java HTTP request handler with lifecycle methods such as `init` and `doGet/doPost`.

Q: What does `@WebServlet` do? A: It declares the URL mapping used by Tomcat.

Q: GET versus POST? A: GET retrieves representations; POST submits actions/data. Booking creation is POST.

Q: What is a session? A: Server-maintained login state associated with a cookie identifier.

Q: Why a WAR? A: This servlet application is packaged for deployment into Tomcat, unlike an executable application JAR.

Q: What does Maven do? A: Dependency resolution, compilation and packaging according to `pom.xml` and plugin configuration.

Q: What is JDBC? A: Java's database API; Connector/J translates its calls into MySQL protocol operations.

Q: Why `PreparedStatement`? A: Bind user values separately from SQL syntax and use typed parameters.

Q: Why `try-with-resources`? A: Deterministic resource closure even when an exception interrupts execution.

Q: Why return 503 for catalogue failure? A: Database inability is a service availability failure, not a successful empty result.

Q: Why root 404 on Render? A: No backend homepage exists; React is hosted on Netlify.

Q: Why two ports? A: HTTP 8080 belongs to Tomcat; Railway's public port belongs to MySQL transport.

Q: What does Docker add? A: A repeatable build/runtime image with Java and Tomcat configured for hosting.

Q: Does `.env` automatically configure Java? A: No; our utility reads process environment or an explicitly selected local properties file.

Q: Why can a teammate push but Render cannot see the repo? A: GitHub app authorization is separate from human collaborator access.

33. Viva question bank: SQL and queue logic

Q: Primary versus foreign key? A: A primary key identifies a row; a foreign key enforces a reference to another table.

Q: Is token the primary key? A: No; queue_id is primary key, and tokens are service-local labels.

Q: Is FIFO always preserved? A: Initial reservations follow FIFO; the documented first-miss penalty and carryover priority adjust waiting order intentionally.

Q: Why a transaction? A: Queue, reservation and event inserts must succeed or fail together.

Q: What race does a lock solve? A: Two clients computing the same next token/order before either commits.

Q: Does MAX+1 alone guarantee unique tokens? A: No; it relies on consistent service locking and would benefit from database constraints as additional defence.

Q: WHERE versus HAVING? A: WHERE filters input rows, HAVING filters aggregate groups.

Q: INNER versus LEFT JOIN? A: INNER requires matches; LEFT keeps the left row when an optional right-side row is absent.

Q: DELETE versus DROP? A: DELETE removes rows; DROP removes the object itself.

Q: What happens after two absences? A: The ticket ends as NO_SHOW; the account is not permanently banned.

Q: What happens at 15:00 to waiting students? A: They roll to the next calendar day's 10 AM queue without a miss penalty.

Q: How do prebookings affect average duration? A: Booking time is excluded; duration starts at actual service start and only scheduled processing overlap is counted.

Q: Why keep historical data? A: It supports measured averages, analytics and an event trail.

Q: Does empty legacy bookings mean nothing was booked? A: No; the current flow uses queue and queue_reservations.

Q: How does Workbench show online changes? A: It connects to the same cloud server; refresh fetches rows. It is not a synchronized local copy.

Q: What if an average jumps after a long task? A: Completed samples update the recent mean; estimates change on subsequent refreshes.

Q: Why position ten can mean twenty-seven minutes at three minutes each? A: Nine people are ahead; position and ahead differ by one.

Q: Is ETA guaranteed? A: No; staffing, task variation, absence and model limitations can change it.

34. Submission checklist and study allocation

Each teammate should understand the overall flow, then own a module walkthrough: student UI, staff UI, queue/service backend, or database/analytics/deployment. Never limit preparation to the branch name; main contains the integrated version, and judges may select any file.

Before presentation: pull main; confirm both URLs and cloud connection; verify staff assignments; open the backend early; select a safe test student; keep a scratch SQL schema ready; check the actual campus time. If outside open hours, demonstrate prebooking and explain the schedule rather than silently changing production rules.

Show no secret .env or password value on a projector. Bring these PDFs, the repository, a known successful API response, local test evidence and a backup set of screenshots. A backup screenshot is evidence of an earlier run, not a claim that the live service currently works.

35. File inventory and responsibility map

This inventory includes retained legacy files and standalone diagnostics. A file being present is not proof it is executed by the current React flow. Follow the imports, servlet mappings and documented current path.

QueueSystemFrontend/index.html

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

QueueSystemFrontend/netlify.toml

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

QueueSystemFrontend/scripts/sync-tomcat.mjs

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

QueueSystemFrontend/src/App.css

Stylesheet controls appearance and responsiveness, not authorization or database state. The student currently imports original-identity.css; retained themes may be legacy.

QueueSystemFrontend/src/App.jsx

Current student component tree, landing/login and authenticated routes. Study BookingForm, Ticket, VirtualQueue and Experience.

QueueSystemFrontend/src/components/BottomNav.jsx

Reusable UI presentation component; check its imports/callers to establish whether the current App uses it.

QueueSystemFrontend/src/components/Layout.jsx

Reusable UI presentation component; check its imports/callers to establish whether the current App uses it.

QueueSystemFrontend/src/components/Sidebar.jsx

Reusable UI presentation component; check its imports/callers to establish whether the current App uses it.

QueueSystemFrontend/src/index.css

Stylesheet controls appearance and responsiveness, not authorization or database state. The student currently imports original-identity.css; retained themes may be legacy.

QueueSystemFrontend/src/main.jsx

Browser entry point: mounts the React app into the root element and imports styles. React StrictMode helps reveal development side-effect problems.

QueueSystemFrontend/src/pages/Bookings.jsx

Older student page retained alongside the current App.jsx component implementation. Check imports before treating it as an active production screen.

QueueSystemFrontend/src/pages/Dashboard.jsx

Older student page retained alongside the current App.jsx component implementation. Check imports before treating it as an active production screen.

QueueSystemFrontend/src/pages/Login.jsx

Older student page retained alongside the current App.jsx component implementation. Check imports before treating it as an active production screen.

QueueSystemFrontend/src/pages/Queue.jsx

Older student page retained alongside the current App.jsx component implementation. Check imports before treating it as an active production screen.

QueueSystemFrontend/src/pages/Settings.jsx

Older student page retained alongside the current App.jsx component implementation. Check imports before treating it as an active production screen.

QueueSystemFrontend/src/queue-layout.js

Pure animation layout helper. Groups large queues visually; it does not calculate authoritative server queue order.

QueueSystemFrontend/src/student-api.js

Student fetch wrapper and shared backend hook. Includes credentials, error handling, polling and focused-tab refresh.

QueueSystemFrontend/src/styles/original-identity.css

Stylesheet controls appearance and responsiveness, not authorization or database state. The student currently imports original-identity.css; retained themes may be legacy.

QueueSystemFrontend/src/styles/premium-theme.css

Stylesheet controls appearance and responsiveness, not authorization or database state. The student currently imports original-identity.css; retained themes may be legacy.

QueueSystemFrontend/vite.config.js

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

database/migrations/001_daily_reservations.sql

Database structure/migration/seed script. Read constraints, dependencies, repeatability and schema selection before executing.

database/migrations/002_processing_duration.sql

Database structure/migration/seed script. Read constraints, dependencies, repeatability and schema selection before executing.

database/schema.sql

Database structure/migration/seed script. Read constraints, dependencies, repeatability and schema selection before executing.

database/seed_services.sql

Database structure/migration/seed script. Read constraints, dependencies, repeatability and schema selection before executing.

frontend/index.html

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

frontend/netlify.toml

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

frontend/src/App.jsx

Staff BrowserRouter, restored login session, navigation and dashboard composition.

frontend/src/components/ActivityFeed.jsx

Recent event/status list presentation. Explain display fields separately from database state transitions.

frontend/src/components/QueueTable.jsx

Queue rows and state/ownership-based action buttons. Server checks remain authoritative.

frontend/src/components/StatCard.jsx

Reusable summary card presentation; values come from the dashboard API.

frontend/src/components/StudentPhone.jsx

Validated telephone display and tel link. Device-mediated calling; no server telephony or queue mutation.

frontend/src/main.jsx

Browser entry point: mounts the React app into the root element and imports styles. React StrictMode helps reveal development side-effect problems.

frontend/src/pages/StaffDashboard.jsx

Staff dashboard orchestration: polls endpoints, derives assigned current service, runs actions and refreshes UI state.

frontend/src/pages/StaffLogin.jsx

Staff login form and error/busy handling. Uses the shared API adapter; it does not read MySQL directly.

frontend/src/services/api.js

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

frontend/src/style.css

Stylesheet controls appearance and responsiveness, not authorization or database state. The student currently imports original-identity.css; retained themes may be legacy.

frontend/vite.config.js

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

pom.xml

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

run_server.ps1

Local PowerShell setup/start/configuration utility. Read path resolution and ignored config handling before execution.

scripts/SeedDemoBookings.java

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

scripts/configure-db.ps1

Local PowerShell setup/start/configuration utility. Read path resolution and ignored config handling before execution.

scripts/create-cloud-staff-seed.py

Local staff provisioning generator using standard-library PBKDF2. Generated SQL stays in ignored .runtime; plaintext input is not committed.

scripts/initialize-db.ps1

Local PowerShell setup/start/configuration utility. Read path resolution and ignored config handling before execution.

scripts/netlify-redirects.mjs

Build-time HTTPS backend-origin validation and API-first rewrite generation, followed by SPA route fallback.

scripts/set-test-clock.ps1

Local PowerShell setup/start/configuration utility. Read path resolution and ignored config handling before execution.

scripts/staff-preview.mjs

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

scripts/start-backend.ps1

Local PowerShell setup/start/configuration utility. Read path resolution and ignored config handling before execution.

scripts/tests/AnalyticsChecks.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

scripts/tests/DailyReservationChecks.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

scripts/tests/IntegrationFixtures.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

scripts/tests/PasswordPolicyChecks.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

scripts/tests/integration-test.mjs

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

scripts/tests/legacy/FixedSlotConcurrencyChecks.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

scripts/tests/legacy/fixed-slot-integration.mjs

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

scripts/tests/queue-layout.test.mjs

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

scripts/tests/reservation-api-test.mjs

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

src/main/java/com/queue/controller/AnalyticsServlet.java

Staff analytics request adapter. Explains how authenticated service-specific statistics are exposed.

src/main/java/com/queue/controller/BookingServlet.java

Retained appointment HTTP surface. Current booking confirmation uses StudentPortalServlet and ReservationService.

src/main/java/com/queue/controller/QueueServlet.java

Queue action HTTP adapter. Authenticates staff and delegates actions into ReservationService; retained routes require care.

src/main/java/com/queue/controller/ServiceServlet.java

Public catalogue adapter. Returns configured services; catches DAO failure as HTTP 503 and writes safe diagnostics to stderr.

src/main/java/com/queue/controller/StaffServlet.java

Staff HTTP adapter for login, profile, service queue, counter availability, summary and activity. Study service scoping and remaining legacy branches.

src/main/java/com/queue/controller/StudentPortalServlet.java

Current student HTTP adapter. Routes login/register/session/overview/preview/reserve/cancel; obtains identity from the server session.

src/main/java/com/queue/controller/StudentServlet.java

Older student HTTP surface. Compare with current StudentPortalServlet before claiming this is the active UI route.

src/main/java/com/queue/controller/TestServlet.java

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

src/main/java/com/queue/dao/CounterDAO.java

Counter lookup and state/storage operations. A counter belongs to one service.

src/main/java/com/queue/dao/CurrentTokenTest.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

src/main/java/com/queue/dao/QueueDAO.java

Retained queue read/DAO operations and student ticket projections. Current transactional writes are mainly in ReservationService.

src/main/java/com/queue/dao/QueueDAOTest.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

src/main/java/com/queue/dao/QueuePositionTest.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

src/main/java/com/queue/dao/QueueStatusTest.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

src/main/java/com/queue/dao/ServiceDAO.java

Catalogue SELECT and model mapping. Now throws on failure so the servlet does not fabricate an empty successful result.

src/main/java/com/queue/dao/ServiceDAOTest.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

src/main/java/com/queue/dao/StaffDAO.java

Staff SQL login/profile queries and password verification. Assigned service and counter originate from stored account data.

src/main/java/com/queue/dao/StudentDAO.java

Student SQL storage/login operations, password verification and hash migration. Read PreparedStatement bindings and generated IDs.

src/main/java/com/queue/dao/StudentDAOTest.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

src/main/java/com/queue/dao/StudentLoginTest.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

src/main/java/com/queue/filter/StudentOwnershipFilter.java

Request filter protecting older endpoints using authenticated ownership/service checks. Not a substitute for current service validation.

src/main/java/com/queue/model/Booking.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

src/main/java/com/queue/model/Counter.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

src/main/java/com/queue/model/Queue.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

src/main/java/com/queue/model/Service.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

src/main/java/com/queue/model/Staff.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

src/main/java/com/queue/model/Student.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

src/main/java/com/queue/service/AnalyticsService.java

SQL aggregation for dashboard reporting. Some elapsed-time metrics are different from ETA scheduled-window processing samples.

src/main/java/com/queue/service/BookingService.java

Retained legacy fixed-slot booking logic. Do not present it as the current same-day reservation algorithm.

src/main/java/com/queue/service/ClosingTimeListener.java

Servlet lifecycle background reconciliation. A scheduled worker checks closing every ten seconds and is stopped on context destruction.

src/main/java/com/queue/service/NoShowService.java

Retained older appointment no-show handling. Current queue absence rules are in ReservationService.

src/main/java/com/queue/service/ReservationService.java

Core transaction coordinator. Study preview, reserve, operate, callNext, cancellation, availability and closing rollover. Follow the service lock and commit/rollback boundary.

src/main/java/com/queue/service/ServiceHours.java

Campus-time policy and processing-window arithmetic. Study real/test time, open boundaries, next opening, lunch projection and interval overlap.

src/main/java/com/queue/service/ServiceService.java

Thin catalogue adapter that delegates to ServiceDAO.

src/main/java/com/queue/service/StaffService.java

Staff business adapter. Resolves login/profile and validates account/counter service assignments.

src/main/java/com/queue/service/StudentService.java

Student business adapter around registration/login DAO operations and server input validation.

src/main/java/com/queue/service/WaitingTimeService.java

Recent completed processing samples and estimate helpers. Distinguish measured average from configuration fallback and preview rounding.

src/main/java/com/queue/util/DBConnection.java

Connection factory. Environment variables override an explicitly loaded local properties file. DriverManager opens the MySQL connection.

src/main/java/com/queue/util/DBConnectionExample.java

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

src/main/java/com/queue/util/DBTest.java

Standalone test or diagnostic program. Read fixture setup, assertions and cleanup; do not assume Maven package executes this file.

src/main/java/com/queue/util/DatabaseSetup.java

Explicit schema/migration runner for local provisioning. A deployment does not automatically execute these scripts merely by pushing Git.

src/main/java/com/queue/util/Json.java

Recursive JSON serializer for null, number, boolean, maps, lists and escaped string values.

src/main/java/com/queue/util/Passwords.java

Random-salt PBKDF2 hash/verification utility. Salt and derived key are Base64 encoded, not encrypted. Legacy migration support remains.

src/main/java/com/queue/util/StudentValidation.java

Registration syntax and length rules. Validates name, email, Indian phone and user-requested password policy; normalizes input.

src/main/java/com/queue/util/TransactionLocks.java

Database parent-row locks. SELECT FOR UPDATE serializes operations per service; locks depend on the surrounding transaction.

src/main/webapp/booking/analytics.jsp

Retained server-rendered JSP page. Tomcat compiles JSP into a servlet; the current primary student/staff dashboards use React instead.

src/main/webapp/booking/booking-status.jsp

Retained server-rendered JSP page. Tomcat compiles JSP into a servlet; the current primary student/staff dashboards use React instead.

src/main/webapp/booking/pre-booking.jsp

Retained server-rendered JSP page. Tomcat compiles JSP into a servlet; the current primary student/staff dashboards use React instead.

src/main/webapp/css/staff.css

Stylesheet controls appearance and responsiveness, not authorization or database state. The student currently imports original-identity.css; retained themes may be legacy.

src/main/webapp/js/smart.js

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

src/main/webapp/js/staff.js

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

src/main/webapp/staff/dashboard.jsp

Retained server-rendered JSP page. Tomcat compiles JSP into a servlet; the current primary student/staff dashboards use React instead.

src/main/webapp/staff/login.js

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

src/main/webapp/staff/login.jsp

Retained server-rendered JSP page. Tomcat compiles JSP into a servlet; the current primary student/staff dashboards use React instead.

src/main/webapp/student/bookings.html

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

src/main/webapp/student/dashboard.html

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

src/main/webapp/student/login.html

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

src/main/webapp/student/queue.html

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

src/main/webapp/student/settings.html

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

36. Numbered source and guided line explanations

Exact production-source snapshot, with source line numbers. Long lines wrap visually but retain their original number. Source punctuation and non-ASCII symbols are escaped for print readability. Explanations highlight meaningful lines; the source remains authoritative.

QueueSystemFrontend/src/App.jsx

Current student component tree, landing/login and authenticated routes. Study BookingForm, Ticket, VirtualQueue and Experience.

Read these lines first

Line 1: Synchronizes UI with polling or another side effect. Explain dependencies and timer cleanup.

Line 185: Synchronizes UI with polling or another side effect. Explain dependencies and timer cleanup.

Line 337: Declares component state and a setter; changes cause React to render the relevant UI.

Line 352: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 354: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 359: Synchronizes UI with polling or another side effect. Explain dependencies and timer cleanup.

Line 385: Declares component state and a setter; changes cause React to render the relevant UI.

Line 386: Declares component state and a setter; changes cause React to render the relevant UI.

Numbered source

```

1 | import { useEffect, useLayoutEffect, useRef, useState } from "react";
2 | import {
3 |   HashRouter,
4 |   NavLink,
5 |   Link,
6 |   Route,
7 |   Routes,
8 |   Navigate,
9 |   useLocation,
10 |  useNavigate,
11 | } from "react-router-dom";
12 | import {
13 |   ArrowUpRight,
14 |   ArrowRight,
15 |   MapPin,
16 |   Plus,
17 |   Sun,
18 |   Moon,
19 |   X,
20 |   Check,
21 |   LayoutGrid,
22 |   Ticket as TicketIcon,
23 |   CalendarDays,
24 |   Settings as SettingsIcon,
25 |   Logout,
26 |   Coins,
27 |   FileText,
28 |   Library,
29 |   Users,
30 | } from "lucide-react";
31 | import gsap from "gsap";
32 | import { studentRequest, useStudentBackend } from "../student-api.js";
33 | import { queueLayout } from "../queue-layout.js";
34 | import "../index.css";
35 | import "../styles/original-identity.css";
36 |
37 | const nav = [
38 |   ["/dashboard", LayoutGrid, "Overview"],
39 |   ["/queue", TicketIcon, "My queue"],
40 |   ["/bookings", CalendarDays, "Bookings"],
41 |   ["/settings", SettingsIcon, "Settings"],
42 | ];
43 | function Brand() {
44 |   return <Link className="brand cq-brand" to="/" aria-label="Campus Queue home"><span className="cq-
    | wordmark"><span>Campus</span><strong>Queue</strong></span></Link>;
45 | }
46 | function Theme({ dark, toggle }) {
47 |   return (
48 |     <button
49 |       className="icon-button"
50 |       onClick={toggle}
51 |       aria-label={`Switch to ${dark ? "light" : "dark"} theme`}
52 |     >
53 |       {dark ? <Sun size={19} /> : <Moon size={19} />}
54 |     </button>
55 |   );
56 | }
57 | function PageHeading({ eyebrow, title, description, children }) {
58 |   return (
59 |     <header className="page-heading">
60 |       <div>
61 |         <p className="eyebrow">{eyebrow}</p>
62 |         <h1>{title}</h1>
63 |         <p className="muted">{description}</p>
64 |       </div>
65 |       {children}
66 |     </header>
67 |   );
68 | }
69 | function Ticket({ ticket, compact = false }) {
70 |   return (
71 |     <article className={`ticket ${compact ? "compact-ticket" : ""}`}>
72 |       <div className="ticket-top">
73 |         <span className="eyebrow">YOUR CAMPUS PASS</span>
74 |         <TicketIcon size={22} />
75 |       </div>
76 |       <div className="ticket-body">
77 |         <p>{ticket.serviceName}</p>
78 |         <div className="token">{ticket.token}</div>
79 |         <span className="ticket-status">
80 |           <span className="status-dot" />{" "}
81 |           {ticket.status === "SERVING"
82 |             ? "Your service is in progress."
83 |             : ticket.status === "CALLED"
84 |             ? "Your turn. Please go to your assigned counter."
85 |             : ticket.rescheduled ? `Rescheduled for ${ticket.date}. Counters open at 10 AM.` :
            | "Booking confirmed. Please wait for your turn."}
86 |         </span>
87 |       </div>
88 |       <div className="ticket-tear" />
89 |       <div className="ticket-bottom">
90 |         <div>
91 |           <span className="eyebrow">DESTINATION</span>
92 |           <strong>{ticket.place}</strong>
93 |         </div>
94 |         <div>
95 |           <span className="eyebrow">COUNTER</span>
96 |           <strong>{ticket.counterName || "Assigned when called"}</strong>
97 |         </div>
98 |       </div>
99 |       <div className="ticket-footer">
100 |         <div className="barcode" aria-hidden="true" />
101 |         <span className="mono">{ticket.token} / STUDENT COPY</span>
102 |       </div>
103 |     </article>
104 |   );
105 | }
106 | function VirtualQueue({ ticket }) {
107 |   const scene = useRef(null);
108 |   const hasPosition = useRef(false);
109 |   const layout = queueLayout(ticket?.ahead ?? 4);
110 |   const { ahead, grouped, hidden, people, you: position } = layout;
111 |   useLayoutEffect(() => {
112 |     const marker = scene.current?.querySelector(".virtual-you");

```

```

113 |     if (!marker) return;
114 |     if (!hasPosition.current) {
115 |         gsap.set(marker, { x: position });
116 |         hasPosition.current = true;
117 |         return;
118 |     }
119 |     const tween = gsap.to(marker, {
120 |         x: position,
121 |         duration: matchMedia("(prefers-reduced-motion: reduce)").matches ? 0 : 0.7,
122 |         ease: "power3.inOut",
123 |     });
124 |     return () => tween.kill();
125 | }, [position]);
126 | return (
127 |     <div className="virtual-queue" ref={scene}>
128 |         <div className="virtual-caption">
129 |             <span className="eyebrow">A LITTLE CLOSER, EVERY TURN</span>
130 |
131 |             </div>
132 |             <svg viewBox="0 0 550 225" role="img" aria-label={`Illustrated virtual queue: ${ahead} people
133 | ahead, between you and the service desk on the right. ${ticket ? "Position follows your ticket." :
134 | "Example queue."}`>
135 |                 <ellipse cx="270" cy="176" rx="238" ry="21" fill="var(--soft)" />
136 |                 <path className="virtual-flow" d="M40 175 H398" fill="none" stroke="var(--orange)"
137 | strokeWidth="2" strokeDasharray="3 10" opacity=".45" />
138 |                 <path d="M381 169 L389 175 L381 181" fill="none" stroke="var(--orange)" strokeWidth="2" />
139 |                 {grouped && <g className="virtual-group">
140 |                     <path d="M97 167 L7 -9 M97 176 L7 -9 M192 167 L7 -9 M192 176 L7 -9" stroke="var(--orange)"
141 | strokeWidth="2" fill="none" />
142 |                     <rect x="111" y="78" width="75" height="79" rx="18" fill="var(--soft)" stroke="var(--line)"
143 | strokeWidth="2" />
144 |                     <g fill="var(--ink)" opacity=".45"><circle cx="137" cy="96" r="5"/><circle cx="148" cy="92"
145 | r="5"/><circle cx="159" cy="96" r="5"/></g>
146 |                     <text x="148" y="123" textAnchor="middle" fill="var(--orange)" fontSize={hidden > 999 ? 18 :
147 | 23} fontWeight="700">+{hidden}</text>
148 |                     <text x="148" y="141" textAnchor="middle" fill="var(--muted)" fontSize="9">MORE AHEAD</text>
149 |                 </g>
150 |                 {people.map((x, i) => (
151 |                     <g key={i} transform={`translate(${x},0)`} className="virtual-person" style={{ "--motion-
152 | delay": `${i} * -0.4}s` }}>
153 |                         <ellipse cx="0" cy="167" rx="19" ry="5" fill="var(--ink)" opacity=".07" />
154 |                         <g className="virtual-body">
155 |                             <circle cy="91" r="12" fill="var(--ink)" opacity=".55" />
156 |                             <path d="M-15 119 Q-15 108 0 108 Q15 108 15 119 L12 141 H-12 Z" fill={i % 2 ? "#b38a42" :
157 | "#8073b8"} opacity=".65" />
158 |                             <path d="M-8 140 L-11 161 M8 140 L11 161" fill="none" stroke="var(--ink)" strokeWidth="6"
159 | strokeLinecap="round" opacity=".55" />
160 |                             <path d="M12 116 L23 127" fill="none" stroke="var(--ink)" strokeWidth="5"
161 | strokeLinecap="round" opacity=".5" />
162 |                         </g>
163 |                     </g>
164 |                 )))
165 |                 <g className="virtual-you">
166 |                     <ellipse cy="167" rx="21" ry="6" fill="var(--orange)" opacity=".18" />
167 |                     <g className="virtual-body">
168 |                         <rect x="-26" y="42" width="52" height="23" rx="11" fill="var(--orange)" />
169 |                         <text y="57" textAnchor="middle" fill="white" fontSize="10" fontWeight="700">YOU</text>
170 |                         <circle cy="91" r="13" fill="var(--ink)" />
171 |                         <path d="M-17 119 Q-17 107 0 107 Q17 107 17 119 L13 141 H-13 Z" fill="var(--orange)" />
172 |                         <path d="M-8 141 L-11 161 M8 141 L11 161" fill="none" stroke="var(--ink)" strokeWidth="7"
173 | strokeLinecap="round" />
174 |                         <path d="M14 118 L26 129" fill="none" stroke="var(--ink)" strokeWidth="5"
175 | strokeLinecap="round" />
176 |                         <rect x="20" y="118" width="16" height="21" rx="3" transform="rotate(-12 28 129)"
177 | fill="var(--surface)" stroke="var(--orange)" strokeWidth="2" />
178 |                         <path d="M24 124 H31 M24 128 H30" stroke="var(--orange)" strokeWidth="1.5" />
179 |                     </g>
180 |                 </g>
181 |                 <g>
182 |                     <rect x="422" y="43" width="79" height="31" rx="8" fill="var(--ink)" />
183 |                     <text x="461" y="63" textAnchor="middle" fill="var(--surface)" fontSize="10"

```

```

    | fontWeight="600">SERVICE DESK</text>
169 |     <circle cx="462" cy="96" r="11" fill="var(--ink)" opacity=".7" />
170 |     <path d="M447 122 Q447 109 462 109 Q477 109 477 122" fill="var(--orange)" opacity=".6" />
171 |     <rect x="411" y="124" width="102" height="45" rx="7" fill="var(--surface)"
    | stroke="var(--line)" strokeWidth="2" />
172 |     <path d="M405 124 H519" stroke="var(--orange)" strokeWidth="5" strokeLinecap="round" />
173 |     <rect x="424" y="101" width="25" height="20" rx="3" fill="var(--ink)" />
174 |     <path d="M434 120 V124" stroke="var(--ink)" strokeWidth="3" />
175 |     <circle className="virtual-beacon" cx="496" cy="144" r="4" fill="var(--orange)" />
176 |   </g>
177 |   <text x="40" y="211" fill="var(--muted)" fontSize="10" fontFamily="monospace">{ticket ?
    | `${ahead} AHEAD / YOUR PLACE IN LINE` : "ILLUSTRATED QUEUE / TAKE YOUR TIME"}</text>
178 | </svg>
179 | <p className="virtual-note" aria-live="polite">{ticket ? "Keep your student ID ready. Go to your
    | assigned counter when your token is called." : "Reserve a place, then follow your ticket to check your
    | turn."}</p>
180 | </div>
181 | );
182 | }
183 | function Overview({ ticket, profile, join, catalog, busy }) {
184 |   const location = useLocation();
185 |   useEffect(() => {
186 |     if (location.state?.scrollToServices) {
187 |       const frame = requestAnimationFrame(() => document.getElementById("services)?.scrollIntoView({
    | behavior: matchMedia("(prefers-reduced-motion: reduce)").matches ? "auto" : "smooth", block: "start"
    | }));
188 |       return () => cancelAnimationFrame(frame);
189 |     }
190 |   }, [location.key, location.state, catalog]);
191 |   return (
192 |     <>
193 |       <PageHeading
194 |         eyebrow="YOUR DAY, ON YOUR TERMS"
195 |         title={
196 |           <>
197 |             Campus services.{ " "}
198 |             <span className="serif">Choose your service.</span>
199 |           </>
200 |         }
201 |         description="Book today, review your estimated time, and track your queue ticket."
202 |       />
203 |       <div className="overview-grid">
204 |         <section className="feature-panel">
205 |           <div className="section-label">
206 |             <span className="eyebrow">
207 |               {ticket ? "YOUR NEXT STOP" : "CAMPUS, WITHOUT THE QUEUES"}
208 |             </span>
209 |             <span className="tag">
210 |               {ticket ? "Active ticket" : "Student edition"}
211 |             </span>
212 |           </div>
213 |           <h2>
214 |             {ticket ? (
215 |               <>
216 |                 Your place is saved.
217 |                 <br />
218 |                 <span className="serif">Your time is yours.</span>
219 |               </>
220 |             ) : (
221 |               <>
222 |                 Less standing around.
223 |                 <br />
224 |                 <span className="serif">More getting around.</span>
225 |               </>
226 |             )}
227 |           </h2>
228 |           <p>
229 |             {ticket
230 |               ? `${ticket.ahead} people ahead \xb7 ${ticket.wait === null ? "Estimate unavailable" :
    | `About ${ticket.wait} minutes to go.`}`
231 |               : "Pick a service below, take a digital ticket, and spend the wait wherever you like."}

```

```

232 |         </p>
233 |         {ticket ? (
234 |             <Link className="button primary" to="/queue">
235 |                 View my ticket <ArrowUpRight size={19} />
236 |             </Link>
237 |         ) : (
238 |             <button
239 |                 className="button primary"
240 |
241 |                 onClick={() =>
242 |                     document.getElementById("services").scrollIntoView({
243 |                         behavior: matchMedia("(prefers-reduced-motion: reduce)")
244 |                             .matches
245 |                             ? "instant"
246 |                             : "smooth",
247 |                     })
248 |                 }
249 |             >
250 |                 Find your next stop <ArrowRight size={19} />
251 |             </button>
252 |         )}
253 |     <VirtualQueue ticket={ticket} />
254 | </section>
255 | <aside className="day-panel">
256 |     <p className="eyebrow">BEFORE YOUR TURN</p>
257 |     <div className="time-art">
258 |         Stay<br /><span className="serif">ready.</span>
259 |         <span className="asterisk">\u2733</span>
260 |     </div>
261 |     <p>
262 |         Check your queue position.
263 |         <br />
264 |         Keep your student ID handy.
265 |         <br />
266 |         Head to the desk when called.
267 |     </p>
268 |     <div className="day-bottom">
269 |         <span className="mono">CAMPUSQUEUE / 01</span>
270 |         <ArrowUpRight size={28} />
271 |     </div>
272 | </div>
273 | <section id="services" className="services-section">
274 |     <div className="section-heading">
275 |         <div>
276 |             <p className="eyebrow">CHOOSE YOUR DESTINATION</p>
277 |             <h2>What brings you here?</h2>
278 |         </div>
279 |         <span className="muted small">
280 |             Average processing time
281 |         </span>
282 |     </div>
283 |     <div className="service-list">
284 |         {!catalog.length && <p className="muted">No campus services have been configured yet. Your
285 | campus team will add them to the database.</p>}
286 |         {catalog.map((s) => {
287 |             const ServiceIcon =
288 |             {
289 |                 "Fee Payment": Coins,
290 |                 Registration: FileText,
291 |                 "Library Services": Library,
292 |                 "General Inquiry": Users,
293 |                 "General Inquiries": Users,
294 |                 "Document Verification": FileText,
295 |             }[s.serviceName] || MapPin;
296 |             return (
297 |                 <button
298 |                     className="service-row"
299 |                     key={s.serviceId}
300 |                     disabled={busy}
301 |                     onClick={() => join(s)}

```

```

301 |         >
302 |         <span
303 |             className="service-code"
304 |             style={{ "--service-color": s.color }}
305 |         >
306 |             <ServiceIcon size={27} aria-hidden="true" />
307 |         </span>
308 |         <span className="service-copy">
309 |             <strong>{s.serviceName}</strong>
310 |             <span>{s.description}</span>
311 |         </span>
312 |         <span className="service-place">
313 |             <MapPin size={14} />
314 |             {s.serviceName}
315 |         </span>
316 |         <span className="service-wait">
317 |             <strong>{s.averageServiceTime < 1 ? Math.max(1, Math.round(s.averageServiceTime *
| 60)) : Number(s.averageServiceTime.toFixed(1))}</strong>
318 |             <span>{s.averageServiceTime < 1 ? "sec" : "min"} / student</span>
319 |         </span>
320 |         <span className="row-arrow">
321 |             <span>Make booking</span><ArrowUpRight size={21} />
322 |         </span>
323 |         <span className="sr-only">Review booking position and time</span>
324 |     </button>
325 | );
326 |     })}
327 | </div>
328 | </section>
329 | <footer className="page-footer">
330 |     <span>Good things happen outside the queue.</span>
331 |     <span className="mono">TAKE A NUMBER. TAKE YOUR TIME.</span>
332 | </footer>
333 | </>
334 | );
335 | }
336 | function QueuePage({ ticket, tickets, selectTicket, refresh, backend }) {
337 |     const [cancelling, setCancelling] = useState(null);
338 |     if (!ticket) return <<PageHeading eyebrow="YOUR PLACE IN LINE" title={<>A ticket to <span
| className="serif">more time.</span></> description="You don't have an active ticket yet." /><div
| className="empty-state"><TicketIcon size={55}/><h2>Where are we headed?</h2><p>Choose a service to join
| its queue.</p><Link to="/dashboard" state={{ scrollToServices: true }} className="button
| primary">Explore services <ArrowRight size={18}/></Link></div></>;
339 |     const called = ticket.status === "CALLED";
340 |     const serving = ticket.status === "SERVING";
341 |     return <>
342 |         <PageHeading eyebrow="YOUR PLACE IN LINE" title={<>Your <span className="serif">queue
| ticket.</span></> description="Your ticket and position, updated from the service queue." />
343 |         {tickets.length > 1 && <div className="tabs" aria-label="Your active service
| queues">{tickets.map((entry) => <button className={entry.queueId === ticket.queueId ? "active" : ""}
| key={entry.queueId} onClick={() => selectTicket(entry.queueId)}>{entry.serviceName}</button>}</div>}
344 |         <div className="queue-grid"><Ticket ticket={ticket}/><div className="queue-info">
345 |             <section className="queue-progress"><div className="section-label"><p className="eyebrow">THE
| ROAD TO YOUR TURN</p><span className="tag"><span className="status-dot"/>{serving ? "Being served" :
| called ? "Called" : "Waiting"}</span></div>
346 |             <div className="wait-big">{serving ? "You're up." : called ? "It's time." : ticket.wait ===
| null ? <span>Estimate unavailable</span> : <>{ticket.wait}<span> min / service</span></></div>
347 |             <p className="muted" aria-live="polite">{serving ? "Your service is in progress." : called ?
| "Staff have called your token. Please head to the service desk." : `${ticket.ahead} people ahead.
| ${ticket.ahead === 0 ? "You're first in line; wait for staff to call your token." : "We'll update your
| position as staff serve the queue."}}</p>
348 |             {(ticket.closingRisk || ticket.rescheduled) && <p className="closing-warning"
| role="status">{ticket.warning}</p>}
349 |             <VirtualQueue ticket={ticket}/>
350 |             <div className="queue-legend"><span>Now serving: {ticket.currentToken || "No token
| called"}</span><span>{ticket.serviceName}</span></div>
351 |             </section><section className="bring-panel"><span className="eyebrow">BEFORE YOU HEAD
| OVER</span><h3>Keep your student ID handy.</h3><p className="muted">{ticket.counterName ? `Head to
| ${ticket.counterName}.` : "Counter hours: 10 AM-1 PM and 2-3 PM. Your counter appears when staff call
| you."}</p>{ticket.missedTurns>0&&<p className="danger">One missed turn recorded. A second miss ends

```

```

    | your ticket.</p></section>
352 |     <div className="queue-actions"><button className="button secondary" onClick={() =>
    | refresh().catch(() => {})}>Refresh position <ArrowRight size={16}/></button>{ticket.status ===
    | "WAITING" && <button className="text-button danger" disabled={backend.busy}
    | onClick={()=>setCancelling(ticket)}>Cancel ticket</button></div>
353 |   </div></div>
354 |   {cancelling && <Modal title="Cancel this ticket?"
    | close={()=>{if(!backend.busy)setCancelling(null);}}><p>{cancelling.serviceName} \xb7 Token
    | {String(cancelling.token).padStart(3, '0')}</p><p className="muted">Cancelling removes your place in
    | this queue. A new booking will put you at the end of the queue.</p><button className="button secondary"
    | disabled={backend.busy} onClick={()=>setCancelling(null)}>Keep ticket</button><button
    | className="button primary" disabled={backend.busy} onClick={async()=>{try{await backend.action('/bookin
    | gs/cancel',{bookingId:cancelling.queueId});setCancelling(null);}catch{}}}>{backend.busy ?
    | 'Cancelling...' : 'Confirm cancellation'}</button></Modal>}
355 |   </>;
356 | }
357 | function Modal({ title, close, children }) {
358 |   const ref = useRef(null);
359 |   useEffect(() => {
360 |     const prior = document.activeElement;
361 |     const dialog = ref.current;
362 |     dialog.showModal();
363 |     return () => {
364 |       dialog.close();
365 |       prior?.focus();
366 |     };
367 |   }, []);
368 |   return (
369 |     <dialog ref={ref} className="modal" onCancel={close}>
370 |       <div className="section-heading">
371 |         <h2>{title}</h2>
372 |         <button
373 |           className="icon-button"
374 |           onClick={close}
375 |           aria-label="Close dialog"
376 |         >
377 |           <X size={20} />
378 |         </button>
379 |       </div>
380 |       {children}
381 |     </dialog>
382 |   );
383 | }
384 | function BookingForm({ catalog, backend, close, initialServiceId }) {
385 |   const [serviceId,setServiceId]=useState(String(initialServiceId || catalog[0]?.serviceId||""));
386 |   const [preview,setPreview]=useState(null);
387 |   const [error,setError]=useState("");
388 |   useEffect(()=>{
389 |     if(!serviceId)return;
390 |     const controller=new AbortController();let inflight=false;
391 |     setPreview(null);setError("");
392 |     async function load(){
393 |       if(inflight)return;inflight=true;
394 |       try {const data=await studentRequest(`/bookings/preview?serviceId=${serviceId}`,undefined,control
    | ler.signal);if(!controller.signal.aborted){setPreview(data);setError("");}}
395 |       catch(failure){if(failure.name!=="AbortError"){setError(failure.message);setPreview(null);}}
396 |       finally{inflight=false;}
397 |     }
398 |     load();const timer=setInterval(load,5000);
399 |     return ()=>{controller.abort();clearInterval(timer);};
400 |   },[serviceId]);
401 |   return <Modal title="Reserve today's visit" close={close}><form className="form"
    | onSubmit={async(event)=>{
402 |     event.preventDefault();setError("");
403 |     try{await backend.action("/bookings/create",{serviceId});close();}
404 |     catch(failure){setError(failure.message);}
405 |   }>
406 |     <label>Service<select value={serviceId} onChange={event=>setServiceId(event.target.value)}
    | required>{catalog.map(service=><option key={service.serviceId}
    | value={service.serviceId}>{service.serviceName}</option>)}</select></label>
407 |     {preview ? <section className="reservation-preview" aria-live="polite"><p className="eyebrow">TODAY

```

```

| \xb7 {preview.date}</p>{preview.testClock && <p className="closing-warning" role="status">Local
| testing clock: {preview.campusTime} today. Bookings are saved to the database.</p><div
| className="reservation-metrics"><div><span>Your projected
| position</span><strong>{preview.position}</strong></div><div><span>People
| ahead</span><strong>{preview.ahead}</strong></div><div><span>Estimated service
| time</span><strong>{preview.projectedServiceTime || preview.estimate || (preview.closed ? "Closed" :
| "Pending")}</strong></div></div><p className="muted small">Counter hours: 10 AM-1 PM \xb7 break 1-2 PM
| \xb7 2-3 PM.</p><p className="muted small">{preview.message}</p>{preview.closingRisk && <p
| className="closing-warning" role="status">{preview.warning}</p>}{preview.wait!==null && <p
| className="muted small">About {preview.wait} minutes from now, including any wait before opening or
| during lunch.</p></section> : !error && <p role="status">Checking today's queue...</p>
408 | <p className="muted small">Order follows confirmed reservations. First missed call: move back up to
| ten places. Second missed call: ticket ends.</p>
409 | {error && <p role="alert" className="danger">{error}</p>}
410 | <button className="button primary" disabled={!preview?.canReserve || !!error ||
| backend.busy}>{backend.busy?"Reserving...":preview?.queueId?"Already reserved":"Reserve my
| place"}<Check size={18}/></button>

411 | </form></Modal>;
412 | }
413 | function BookingsPage({ backend }) {
414 |   const location=useLocation(),navigate=useNavigate();
415 |   const initialServiceId=backend.catalog.some(service=>service.serviceId===location.state?.bookingServi
| ceId)?location.state.bookingServiceId:null;
416 |   const [tab,setTab]=useState("Upcoming"),[editing,setEditing]=useState(Boolean(initialServiceId)),[can
| cel,setCancel]=useState(null);
417 |   const closeBooking={()=>{setEditing(false);navigate("/bookings",{replace:true,state:null})}};
418 |   const today=new Intl.DateTimeFormat('en-
| CA',{timeZone:'Asia/Kolkata',year:'numeric',month:'2-digit',day:'2-digit'}).format(new Date());
419 |   const active=status=>['WAITING','CALLED','SERVING'].includes(status);
420 |   const filtered=backend.bookings.filter(booking=>tab==='Cancelled'?booking.status==='CANCELLED':tab===
| 'Past'?booking.status!=='CANCELLED'&&(booking.date<today||!active(booking.status)):booking.date>=today&
| &active(booking.status));
421 |   const serviceName=id=>backend.catalog.find(service=>service.serviceId===id)?.serviceName||`Service
| ${id}`;
422 |   return <>
423 |     <PageHeading eyebrow="YOUR PLACE, BEFORE YOU ARRIVE" title={<>Your <span
| className="serif">bookings.</span></>} description="Reserve today, including before opening. No time
| slots."><button className="button primary" disabled={!backend.catalog.length||backend.busy}
| onClick={()=>setEditing(true)}>Plus size={18}/>New booking</PageHeading>
424 |     <div className="tabs" role="tablist" aria-label="Visit
| status">[{'Upcoming','Past','Cancelled'].map(name=><button role="tab" aria-selected={tab===name}
| className={tab===name?'active':''} key={name} onClick={()=>setTab(name)}>{name}</button>)}</div>
425 |     {!filtered.length?<div className="empty-state"><CalendarDays size={45}/><h2>No {tab.toLowerCase()}
| visits.</h2><p>Counter hours: 10 AM-1 PM and 2-3 PM.</p></div>:<div
| className="agenda">{filtered.map(booking=>{
426 |     const ticket=backend.tickets.find(entry=>entry.queueId===booking.id);
427 |     return <article className="agenda-row" key={booking.id}><div className="agenda-date"><span>{new D
| ate(`${booking.date}T12:00`).toLocaleDateString('en',{month:'short'})}</span><strong>{booking.date.slic
| e(-2)}</strong></div><div className="agenda-detail"><span className="mono">TOKEN
| {String(booking.token).padStart(3,'0')}</span><h3>{serviceName(booking.serviceId)}</h3><p
| className="muted"><span className="tag">{booking.status === 'CLOSED_UNSERVED' ? 'Closed - not served' :
| booking.status.replaceAll('_', ' ')}</span>{ticket&& \xb7 Position ${ticket.ahead+1} \xb7 Estimated
| service ${ticket.estimate||'pending'}}</p>{ticket?.closingRisk || ticket?.rescheduled}&&<p
| className="closing-warning" role="status">{ticket.warning}</p>}{booking.status === 'CLOSED_UNSERVED' &&
| <p className="muted small">The counter closed before your turn. Please reserve again tomorrow. No
| missed-turn penalty was applied.</p>}{booking.missedTurns>0&&<p className="muted
| small">{booking.missedTurns>2?'Ticket ended after two missed calls. You may reserve again.':One
| missed call recorded. Your place moved back; a second miss ends this
| ticket.'}</p></div>{booking.date===today&&booking.status==='WAITING'&&<button disabled={backend.busy}
| className="text-button danger" onClick={()=>setCancel(booking)}>Cancel</button>}</article>;
428 |     }</div>}

429 |     {editing&&<BookingForm catalog={backend.catalog} backend={backend}
| initialServiceId={initialServiceId} close={closeBooking}/>
430 |     {cancel&&<Modal title="Cancel this reservation?"
| close={()=>setCancel(null)}><p>{serviceName(cancel.serviceId)} \xb7 {cancel.date}</p><button
| className="button primary" disabled={backend.busy} onClick={async()=>{try{await
| backend.action('/bookings/cancel',{bookingId:cancel.id});setCancel(null);}catch{}}}>Cancel
| reservation</button></Modal>}
431 |   </>;
432 | }function SettingsPage({ profile, dark, toggle, logout, busy }) {

```

```

433 |   return <> <PageHeading eyebrow="THE PERSONAL DETAILS" title={<>Profile & <span
|   className="serif">settings.</span></>} description="Your account and appearance preferences."/>
434 |   <div className="settings-grid"><section className="settings-panel"><div className="section-
|   heading"><h2>Your student profile</h2><span className="avatar">{profile.name[0]}</span></div><div
|   className="form"><label>Full name<input value={profile.name} readOnly/></label><label>Email
|   address<input value={profile.email} readOnly/></label><label>Student ID<input value={profile.studentId}
|   readOnly/></label><p className="muted small">Your registered campus account.</p></div></section>
435 |   <section className="settings-panel"><h2>The way you like it</h2><div
|   className="preference"><div><h3>Dark appearance</h3><p className="muted small">A quieter palette for
|   after hours.</p></div><button role="switch" aria-checked={dark} aria-label="Dark appearance"
|   className={`toggle ${dark ? "on" : ""}`} onClick={toggle}><span/></button></div><div disabled={busy}
|   className="text-button danger signout" onClick={logout}><LogOut size={16}/>Sign
|   out</button></div></section></div>
436 |   </>;
437 | }
438 | function LandingPage({ dark, toggle }) {
439 |   return <div className="landing-page">
440 |     <header className="landing-nav"><Brand/><nav aria-label="Public navigation"><Theme dark={dark}
|   toggle={toggle}/><Link className="button secondary" to="/login">Log in</Link></nav></header>
441 |     <main><section className="landing-hero"><div><p className="eyebrow">CAMPUS SERVICES \xb7 WITHOUT
|   THE STANDING</p><h1>Book your campus queue.<br/><span className="serif">Skip standing in
|   line.</span></h1><p class="muted">Choose a campus service, review your estimated turn, and reserve
|   your place. Follow your ticket online so you know when to head to the counter.</p><div
|   className="landing-actions"><Link className="button primary" to="/login?signup=1">Sign up <ArrowRight
|   size={18}/></Link><Link className="button secondary" to="/login">Log in</Link></div></div><section
|   className="feature-panel"><p class="eyebrow">A DIGITAL PLACE IN LINE</p><VirtualQueue/><p
|   className="muted small">Illustration only. Your actual position appears after you confirm a
|   booking.</p></section></section>
442 |     <section className="landing-section"><p class="eyebrow">HOW IT WORKS</p><h2>Three steps to
|   <span className="serif">your turn.</span></h2><div className="landing-steps">{[['01', 'Choose a
|   service', 'Select General Inquiries, Fee Payment, or Document Verification. Review your position and
|   estimated time before booking.'], ['02', 'Confirm your place', 'Click Reserve my place to get your ticket.
|   Bookings are for today only, including before counters open.'], ['03', 'Track your turn', 'Open My queue
|   to check your position. Bring your student ID and approach the assigned counter when staff call your
|   token.']].map(([, number, title, text]) => <article key={number}><span
|
|   className="eyebrow">{number}</span><h3>{title}</h3><p
|   className="muted">{text}</p></article>)}</div></section>
443 |     <section className="landing-rules"><div><p class="eyebrow">COUNTER HOURS</p><h2>10 AM-1
|   PM<br/><span class="serif">2 PM-3 PM</span></h2><p>Lunch break: 1 PM-2 PM.</p></div><div><h3>Before
|   you book</h3><p>Places follow confirmed booking order. Estimated times can change as the queue
|   moves.</p><p>First missed call: move back up to ten places. Second missed call: your ticket
|   ends.</p><p>If your estimated turn reaches closing time, return tomorrow. Unserved waiting tickets move
|   to the next day at 10 AM without an extra missed-call penalty.</p><Link className="button primary"
|   to="/login?signup=1">Create student account <ArrowRight size={18}/></Link></div></section></main>
444 |     <footer className="page-footer"><span>CampusQueue \xb7 Student services</span><Link
|   to="/login">Student login</Link></footer>
445 |   </div>;
446 | }
447 | function LoginPage({ dark, toggle, backend }) {
448 |   const authLocation = useLocation();
449 |   const [register, setRegister] = useState(authLocation.search === "?signup=1");
450 |   const navigate = useNavigate();
451 |   return (
452 |     <div className="login-page">
453 |       <section className="login-story">
454 |         <Brand />
455 |         <div className="login-message">
456 |           <p class="eyebrow">YOUR CAMPUS. YOUR PACE.</p>
457 |           <h1>
458 |             No more
459 |             <br />
460 |             standing in
461 |             <br />
462 |             <span class="serif">lines.</span>
463 |           </h1>
464 |           <p>
465 |             Take a number. Skip the standing around.
466 |             <br />
467 |             There's a whole campus out there.
468 |           </p>
469 |         </div>

```

```

470 |         className="login-floats"
471 |         aria-label="Illustrative queue information"
472 |     >
473 |         <div className="floating-stat now-serving">
474 |             <span className="eyebrow">YOUR PLACE, KEPT</span>
475 |             <strong>A digital ticket</strong>
476 |         </div>
477 |         <div className="floating-stat estimated-wait">
478 |             <span className="eyebrow">MORE TIME FOR</span>
479 |             <strong>Your campus day</strong>
480 |         </div>
481 |     </div>
482 | </div>
483 | <span className="mono small">A BETTER WAY TO WAIT / CAMPUSQUEUE</span>
484 | </section>
485 | <section className="login-form-side">
486 |     <div className="login-theme">
487 |         <Theme dark={dark} toggle={toggle} />
488 |     </div>
489 |     <div className="auth-form">
490 |         <p className="eyebrow">LET'S GET YOU GOING</p>
491 |         <h2>{register ? "Create your account" : "Student login"}</h2>
492 |         <p className="muted">
493 |             {register
494 |               ? "Create your campus student account."
495 |               : "Sign in to book a service and track your queue."}
496 |         </p>
497 |         <form
498 |             className="form"
499 |             onSubmit={async (e) => {
500 |                 e.preventDefault();
501 |                 const data = new FormData(e.currentTarget);
502 |                 try {
503 |                     await backend.action(register ? "/register" : "/login", {
504 |                         studentName: data.get("name") || "",
505 |                         email: data.get("email"), password: data.get("password"), phone: data.get("phone") ||
506 |                         "",
507 |                     });
508 |                     navigate("/dashboard");
509 |                 } catch {}
510 |             }}
511 |         >
512 |             {register && (
513 |                 <label>
514 |                     Your name
515 |                     <input
516 |                         name="name"
517 |                         autoComplete="name"
518 |                         placeholder="Full name"
519 |                         required
520 |                     />
521 |                 </label>
522 |             )}
523 |             <label>
524 |                 College email
525 |                 <input
526 |                     name="email"
527 |                     type="email"
528 |                     autoComplete="email"
529 |                     placeholder="you@college.edu"
530 |                     maxLength={100}
531 |                     required
532 |                 />
533 |             </label>
534 |             {register && <label>Mobile number<input name="phone" type="tel" autoComplete="tel"
535 |                 inputMode="tel" placeholder="9876543210" pattern="(?:\+91)?[6-9][0-9]{9}" required/><span
536 |                 className="muted small">10-digit Indian mobile number, optionally with +91.</span></label>}
537 |             <label>
538 |                 Password
539 |                 <input

```

```

537 |         name="password"
538 |         type="password"
539 |         autoComplete={register ? "new-password" : "current-password"}
540 |         placeholder="Your password"
541 |         required
542 |         minLength={register ? 6 : undefined}
543 |         pattern={register ? "(?=.*[A-Za-z])(?=.*[0-9]).{6,128}" : undefined}
544 |         maxLength={register ? 128 : undefined}
545 |       />
546 |     </label>
547 |     <button className="button primary" disabled={backend.busy}>{backend.busy ? "Please wait..."
| : register ? "Create account" : "Sign in"}
548 |       <ArrowRight size={18} />
549 |     </button>
550 |   </form>
551 |   {register && <p className="muted small">Use 6-128 characters, including at least one letter
| and one number.</p>}
552 |   <p className="auth-switch">
553 |     {register ? "Already have a profile?" : "New around here?"}{ " "}
554 |     <button
555 |       className="text-button"
556 |       onClick={() => setRegister(!register)}
557 |     >
558 |       {register ? "Sign in" : "Get started"}
559 |     </button>
560 |   </p>

561 |   <p className="demo-note">Sign in with your registered campus account.</p><Link to="/">\u2190
| How CampusQueue works</Link>
562 |   {backend.error && <p className="danger" role="alert">{backend.error}</p>}
563 | </div>
564 | <span className="login-foot">Less wait. More campus.</span>
565 | </section>
566 | </div>
567 | );
568 | }
569 | function Experience() {
570 |   const location = useLocation();
571 |   const navigate = useNavigate();
572 |   const root = useRef(null);
573 |   const backend = useStudentBackend();
574 |   const [selectedTicket, selectTicket] = useState(null);
575 |   const profile = backend.profile;
576 |   const ticket = backend.tickets.find((entry) => entry.queueId === selectedTicket) ||
| backend.tickets[0] || null;
577 |   const [dark, setDark] = useState(localStorage.getItem("theme") === "dark");
578 |
579 |   useEffect(() => {
580 |     document.documentElement.dataset.theme = dark ? "dark" : "light";
581 |     localStorage.setItem("theme", dark ? "dark" : "light");
582 |   }, [dark]);
583 |   useEffect(() => {
584 |     document.title = `${nav.find((n) => n[0] === location.pathname)?.[2] || "Welcome"} | CampusQueue`;
585 |     if (matchMedia("(prefers-reduced-motion: reduce)").matches) return;
586 |     const ctx = gsap.context(() => {
587 |       const panels = root.current.querySelectorAll(
588 |         ".page-heading, .overview-grid, .services-section, .queue-grid, .settings-grid, .auth-form,
| .login-message",
589 |       );
590 |       if (panels.length)
591 |         gsap.fromTo(
592 |           panels,
593 |           { y: 18, opacity: 0 },
594 |           {
595 |             y: 0,
596 |             opacity: 1,
597 |             duration: 0.55,
598 |             stagger: 0.07,
599 |             ease: "power3.out",
600 |           },
601 |         );

```

```

602 |     const routes = root.current.querySelectorAll(".map-route");
603 |     if (routes.length)
604 |         gsap.fromTo(
605 |             routes,
606 |             { strokeDasharray: 600, strokeDashoffset: 600 },
607 |             { strokeDashoffset: 0, duration: 1.4, ease: "power2.out" },
608 |         );
609 |     }, root);
610 |     return () => ctx.revert();
611 | }, [location.pathname]);
612 | function create(service) {
613 |     navigate("/bookings", { state: { bookingServiceId: service.serviceId } });
614 | }
615 | const toggle = () => setDark(!dark);
616 | if (!backend.ready) return <div ref={root} className="empty-state" role="status">Connecting to your
| campus...</div>;
617 | if (!profile && location.pathname === "/") return <div ref={root}><LandingPage dark={dark}
| toggle={toggle}/></div>;
618 | if (profile && (location.pathname === "/" || location.pathname === "/login")) return <Navigate
| to="/dashboard" replace/>;
619 | if (!profile)
620 |     return (
621 |         <div ref={root}>
622 |             <LoginPage dark={dark} toggle={toggle} backend={backend} />
623 |         </div>
624 |     );
625 | return (
626 |     <div ref={root} className="app-shell">
627 |         <aside className="sidebar">
628 |             <Brand />
629 |             <div className="sidebar-label eyebrow">THE STUDENT SPACE</div>
630 |             <nav aria-label="Main navigation">
631 |                 {nav.map(([path, Icon, label], i) => (
632 |                     <NavLink key={path} to={path}>
633 |                         <Icon size={19} />
634 |                         <span>{label}</span>
635 |                         <span className="nav-number">0{i + 1}</span>
636 |                     </NavLink>
637 |                 ))}
638 |             </nav>
639 |             <div className="sidebar-bottom">
640 |                 <div className="sidebar-note">
641 |                     <span className="asterisk">\u2733</span>
642 |                     <p>
643 |                         A whole campus.
644 |                         <br />
645 |                         <span className="serif">Beyond the queue.</span>
646 |                     </p>
647 |                 </div>
648 |                 <div className="student-profile">
649 |                     <span className="avatar">{profile.name[0]}</span>
650 |                     <span>
651 |                         <strong>{profile.name}</strong>
652 |                         <small>Student</small>
653 |                     </span>
654 |                 </div>
655 |             </div>
656 |         </aside>
657 |         <div className="main-shell">
658 |             <div className="topbar">
659 |                 <span className="topbar-brand">campusqueue</span>
660 |                 <span className="breadcrumb">
661 |                     STUDENT SPACE <span>/</span>{ " " }
662 |                     {nav.find((n) => n[0] === location.pathname)?.[2]?.toUpperCase()}
663 |                 </span>
664 |                 <div className="topbar-right">
665 |                     <span className="mono date-label">
666 |                         {new Date().toLocaleDateString("en-IN", {
667 |                             day: "2-digit",
668 |                             month: "short",
669 |                             year: "numeric",

```

```

670 |         }}}
671 |         </span>
672 |         <Theme dark={dark} toggle={toggle} />
673 |       </div>
674 |     </div>
675 |     <main>
676 |       {backend.error && <div className="backend-error" role="alert">{backend.error}<button
| className="text-button" onClick={() => backend.refresh().catch(() => {})}>Try again</button></div>}
677 |       <Routes>
678 |         <Route
679 |           path="/dashboard"
680 |           element={
681 |             <Overview
682 |               ticket={ticket}
683 |               profile={profile}
684 |               catalog={backend.catalog}
685 |               busy={backend.busy}
686 |               join={create}
687 |             />
688 |           }
689 |         />
690 |         <Route
691 |           path="/queue"
692 |           element={<QueuePage ticket={ticket} tickets={backend.tickets} selectTicket={selectTicket}
| refresh={backend.refresh} backend={backend} />}
693 |         />
694 |         <Route path="/bookings" element={<BookingsPage backend={backend} />} />
695 |         <Route
696 |           path="/settings"
697 |           element={
698 |             <SettingsPage
699 |               profile={profile}
700 |               busy={backend.busy}
701 |               logout={() => backend.action("/logout", {}).then(() => navigate("/login")).catch(()
| => {})}
702 |             dark={dark}
703 |             toggle={toggle}
704 |           />
705 |           }
706 |         />
707 |         <Route path="*" element={<Navigate to="/dashboard" replace />} />
708 |       </Routes>
709 |     </main>
710 |   </div>
711 |   <nav className="bottom-nav" aria-label="Mobile navigation">
712 |     {nav.map(([path, Icon, label]) => (
713 |       <NavLink key={path} to={path}>
714 |         <Icon size={20} />
715 |         <span>{label}</span>
716 |       </NavLink>
717 |     ))}
718 |   </nav>
719 | </div>
720 | );
721 | }
722 | export default function App() {
723 |   return (
724 |     <HashRouter>
725 |       <Experience />
726 |     </HashRouter>
727 |   );
728 | }

```

QueueSystemFrontend/src/main.jsx

Browser entry point: mounts the React app into the root element and imports styles. React StrictMode helps reveal development side-effect problems.

Numbered source

```
1 | import { StrictMode } from 'react'
2 | import { createRoot } from 'react-dom/client'
3 | import './index.css'
4 | import App from './App.jsx'
5 |
6 | createRoot(document.getElementById('root')).render(
7 |   <StrictMode>
8 |     <App />
9 |   </StrictMode>,
10 | )
```

QueueSystemFrontend/src/queue-layout.js

Pure animation layout helper. Groups large queues visually; it does not calculate authoritative server queue order.

Numbered source

```
1 | export function queueLayout(value) {
2 |   const count = Number(value);
3 |   const ahead = Number.isFinite(count) ? Math.max(0, Math.trunc(count)) : 0;
4 |   const grouped = ahead > 4;
5 |   const shown = grouped ? 3 : ahead;
6 |   return {
7 |     ahead,
8 |     grouped,
9 |     hidden: ahead - shown,
10 |     you: grouped ? 55 : 342 - shown * 66,
11 |     people: Array.from({ length: shown }, (_, index) => 342 - index * (grouped ? 56 : 66)),
12 |   };
13 | }
```

QueueSystemFrontend/src/student-api.js

Student fetch wrapper and shared backend hook. Includes credentials, error handling, polling and focused-tab refresh.

Read these lines first

Line 1: Synchronizes UI with polling or another side effect. Explain dependencies and timer cleanup.

Line 4: Sends an HTTP request. Explain method, credentials, body encoding and handling of unsuccessful responses.

Line 6: Includes cookie credentials so the backend can restore the authenticated session through the proxy.

Line 14: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 24: Declares component state and a setter; changes cause React to render the relevant UI.

Line 25: Declares component state and a setter; changes cause React to render the relevant UI.

Line 26: Declares component state and a setter; changes cause React to render the relevant UI.

Line 27: Declares component state and a setter; changes cause React to render the relevant UI.

Numbered source

```

1 | import { useCallback, useEffect, useState } from "react";
2 |
3 | export async function studentRequest(path, payload, signal) {
4 |   const response = await fetch(`/api/student${path}`, {
5 |     method: payload ? "POST" : "GET",
6 |     credentials: "include",
7 |     cache: "no-store",
8 |     signal: signal || AbortSignal.timeout(12000),
9 |     headers: { Accept: "application/json", "X-Requested-With": "XMLHttpRequest" },
10 |    body: payload ? new URLSearchParams(payload) : undefined,
11 |  });
12 |  let body;
13 |  try { body = await response.json(); }
14 |  catch { throw new Error("The Java backend is not responding. Please try again shortly."); }
15 |  if (!response.ok || !body.success) {
16 |    const error = new Error(body.message || "The request could not be completed.");
17 |    error.status = response.status;
18 |    throw error;
19 |  }
20 |  return body.data;
21 | }
22 |
23 | export function useStudentBackend() {
24 |   const [profile, setProfile] = useState(null);
25 |   const [ready, setReady] = useState(false);
26 |   const [catalog, setCatalog] = useState([]);
27 |   const [tickets, setTickets] = useState([]);
28 |   const [bookings, setBookings] = useState([]);
29 |   const [error, setError] = useState("");
30 |   const [busy, setBusy] = useState(false);
31 |   const refresh = useCallback(async () => {
32 |     try {
33 |       const [overview, appointments] = await Promise.all([studentRequest("/overview"),
34 | studentRequest("/bookings")]);
35 |       setCatalog(overview.services); setTickets(overview.tickets); setBookings(appointments);
36 |       setError("");
37 |     } catch (failure) {
38 |       setError(failure.message);
39 |       if (failure.status === 401) { setProfile(null); setTickets([]); setBookings([]); setCatalog([]);
40 |     }
41 |     throw failure;
42 |   }
43 | }, []);
44 |   useEffect(() => {

```

```
42 |     const controller = new AbortController();
43 |     studentRequest("/session", undefined, controller.signal).then(setProfile).catch((failure) => {
44 |       if (failure.name !== "AbortError") setError(failure.message);
45 |     }).finally(() => setReady(true));
46 |     return () => controller.abort();
47 |   }, []);
48 |   useEffect(() => {
49 |     if (!profile) return;
50 |     refresh().catch(() => {});
51 |     const timer = setInterval(() => { if (!document.hidden) refresh().catch(() => {}); }, 8000);
52 |     const refreshWhenVisible = () => { if (!document.hidden) refresh().catch(() => {}); };
53 |     window.addEventListener("focus", refreshWhenVisible);
54 |     document.addEventListener("visibilitychange", refreshWhenVisible);
55 |     return () => {
56 |       clearInterval(timer);
57 |       window.removeEventListener("focus", refreshWhenVisible);
58 |       document.removeEventListener("visibilitychange", refreshWhenVisible);
59 |     };
60 |   }, [profile, refresh]);
61 |   async function action(path, payload) {
62 |     setBusy(true); setError("");
63 |     try {
64 |       const data = await studentRequest(path, payload);
65 |       if (path === "/login" || path === "/register") setProfile(data);
66 |       else if (path === "/logout") { setProfile(null); setCatalog([]); setTickets([]); setBookings([]);
67 |     }
68 |       else await refresh();
69 |       return data;
70 |     } catch (failure) { setError(failure.message); throw failure; }
71 |     finally { setBusy(false); }
72 |   }
73 |   return { profile, ready, catalog, tickets, bookings, error, busy, refresh, action };
}
```

frontend/src/App.jsx

Staff BrowserRouter, restored login session, navigation and dashboard composition.

Read these lines first

Line 9: Synchronizes UI with polling or another side effect. Explain dependencies and timer cleanup.

Line 35: Declares component state and a setter; changes cause React to render the relevant UI.

Line 57: Declares component state and a setter; changes cause React to render the relevant UI.

Line 58: Declares component state and a setter; changes cause React to render the relevant UI.

Line 59: Declares component state and a setter; changes cause React to render the relevant UI.

Line 60: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 61: Synchronizes UI with polling or another side effect. Explain dependencies and timer cleanup.

Line 62: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | import {
2 |   BrowserRouter,
3 |   Routes,
4 |   Route,
5 |   Navigate,
6 |   useNavigate
7 | } from "react-router-dom";
8 |
9 | import { useState, useEffect, useCallback } from "react";
10 | import { api } from "../services/api";
11 |
12 | import StaffLogin from "../pages/StaffLogin";
13 | import StaffDashboard from "../pages/StaffDashboard";
14 |
15 |
16 | function LoginPage({ onLogin }) {
17 |   const navigate = useNavigate();
18 |
19 |   function handleLogin(staffData) {
20 |     onLogin(staffData);
21 |     navigate("/dashboard");
22 |   }
23 |
24 |   return (
25 |     <StaffLogin onLogin={handleLogin} />
26 |   );
27 | }
28 |
29 |
30 | function DashboardPage({ staff, onLogout }) {
31 |
32 |   const navigate = useNavigate();
33 |
34 |   // This controls which section of the staff dashboard is displayed
35 |   const [page, setPage] = useState("dashboard");
36 |
37 |
38 |   async function handleLogout() {
39 |     await onLogout();
40 |     navigate("/");
41 |   }
42 |
43 |
44 |   return (

```

```

45 |         <StaffDashboard
46 |             staff={staff}
47 |             onLogout={handleLogout}
48 |             page={page}
49 |             onNavigate={setPage}
50 |         />
51 |     );
52 | }
53 |
54 |
55 | function App() {
56 |
57 |     const [staff, setStaff] = useState(null);
58 |     const [ready, setReady]=useState(false);
59 |     const [error, setError]=useState('');
60 |     const restore=useCallback(()=>{setError('');api.profile().then(result=>setStaff(result.data)).catch
61 | (err=>{if(err.status!==401)setError(err.message);}).finally(()=>setReady(true));}, []);
62 |     useEffect(()=>{restore();}, [restore]);
63 |     const logout=useCallback(async()=>{try{await
64 | api.logout();setStaff(null);}catch(err){setError(err.message);}}, []);
65 |     if(!ready)return <div className="empty-state">Checking staff session...</div>;
66 |     if(error)return <div className="error-banner" role="alert">{error}<button
67 | onClick={restore}>Retry</button></div>;
68 |
69 |     return (
70 |         <BrowserRouter>
71 |
72 |             <Routes>
73 |
74 |                 <Route
75 |                     path="/"
76 |                     element={
77 |                         <LoginPage
78 |                             onLogin={setStaff}
79 |                         />
80 |                     }
81 |                 />
82 |
83 |                 <Route
84 |                     path="/dashboard"
85 |                     element={
86 |                         staff ? (
87 |                             <DashboardPage
88 |
89 |                                 staff={staff}
90 |                                 onLogout={logout}
91 |                             />
92 |                         ) : (
93 |                             <Navigate
94 |                                 to="/"
95 |                                 replace
96 |                             />
97 |                         )
98 |                     }
99 |                 />
100 |             </Routes>
101 |         </BrowserRouter>
102 |     );
103 | }
104 |
105 | export default App;

```

frontend/src/components/ActivityFeed.jsx

Recent event/status list presentation. Explain display fields separately from database state transitions.

Numbered source

```
1 | export default function ActivityFeed({items=[]}) {  
2 |   return <div className="activity-feed">{items.length ? items.map((item,index)=><div  
  |     className="activity-item"  
  |     key={` ${item.queueId}-${item.action}-${index}`}><strong>A{String(item.tokenNumber ??  
  |     ' ').padStart(3, '0')}</strong><span>{item.action?.replaceAll('_', '  
  |     ')}</span><small>{item.timestamp}</small></div>) : <div className="empty-state">No recent  
  |     activity.</div></div>  
3 | }
```

frontend/src/components/QueueTable.jsx

Queue rows and state/ownership-based action buttons. Server checks remain authoritative.

Numbered source

```

1 | import StudentPhone from './StudentPhone'
2 | export default function QueueTable({queue, busy, staff, onStart, onComplete, onNoShow}) {
3 |   return <div className="table-wrap"><table><thead><tr><th>Token</th><th>Queue
  | ID</th><th>Student</th><th>Status</th><th>Action</th></tr></thead><tbody>{queue.map(q=>{
4 |     const owned=q.counterId===staff?.counterId&&q.assignedStaffId===staff?.staffId;
5 |     return <tr key={q.queueId}><td><strong>{String(q.tokenNumber).padStart(3, '0')}</strong></td><td>{q.
  | queueId}</td><td><strong>{q.studentName||`Student
  | #${q.studentId}`}</strong><div>{q.studentEmail}</div><div><StudentPhone phone={q.studentPhone}
  | name={q.studentName} /></div></td><td>{q.status}</td><td>{owned&&q.status==='CALLED'&&<button
  | disabled={busy} onClick={()=>onStart(q.queueId)}>Start</button><button disabled={busy}
  | onClick={()=>onNoShow(q.queueId)}>Mark absent</button></td>}</td><td>{owned&&q.status==='SERVING'&&<button
  | disabled={busy}
  | onClick={()=>onComplete(q.queueId)}>Complete</button>}<td>{!owned&&q.status!=='WAITING'&&<span>Assigned to
  | another operator</span>}<td>{q.status==='WAITING'&&<span>Waiting to be summoned</span></td></tr>;
6 |   })}</tbody></table>{!queue.length&&<div className="empty-state">No active students in the
  | queue.</div></div>
7 | }

```

frontend/src/components/StatCard.jsx

Reusable summary card presentation; values come from the dashboard API.

Numbered source

```
1 | export default function StatCard({label,value,suffix='',icon}) {  
2 |   return <div className="stat-card"><span className="stat-icon">{icon}</span><div><span  
   |   className="eyebrow">{label}</span><strong>{value ?? '-'}<small>{suffix}</small></strong></div></div>  
3 | }
```

frontend/src/components/StudentPhone.jsx

Validated telephone display and tel link. Device-mediated calling; no server telephony or queue mutation.

Numbered source

```

1 | export default function StudentPhone({ phone, name = 'student' }) {
2 |   const number = String(phone || '').replace(/\s()-/g, '')
3 |   if (!/^\+?[0-9]{10,15}$/.test(number)) return <span>Phone not provided</span>
4 |   return <span className="student-phone"><span>{phone}</span><a className="phone-action"
      | href={`tel:${number}`} aria-label={`Phone ${name}`} title={`Phone ${name}`}><svg width="18" height="18"
      | viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" aria-hidden="true"><path
      | d="M22 16.9v3a2 2 0 0 1-2.2 2 19.8 19.8 0 0 1-8.6-3.1 19.5 19.5 0 0 1-6-6A19.8 19.8 0 0 1 2.1 4.2 2 2 0
      | 0 1 4.1 2h3a2 2 0 0 1 2 1.7c.1 1 .4 2 .7 2.9a2 2 0 0 1-.5 2.1L8 10a16 16 0 0 0 6 6l1.3-1.3a2 2 0 0 1
      | 2.1-.5c.9.3 1.9.6 2.9.7a2 2 0 0 1 1.7 2z"/></svg></a></span>
5 | }
```

frontend/src/main.jsx

Browser entry point: mounts the React app into the root element and imports styles. React StrictMode helps reveal development side-effect problems.

Numbered source

```
1 | import React from 'react'
2 | import { createRoot } from 'react-dom/client'
3 | import App from './App'
4 | import './style.css'
5 | createRoot(document.getElementById('root')).render(<App />)
```

frontend/src/pages/StaffDashboard.jsx

Staff dashboard orchestration: polls endpoints, derives assigned current service, runs actions and refreshes UI state.

Read these lines first

Line 2: Synchronizes UI with polling or another side effect. Explain dependencies and timer cleanup.

Line 16: Declares component state and a setter; changes cause React to render the relevant UI.

Line 17: Declares component state and a setter; changes cause React to render the relevant UI.

Line 18: Declares component state and a setter; changes cause React to render the relevant UI.

Line 19: Declares component state and a setter; changes cause React to render the relevant UI.

Line 20: Declares component state and a setter; changes cause React to render the relevant UI.

Line 21: Declares component state and a setter; changes cause React to render the relevant UI.

Line 22: Declares component state and a setter; changes cause React to render the relevant UI.

Numbered source

```

1 | import StudentPhone from '../components/StudentPhone'
2 | import { useCallback, useEffect, useMemo, useState } from 'react'
3 | import { api } from '../services/api'
4 | import StatCard from '../components/StatCard'
5 | import QueueTable from '../components/QueueTable'
6 | import ActivityFeed from '../components/ActivityFeed'
7 |
8 | const pageLabels = {
9 |   dashboard: ['STAFF DASHBOARD', 'Dashboard'],
10 |   queue: ['QUEUE MANAGEMENT', 'Queue Management'],
11 |   bookings: ['BOOKINGS', 'Bookings'],
12 |   analytics: ['ANALYTICS', 'Analytics'],
13 | }
14 |
15 | export default function StaffDashboard({ staff, onLogout, page = 'dashboard', onNavigate }) {
16 |   const [summary, setSummary] = useState(null)
17 |   const [queue, setQueue] = useState([])
18 |   const [counter, setCounter] = useState(null)
19 |   const [activity, setActivity] = useState([])
20 |   const [busy, setBusy] = useState(false)
21 |   const [error, setError] = useState('')
22 |   const [lastUpdated, setLastUpdated] = useState(null)
23 |
24 |   const load = useCallback(async (silent = false) => {
25 |     try {
26 |       if (!silent) setError('')
27 |       const [summaryResult, queueResult, counterResult, activityResult] = await Promise.all([
28 |         api.summary(), api.queue(), api.counterStatus(), api.recentActivity(),
29 |       ])
30 |       setSummary(summaryResult.data)
31 |       setQueue(queueResult.data || [])
32 |       setCounter(counterResult.data)
33 |       setActivity(activityResult.data || [])
34 |       setLastUpdated(new Date())
35 |     } catch (err) {
36 |       if (err.status === 401) {
37 |         onLogout()
38 |         return
39 |       }
40 |       setError(err.message)
41 |     }
42 |   }, [onLogout])
43 |
44 |   useEffect(() => {

```

```

45 |     load()
46 |     const timer = setInterval(() => load(true), 7000)
47 |     return () => clearInterval(timer)
48 |   }, [load])
49 |
50 |   const currentService = useMemo(() => queue.find(q => q.counterId === staff.counterId &&
| q.assignedStaffId === staff.staffId && q.status === 'SERVING') || queue.find(q => q.counterId ===
| staff.counterId && q.assignedStaffId === staff.staffId && q.status === 'CALLED'), [queue,
| staff.counterId, staff.staffId])
51 |   const waiting = queue.filter(q => q.status === 'WAITING').length
52 |
53 |   async function runAction(action) {
54 |     try {
55 |       setBusy(true)
56 |       setError('')
57 |       await action()
58 |       await load(true)
59 |     } catch (err) {
60 |       setError(err.message)
61 |     } finally {
62 |       setBusy(false)
63 |     }
64 |   }
65 |
66 |   const setCounterStatus = (status) => runAction(async () => {
67 |     const result = await api.setCounterStatus(status)
68 |     setCounter(result.data)
69 |   })
70 |
71 |   const statusClass = counter?.status?.toLowerCase() || 'unknown'
72 |   const [eyebrow, titleLabel] = pageLabels[page] || pageLabels.dashboard
73 |   const firstName = staff.staffName.split(' ')[0]
74 |
75 |   const nav = (target) => onNavigate(target)
76 |
77 |   return (
78 |     <div className="app-shell">
79 |       <aside className="sidebar">
80 |         <div className="side-brand cq-brand" aria-label="Campus Queue"><span className="cq-
| wordmark"><span>Campus</span><strong>Queue</strong></span></div>
81 |         <nav>
82 |           <div className="nav-section">STAFF CONSOLE</div>
83 |           <button className={`nav-item ${page === 'dashboard' ? 'active' : ''}`} onClick={() =>
| nav('dashboard')}><span>\u25a6</span> Dashboard</button>
84 |           <button className={`nav-item ${page === 'queue' ? 'active' : ''}`} onClick={() =>
| nav('queue')}><span>\u25f7</span> Queue Management</button>
85 |           <button className={`nav-item ${page === 'bookings' ? 'active' : ''}`} onClick={() =>
| nav('bookings')}><span>\u25a3</span> Bookings</button>
86 |           <button className={`nav-item ${page === 'analytics' ? 'active' : ''}`} onClick={() =>
| nav('analytics')}><span>\u25d2</span> Analytics</button>
87 |         </nav>
88 |         <div className="side-bottom">
89 |           <div className="session-label">SIGNED IN AS</div>
90 |           <strong>{staff.staffName}</strong>
91 |           <span>{staff.email}</span>
92 |           <button className="logout-btn" onClick={onLogout}>Sign out</button>
93 |         </div>
94 |       </aside>
95 |
96 |       <main className="main-content">
97 |         <header className="topbar">
98 |           <div>
99 |             <div className="eyebrow">{eyebrow}</div>
100 |             <h1>{page === 'dashboard' ? `Good ${new Date().getHours() < 12 ? 'morning' : new
| Date().getHours() < 18 ? 'afternoon' : 'evening'}`, ${firstName}.` : titleLabel}</h1>
101 |           </div>
102 |           <div className="topbar-right">
103 |             <div className="live-pill"><span /> Live</div>
104 |             <div className="clock">{new Date().toLocaleTimeString([], { hour: '2-digit', minute:
| '2-digit' })}</div>

```

```

105 |         </div>
106 |     </header>
107 |
108 |     {error && <div className="error-banner">{error}</div>}
109 |
110 |     <section className="context-bar">
111 |         <div><span>Service</span><strong>{counter?.serviceName || `Service
112 | | ${staff.serviceId}`}</strong></div>
113 |         <div><span>Counter</span><strong>{counter?.counterName || `Counter
114 | | ${staff.counterId}`}</strong></div>
115 |         <div><span>Last refresh</span><strong>{lastUpdated ? lastUpdated.toLocaleTimeString([], {
116 | | hour: '2-digit', minute: '2-digit', second: '2-digit' }) : 'Loading...'}</strong></div>
117 |         <div className="counter-control">
118 |             <span>Counter status</span>
119 |             <select value={counter?.status || ''} onChange={(e) => setCounterStatus(e.target.value)}
120 | | disabled={busy || !counter}>
121 |                 <option value="AVAILABLE">Available</option>
122 |
123 |                 <option value="PAUSED">Paused</option>
124 |                 <option value="CHECKED_OUT">Checked out</option>
125 |             </select>
126 |         </div>
127 |     </section>
128 |
129 |     {page === 'dashboard' && (
130 |         <DashboardView staff={staff} summary={summary} queue={queue} waiting={waiting}
131 | | counter={counter} activity={activity} busy={busy} runAction={runAction} currentService={currentService}
132 | | statusClass={statusClass} />
133 |     )}
134 |
135 |     {page === 'queue' && (
136 |         <QueueManagementView staff={staff} queue={queue} waiting={waiting} counter={counter}
137 | | busy={busy} runAction={runAction} />
138 |     )}
139 |
140 |     {page === 'bookings' && <BookingsView summary={summary} queue={queue} />}
141 |
142 |     {page === 'analytics' && <AnalyticsView summary={summary} activity={activity} queue={queue} />}
143 | </main>
144 | </div>
145 | )
146 | }
147 |
148 | function DashboardView({ staff, summary, queue, waiting, counter, activity, busy, runAction,
149 | | currentService, statusClass }) {
150 |     return <>
151 |         <section className="stats-grid">
152 |             <StatCard label="Students Waiting" value={summary?.studentsWaiting ?? waiting} icon="\u25c9" />
153 |             <StatCard label="Served Today" value={summary?.servedToday} icon="\u2713" />
154 |             <StatCard label="Avg. Wait" value={summary?.averageWaitMinutes} suffix=" min" icon="\u25f7" />
155 |             <StatCard label="Avg. Service" value={summary?.averageServiceMinutes} suffix=" min" icon="\u2197" />
156 |
157 |             <StatCard label="No-shows" value={summary?.noShowsToday} icon="!" />
158 |             <StatCard label="Peak Hour" value={summary?.peakHour || '-'} icon="\u2301" />
159 |         </section>
160 |
161 |         <section className="workspace-grid">
162 |             <div className="panel queue-panel">
163 |                 <div className="panel-head">
164 |                     <div><h2>Live Queue</h2><p>{queue.length} active entries \xb7 {waiting} waiting</p></div>
165 |                     {!queue.some(q=>q.counterId===staff.counterId && ['CALLED', 'SERVING'].includes(q.status)) &&
166 | | <button className="primary-btn" disabled={busy || counter?.status !== 'AVAILABLE' ||
167 | | !queue.some(q=>q.status==='WAITING')} || queue.some(q=>q.counterId===staff.counterId &&
168 | | ['CALLED', 'SERVING'].includes(q.status))} onClick={() => runAction(api.callNext)}>Summon next
169 | | student</button>}
170 |                 </div>
171 |                 <QueueTable staff={staff} queue={queue} busy={busy} onCall={(id) => runAction(() =>
172 | | api.callStudent(id))} onStart={(id) => runAction(() => api.startService(id))} onComplete={(id) =>
173 | | runAction(() => api.completeService(id))} onNoShow={(id) => {if(window.confirm('Mark this student
174 | | absent? First absence moves the ticket back up to ten places; second absence ends this
175 | | ticket.'))runAction(() => api.noShow(id))}} />

```

```

158 |         </div>
159 |         <div className="right-column">
160 |             <div className="panel current-panel">
161 |                 <div className="panel-head"><div><h2>Current Service</h2><p>Counter assignment</p></div><span
|                 className={`status status-${statusClass}`}>{counter?.status || '-'}</span></div>
162 |                 {currentService ? <div className="current-card"><div className="current-
|                 token">A{String(currentService.tokenNumber).padStart(3, '0')}</div><div><span>Queue
|                 #{currentService.queueId}</span><strong>{currentService.status === 'SERVING' ? 'Service in progress' :
|                 'Student called'}</strong></div></div> : <div className="empty-state">No student is currently being
|                 served.</div>}
163 |                 {currentService && <div className="current-student"><strong>{currentService.studentName ||
|                 'Student #${currentService.studentId}'}</strong><div>{currentService.studentEmail}</div><StudentPhone
|                 phone={currentService.studentPhone} name={currentService.studentName} /></div>}
164 |                 <div className="current-actions">
165 |                     {currentService?.status === 'CALLED' && <button className="primary-btn full"
|                     disabled={busy} onClick={() => runAction(() => api.startService(currentService.queueId))}>Start
|                     Service</button>}
166 |                     {currentService?.status === 'SERVING' && <button className="primary-btn full"
|                     disabled={busy} onClick={() => runAction(() => api.completeService(currentService.queueId))}>Complete
|                     Service</button>}
167 |                     {currentService?.status === 'CALLED' && <button className="secondary-btn full"
|                     disabled={busy} onClick={() => { if(window.confirm('Mark this student absent? First absence moves the
|                     ticket back up to ten places; second absence ends this ticket.'))runAction(() =>
|                     api.noShow(currentService.queueId)) }}>Mark absent</button>}
168 |                 </div>
169 |             </div>
170 |             <div className="panel activity-panel"><div className="panel-head"><div><h2>Recent
|             Activity</h2><p>Latest queue actions</p></div></div><ActivityFeed items={activity} /></div>
171 |             </div>
172 |         </section>
173 |     </div>
174 |
175 |     <section className="bottom-grid">
176 |         <div className="mini-panel"><span>Today Reservations</span><strong>{summary?.upcomingBookings ??
|         '-'}</strong><small>for this service</small></div>
177 |         <div className="mini-panel"><span>Counter Assignment</span><strong>{counter?.counterName ||
|         '-'}</strong><small>{counter?.serviceName || ''}</small></div>
178 |         <div className="mini-panel"><span>System</span><strong>Connected</strong><small>Auto-refresh
|         every 7 seconds</small></div>
179 |     </section>
180 | </>
181 | }
182 |
183 | function QueueManagementView({ staff, queue, waiting, counter, busy, runAction }) {
184 |     return <>
185 |         <section className="stats-grid compact-stats">
186 |             <StatCard label="Active Entries" value={queue.length} icon="\u25f7" />
187 |             <StatCard label="Waiting" value={waiting} icon="\u25c9" />
188 |             <StatCard label="Called" value={queue.filter(q => q.status === 'CALLED').length} icon="\u2197" />
189 |             <StatCard label="Serving" value={queue.filter(q => q.status === 'SERVING').length} icon="\u2713"
|             />
190 |         </section>
191 |         <section className="panel queue-panel full-panel">
192 |             <div className="panel-head"><div><h2>Queue Management</h2><p>Manage the live queue for
|             {counter?.serviceName || 'your assigned
|             service'}.</p></div>{!queue.some(q=>q.counterId===staff.counterId &&
|             ['CALLED', 'SERVING'].includes(q.status)) && <button className="primary-btn" disabled={busy ||
|             counter?.status !== 'AVAILABLE' || !queue.some(q=>q.status==='WAITING')} ||
|             queue.some(q=>q.counterId===staff.counterId && ['CALLED', 'SERVING'].includes(q.status))} onClick={() =>
|             runAction(api.callNext)}>Summon next student</button></div>
193 |             {queue.length ? <QueueTable staff={staff} queue={queue} busy={busy} onCall={(id) => runAction((
|             => api.callStudent(id))) onStart={(id) => runAction(() => api.startService(id))} onComplete={(id) =>
|             runAction(() => api.completeService(id))} onNoShow={(id) => {if(window.confirm('Mark this student
|             absent? First absence moves the ticket back up to ten places; second absence ends this
|             ticket.'))runAction(() => api.noShow(id))}} /> : <div className="empty-state large-empty"><strong>No
|             students are currently in the live queue.</strong><span>When a student joins this service, their token
|             will appear here automatically.</span></div>}
194 |             </div>
195 |         </section>
196 |     </>
197 | }

```

```

198 | function BookingsView({ summary, queue }) {
199 |   return <section className="page-grid">
200 |     <div className="panel info-panel">
201 |       <div className="panel-head"><h2>Today Reservations</h2><p>Bookings assigned to your
| service.</p></div><span className="metric-badge">{summary?.upcomingBookings ?? 0}</span></div>
202 |       <div className="table-
| wrap"><table><thead><tr><th>Token</th><th>Student</th><th>Status</th><th>Missed
| calls</th></tr></thead><tbody>{queue.map(q=><tr
|
|   key={q.queueId}><td>{String(q.tokenNumber).padStart(3, "0")}</td><td>Student
|   #{q.studentId}</td><td>{q.status}</td><td>{q.missedTurns}</td></tr>)}</tbody></table>{!queue.length &&
|   <div className="empty-state">No active reservations for today.</div></div>
203 |     </div>
204 |     <div className="panel info-panel"><h2>Booking workflow</h2><div className="workflow-
| list"><div><b>1</b></div><span>Student reserves a place in today's
| queue.</span></div><div><b>2</b></div><span>Reservations follow confirmation order within the
| service.</span></div><div><b>3</b></div><span>Counters open 10 AM-1 PM and 2-3
| PM.</span></div><div><b>4</b></div><span>First missed call moves back; second missed call ends the
| ticket.</span></div></div></div>
205 |   </section>
206 | }
207 |
208 | function AnalyticsView({ summary, activity, queue }) {
209 |   return <>
210 |     <section className="stats-grid">
211 |       <StatCard label="Students Served Today" value={summary?.servedToday} icon="\u2713" />
212 |       <StatCard label="Average Wait" value={summary?.averageWaitMinutes} suffix=" min" icon="\u25f7" />
213 |       <StatCard label="Average Service" value={summary?.averageServiceMinutes} suffix=" min"
| icon="\u2197" />
214 |       <StatCard label="No-shows Today" value={summary?.noShowsToday} icon="!" />
215 |       <StatCard label="Peak Hour" value={summary?.peakHour || '-'} icon="\u2301" />
216 |       <StatCard label="Current Queue" value={queue.length} icon="\u25c9" />
217 |     </section>
218 |     <section className="page-grid">
219 |       <div className="panel info-panel"><h2>Operational Snapshot</h2><div className="analytics-
| rows"><div><span>Students waiting</span><strong>{summary?.studentsWaiting ??
| 0}</strong></div><div><span>Upcoming bookings</span><strong>{summary?.upcomingBookings ??
| 0}</strong></div><div><span>Recent recorded
| actions</span><strong>{activity.length}</strong></div><div><span>Refresh interval</span><strong>7
| sec</strong></div></div></div>
220 |       <div className="panel info-panel"><h2>Recent Activity</h2><ActivityFeed items={activity} /></div>
221 |     </section>
222 |   </>
223 | }

```

frontend/src/pages/StaffLogin.jsx

Staff login form and error/busy handling. Uses the shared API adapter; it does not read MySQL directly.

Read these lines first

Line 1: Declares component state and a setter; changes cause React to render the relevant UI.

Line 5: Declares component state and a setter; changes cause React to render the relevant UI.

Line 6: Declares component state and a setter; changes cause React to render the relevant UI.

Line 7: Declares component state and a setter; changes cause React to render the relevant UI.

Line 8: Declares component state and a setter; changes cause React to render the relevant UI.

Line 9: Declares component state and a setter; changes cause React to render the relevant UI.

Line 32: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | import { useState } from 'react'
2 | import { api } from '../services/api'
3 |
4 | export default function StaffLogin({ onLogin }) {
5 |   const [email, setEmail] = useState('')
6 |   const [password, setPassword] = useState('')
7 |   const [showPassword, setShowPassword] = useState(false)
8 |   const [loading, setLoading] = useState(false)
9 |   const [error, setError] = useState('')
10 |
11 |   async function submit(event) {
12 |     event.preventDefault()
13 |     setError('')
14 |
15 |     if (!email.trim() || !password) {
16 |       setError('Enter both email and password.')
17 |       return
18 |     }
19 |
20 |     try {
21 |       setLoading(true)
22 |
23 |       // Step 1: authenticate and create the backend session
24 |       await api.login(email.trim(), password)
25 |
26 |       // Step 2: get the logged-in staff member from that session
27 |       const profile = await api.profile()
28 |
29 |       // Step 3: pass the real staff data to App.jsx
30 |       onLogin(profile.data)
31 |
32 |     } catch (err) {
33 |       setError(err.message)
34 |     } finally {
35 |       setLoading(false)
36 |     }
37 |   }
38 |
39 |   return (
40 |     <main className="login-page">
41 |       <section className="login-card">
42 |         <div className="cq-brand" aria-label="Campus Queue"><span className="cq-
| wordmark"><span>Campus</span><strong>Queue</strong></span></div>
43 |
44 |         <div className="eyebrow">CAMPUS QUEUE</div>
45 |         <h1>Staff Operations</h1>

```

```
45 |         <p className="login-subtitle">Sign in to manage your assigned service counter.</p>
46 |         <form onSubmit={submit}>
47 |             <label>Email</label>
48 |             <input value={email} onChange={(e) => setEmail(e.target.value)} type="email"
| placeholder="staff@college.edu" autoComplete="username" />
49 |             <label>Password</label>
50 |             <div className="password-wrap">
51 |                 <input value={password} onChange={(e) => setPassword(e.target.value)} type={showPassword ?
| 'text' : 'password'} placeholder="Enter password" autoComplete="current-password" />
52 |                 <button type="button" onClick={() => setShowPassword(v => !v)}>{showPassword ? 'Hide' :
| 'Show'}</button>
53 |             </div>
54 |             {error && <div className="error-box">{error}</div>}
55 |             <button className="login-btn" disabled={loading}>{loading ? 'Signing in...' : 'Sign
| in'}</button>
56 |         </form>
57 |         <div className="login-footer">Secure session-based staff access</div>
58 |     </section>
59 | </main>
60 | )
61 | }
```

frontend/src/services/api.js

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

Read these lines first

Line 4: Sends an HTTP request. Explain method, credentials, body encoding and handling of unsuccessful responses.

Line 5: Includes cookie credentials so the backend can restore the authenticated session through the proxy.

Line 15: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | const API_BASE = "/api";
2 |
3 | async function apiRequest(url, options = {}) {
4 |     const response = await fetch(`${API_BASE}${url}`, {
5 |         credentials: "include",
6 |         ...options,
7 |     });
8 |
9 |     const text = await response.text();
10 |
11 |     let data;
12 |
13 |     try {
14 |         data = JSON.parse(text);
15 |     } catch {
16 |         throw new Error(
17 |             `Server returned an invalid response (${response.status}).`
18 |         );
19 |     }
20 |
21 |     if (!response.ok || data.success === false) {
22 |         const error = new Error(data.message || "API request failed."); error.status = response.status;
23 |         throw error;
24 |     }
25 |     return data;
26 | }
27 |
28 | function formBody(values) {
29 |     return new URLSearchParams(values);
30 | }
31 |
32 | export const api = {
33 |     logout: () => apiRequest("/staff/logout", {method: "POST"}),
34 |     login: (email, password) =>
35 |         apiRequest("/staff/login", {
36 |             method: "POST",
37 |             headers: {
38 |                 "Content-Type": "application/x-www-form-urlencoded",
39 |             },
40 |             body: formBody({ email, password }),
41 |         }),
42 |
43 |     profile: () =>
44 |         apiRequest("/staff/profile"),
45 |
46 |     summary: () =>
47 |         apiRequest("/staff/dashboard-summary"),
48 |
49 |     queue: () =>
50 |         apiRequest("/staff/queue"),
51 |

```

```
52 | recentActivity: () =>
53 |   apiRequest("/staff/recent-activity"),
54 |
55 | counterStatus: () =>
56 |   apiRequest("/staff/counter/status"),
57 |
58 | setCounterStatus: (status) =>
59 |   apiRequest("/staff/counter/status", {
60 |     method: "POST",
61 |     headers: {
62 |       "Content-Type": "application/x-www-form-urlencoded",
63 |     },
64 |     body: formBody({ status }),
65 |   }),
66 |
67 | callNext: () =>
68 |   apiRequest("/queue/call-next", {
69 |     method: "POST",
70 |   }),
71 |
72 | callStudent: (queueId) =>
73 |   apiRequest("/queue/call", {
74 |     method: "POST",
75 |     headers: {
76 |       "Content-Type": "application/x-www-form-urlencoded",
77 |     },
78 |     body: formBody({ queueId }),
79 |   }),
80 |
81 | startService: (queueId) =>
82 |   queueAction("/start", queueId),
83 |
84 | completeService: (queueId) =>
85 |   queueAction("/complete", queueId),
86 |
87 | noShow: (queueId) => queueAction("/no-show", queueId),
88 |
89 | skipStudent: (queueId) =>
90 |   queueAction("/skip", queueId),
91 | };
92 | function queueAction(path, queueId) {
93 |   return apiRequest(`/queue${path}`, {
94 |     method: "POST",
95 |     headers: {
96 |       "Content-Type": "application/x-www-form-urlencoded",
97 |     },
98 |     body: formBody({ queueId }),
99 |   });
100 | }
```

scripts/netlify-redirects.mjs

Build-time HTTPS backend-origin validation and API-first rewrite generation, followed by SPA route fallback.

Numbered source

```
1 | import { writeFileSync } from 'node:fs';
2 |
3 | const backend = new URL(process.env.QUEUE_BACKEND_URL || '');
4 | if (backend.protocol !== 'https:' || backend.username || backend.password || backend.pathname !== '/')
5 |   throw new Error('Set QUEUE_BACKEND_URL to the HTTPS origin of the Render backend, without credentials
6 |   or a path.');
```

```
7 | writeFileSync('dist/_redirects', `api/* ${backend.origin}/api/:splat 200!\n/* /index.html 200\n`);
8 | console.log('Netlify API proxy and app routing configured.');
```

src/main/java/com/queue/controller/AnalyticsServlet.java

Staff analytics request adapter. Explains how authenticated service-specific statistics are exposed.

Read these lines first

Line 15: Registers this servlet URL mapping; explain which browser/API caller uses the route.

Line 24: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 26: Builds the response consumed by another layer. Identify which fields the frontend renders.

Line 32: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 36: Reads client-supplied input; validation and authorization are needed before using it.

Numbered source

```

1 | package com.queue.controller;
2 |
3 | import com.queue.service.AnalyticsService;
4 |
5 | import jakarta.servlet.ServletException;
6 | import jakarta.servlet.annotation.WebServlet;
7 | import jakarta.servlet.http.HttpServlet;
8 | import jakarta.servlet.http.HttpServletRequest;
9 | import jakarta.servlet.http.HttpServletResponse;
10 |
11 | import java.io.IOException;
12 | import java.io.PrintWriter;
13 | import java.util.Map;
14 |
15 | @WebServlet(urlPatterns = {"/analytics", "/api/analytics"})
16 | public class AnalyticsServlet extends HttpServlet {
17 |
18 |     private final AnalyticsService analyticsService = new AnalyticsService();
19 |
20 |     @Override
21 |     protected void doGet(HttpServletRequest request, HttpServletResponse response)
22 |         throws ServletException, IOException {
23 |
24 |         jakarta.servlet.http.HttpSession session=request.getSession(false);
25 |         if(session==null || !(session.getAttribute("staffId") instanceof Integer)) {
26 |             response.setStatus(401);response.setContentType("application/json");response.getWriter().wr
27 |             ite("{\"success\":false,\"message\":\"Staff is not logged in\"}");return;
28 |         }
29 |         com.queue.model.Staff staff=new
30 |         com.queue.service.StaffService().getStaffProfile((Integer)session.getAttribute("staffId"));
31 |         if(staff==null){response.setStatus(401);return;}
32 |         Map<String,Object> metrics;
33 |         try{metrics=analyticsService.getServiceDaySummary(staff.getServiceId());}
34 |         catch(java.sql.SQLException error){response.setStatus(503);return;}
35 |         Map<String,Object> data=new java.util.HashMap<>();
36 |         data.put("studentsServedToday",metrics.get("servedToday"));data.put("averageServiceTime",metric
37 |         s.get("averageServiceMinutes"));data.put("averageWaitingTime",metrics.get("averageWaitMinutes"));data.p
38 |         ut("peakHour",metrics.get("peakHour"));data.put("noShowCount",metrics.get("noShowsToday"));
39 |
40 |         String format = request.getParameter("format");
41 |         String acceptHeader = request.getHeader("Accept");
42 |         String requestedWith = request.getHeader("X-Requested-With");
43 |
44 |         boolean isJson = "json".equalsIgnoreCase(format)
45 |
46 |         || "/api/analytics".equals(request.getServletPath())
47 |         || "XMLHttpRequest".equalsIgnoreCase(requestedWith)
48 |         || (acceptHeader != null && acceptHeader.contains("application/json"));

```

```
45 |         if (isJson) {
46 |             response.setContentType("application/json");
47 |             response.setCharacterEncoding("UTF-8");
48 |
49 |             PrintWriter out = response.getWriter();
50 |             out.print("{ " +
51 |                 "\"studentsServedToday\": " + data.get("studentsServedToday") + ", " +
52 |                 "\"averageServiceTime\": " + data.get("averageServiceTime") + ", " +
53 |                 "\"averageWaitingTime\": " + data.get("averageWaitingTime") + ", " +
54 |                 "\"peakHour\": \"\" + data.get("peakHour") + "\", " +
55 |                 "\"noShowCount\": " + data.get("noShowCount") +
56 |                 "}");
57 |         } else {
58 |             request.setAttribute("studentsServedToday", data.get("studentsServedToday"));
59 |             request.setAttribute("averageServiceTime", data.get("averageServiceTime"));
60 |             request.setAttribute("averageWaitingTime", data.get("averageWaitingTime"));
61 |             request.setAttribute("peakHour", data.get("peakHour"));
62 |             request.setAttribute("noShowCount", data.get("noShowCount"));
63 |
64 |             request.getRequestDispatcher("/booking/analytics.jsp").forward(request, response);
65 |         }
66 |     }
67 | }
```

src/main/java/com/queue/controller/BookingServlet.java

Retained appointment HTTP surface. Current booking confirmation uses StudentPortalServlet and ReservationService.

Read these lines first

Line 19: Registers this servlet URL mapping; explain which browser/API caller uses the route.

Line 42: Reads client-supplied input; validation and authorization are needed before using it.

Line 82: Reads client-supplied input; validation and authorization are needed before using it.

Line 83: Reads client-supplied input; validation and authorization are needed before using it.

Line 100: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 112: Reads client-supplied input; validation and authorization are needed before using it.

Line 113: Reads client-supplied input; validation and authorization are needed before using it.

Line 114: Reads client-supplied input; validation and authorization are needed before using it.

Numbered source

```

1 | package com.queue.controller;
2 |
3 | import com.queue.model.Booking;
4 | import com.queue.service.BookingService;
5 | import com.queue.service.NoShowService;
6 |
7 | import jakarta.servlet.ServletException;
8 | import jakarta.servlet.annotation.WebServlet;
9 | import jakarta.servlet.http.HttpServlet;
10 | import jakarta.servlet.http.HttpServletRequest;
11 | import jakarta.servlet.http.HttpServletResponse;
12 |
13 | import java.io.IOException;
14 | import java.io.PrintWriter;
15 | import java.sql.Date;
16 | import java.sql.Time;
17 | import java.util.List;
18 |
19 | @WebServlet(urlPatterns = {
20 |     "/create-booking",
21 |     "/cancel-booking",
22 |     "/booking-status",
23 |     "/api/bookings/create",
24 |     "/api/bookings/cancel",
25 |     "/api/bookings/status",
26 |     "/api/bookings/available-slots",
27 |     "/api/bookings/upcoming"
28 | })
29 | public class BookingServlet extends HttpServlet {
30 |
31 |     private final BookingService bookingService = new BookingService();
32 |     private final NoShowService noShowService = new NoShowService();
33 |
34 |     @Override
35 |     protected void doGet(HttpServletRequest request, HttpServletResponse response)
36 |         throws ServletException, IOException {
37 |
38 |         // Auto-check and release any no-show bookings before handling request
39 |         // Legacy appointments are retained as history; new visits use daily queue reservations.
40 |
41 |         String servletPath = request.getServletPath();
42 |         String action = request.getParameter("action");
43 |
44 |         if ("/api/bookings/available-slots".equals(servletPath) || "available-
45 |             | slots".equalsIgnoreCase(action)) {
46 |             response.setStatus(410); response.setContentType("application/json"); response.getWriter().wr
47 |             | ite("{\"success\":false,\"message\":\"Time slots were replaced by daily queue reservations.\"}");
48 |             } else if ("/api/bookings/upcoming".equals(servletPath) || "upcoming".equalsIgnoreCase(action))
49 |             | {
50 |                 handleUpcomingBookings(request, response);
51 |             } else if ("/booking-status".equals(servletPath) || "/api/bookings/status".equals(servletPath))
52 |             | {
53 |                 handleBookingStatus(request, response);
54 |             } else {
55 |                 // Forward to pre-booking page by default
56 |                 request.getRequestDispatcher("/booking/pre-booking.jsp").forward(request, response);
57 |             }
58 |         }
59 |
60 |     @Override
61 |     protected void doPost(HttpServletRequest request, HttpServletResponse response)
62 |         throws ServletException, IOException {
63 |
64 |         // Auto-check and release no-shows
65 |         // Do not mark historic slot records absent on a page visit.
66 |
67 |         String servletPath = request.getServletPath();
68 |
69 |         if ("/create-booking".equals(servletPath) || "/api/bookings/create".equals(servletPath)) {

```

```

66 |         response.setStatus(410);response.setContentType("application/json");response.getWriter().wr
67 | ite("{\"success\":false,\"message\":\"Use the student daily reservation flow.\"}");
68 |     } else if ("/cancel-booking".equals(servletPath) || "/api/bookings/cancel".equals(servletPath))
69 |     {
70 |         handleCancelBooking(request, response);
71 |     } else {
72 |         doGet(request, response);
73 |     }
74 |
75 | private void handleAvailableSlots(HttpServletRequest request, HttpServletResponse response)
76 |     throws IOException {
77 |
78 |     response.setContentType("application/json");
79 |     response.setCharacterEncoding("UTF-8");
80 |     PrintWriter out = response.getWriter();
81 |
82 |     try {
83 |         int serviceId = Integer.parseInt(request.getParameter("serviceId"));
84 |
85 |         String dateStr = request.getParameter("date");
86 |         Date bookingDate = (dateStr != null && !dateStr.trim().isEmpty())
87 |             ? Date.valueOf(dateStr.trim())
88 |             : new Date(System.currentTimeMillis());
89 |
90 |         List<String> slots = bookingService.getAvailableSlots(serviceId, bookingDate);
91 |
92 |         StringBuilder json = new StringBuilder("[");
93 |         for (int i = 0; i < slots.size(); i++) {
94 |             json.append("\"").append(slots.get(i)).append("\");
95 |             if (i < slots.size() - 1) {
96 |                 json.append(",");
97 |             }
98 |         }
99 |         json.append("]");
100 |
101 |         out.print(json.toString());
102 |     } catch (Exception e) {
103 |         response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
104 |         out.print("{\"error\":\"\" + escapeJson(e.getMessage()) + "\"}");
105 |     }
106 |
107 | private void handleCreateBooking(HttpServletRequest request, HttpServletResponse response)
108 |     throws ServletException, IOException {
109 |
110 |     boolean isAjax = isAjaxRequest(request);
111 |
112 |     try {
113 |         int studentId = Integer.parseInt(request.getParameter("studentId"));
114 |         int serviceId = Integer.parseInt(request.getParameter("serviceId"));
115 |         String dateStr = request.getParameter("bookingDate");
116 |         String timeStr = request.getParameter("bookingTime");
117 |
118 |         if (timeStr != null && timeStr.length() == 5) {
119 |             timeStr += ":00"; // format HH:mm to HH:mm:ss
120 |         }
121 |
122 |         Booking booking = new Booking();
123 |         booking.setStudentId(studentId);
124 |         booking.setServiceId(serviceId);
125 |         booking.setBookingDate(Date.valueOf(dateStr));
126 |         booking.setBookingTime(Time.valueOf(timeStr));
127 |
128 |         String result = bookingService.createBooking(booking);
129 |
130 |         if ("SUCCESS".equalsIgnoreCase(result)) {
131 |             if (isAjax) {
132 |                 response.setContentType("application/json");
133 |                 response.getWriter().print("{\"success\":true,\"message\":\"Booking confirmed
134 | successfully!\",\"bookingId\":\" + booking.getBookingId() + \"}");

```

```

133 |         } else {
134 |             request.setAttribute("message", "Booking confirmed successfully! Booking ID: " +
| booking.getBookingId());
135 |             request.setAttribute("booking", booking);
136 |             response.sendRedirect(request.getContextPath() + "/booking-status?studentId=" +
| studentId);
137 |         }
138 |     } else {
139 |         if (isAjax) {
140 |             response.setContentType("application/json");
141 |             response.getWriter().print("{\"success\":false,\"message\":\"" + escapeJson(result)
| + "\"}");
142 |         } else {
143 |             request.setAttribute("errorMessage", result);
144 |             request.getRequestDispatcher("/booking/pre-booking.jsp").forward(request,
| response);
145 |         }
146 |     }
147 |
148 | } catch (Exception e) {
149 |     if (isAjax) {
150 |         response.setContentType("application/json");
151 |         response.getWriter().print("{\"success\":false,\"message\":\"Invalid input: " +
| escapeJson(e.getMessage()) + "\"}");
152 |     } else {
153 |         request.setAttribute("errorMessage", "Error processing booking: " + e.getMessage());
154 |         request.getRequestDispatcher("/booking/pre-booking.jsp").forward(request, response);
155 |     }
156 | }
157 | }
158 |
159 | private void handleCancelBooking(HttpServletRequest request, HttpServletResponse response)
160 |     throws ServletException, IOException {
161 |
162 |     boolean isAjax = isAjaxRequest(request);
163 |
164 |     try {
165 |
166 |         int bookingId = Integer.parseInt(request.getParameter("bookingId"));
167 |         int studentId = 0;
168 |         if (request.getParameter("studentId") != null &&
| !request.getParameter("studentId").trim().isEmpty()) {
169 |             studentId = Integer.parseInt(request.getParameter("studentId").trim());
170 |         }
171 |
172 |         boolean cancelled = bookingService.cancelBooking(bookingId, studentId);
173 |
174 |         if (isAjax) {
175 |             response.setContentType("application/json");
176 |             if (cancelled) {
177 |                 response.getWriter().print("{\"success\":true,\"message\":\"Booking #" + bookingId
| + " has been cancelled successfully.\"}");
178 |             } else {
179 |                 response.getWriter().print("{\"success\":false,\"message\":\"Failed to cancel
| booking. It may already be cancelled or invalid.\"}");
180 |             }
181 |         } else {
182 |             if (cancelled) {
183 |                 request.setAttribute("message", "Booking #" + bookingId + " has been cancelled.");
184 |             } else {
185 |                 request.setAttribute("errorMessage", "Could not cancel booking #" + bookingId +
| ".");
186 |             }
187 |             if (studentId > 0) {
188 |                 response.sendRedirect(request.getContextPath() + "/booking-status?studentId=" +
| studentId);
189 |             } else {
190 |                 response.sendRedirect(request.getContextPath() + "/booking-status");
191 |             }
192 |         }
193 |     } catch (Exception e) {

```

```

194 |         if (isAjax) {
195 |             response.setContentType("application/json");
196 |             response.getWriter().print("{\"success\":false,\"message\":\"Error: \" +
| escapeJson(e.getMessage()) + "\"}");
197 |         } else {
198 |             request.setAttribute("errorMessage", "Invalid request: " + e.getMessage());
199 |             request.getRequestDispatcher("/booking/booking-status.jsp").forward(request, response);
200 |         }
201 |     }
202 | }

203 |
204 | private void handleUpcomingBookings(HttpServletRequest request, HttpServletResponse response)
205 |     throws IOException {
206 |
207 |     response.setContentType("application/json");
208 |     response.setCharacterEncoding("UTF-8");
209 |     PrintWriter out = response.getWriter();
210 |
211 |     try {
212 |         int serviceId = Integer.parseInt(request.getParameter("serviceId"));
213 |         String dateStr = request.getParameter("date");
214 |
215 |         List<Booking> upcoming;
216 |         if (dateStr != null && !dateStr.trim().isEmpty()) {
217 |             Date bookingDate = Date.valueOf(dateStr.trim());
218 |             upcoming = bookingService.getUpcomingBookings(serviceId, bookingDate);
219 |         } else {
220 |             upcoming = bookingService.getUpcomingBookings(serviceId);
221 |         }
222 |
223 |         StringBuilder sb = new StringBuilder("[");
224 |         for (int i = 0; i < upcoming.size(); i++) {
225 |             Booking b = upcoming.get(i);
226 |             sb.append(formatBookingJson(b));
227 |             if (i < upcoming.size() - 1) sb.append(",");
228 |         }
229 |         sb.append("]");
230 |
231 |         out.print(sb.toString());
232 |     } catch (Exception e) {
233 |         response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
234 |         out.print("{\"error\":\"\" + escapeJson(e.getMessage()) + "\"}");
235 |     }
236 | }

237 |
238 | private void handleBookingStatus(HttpServletRequest request, HttpServletResponse response)
239 |     throws ServletException, IOException {
240 |
241 |     String studentIdParam = request.getParameter("studentId");
242 |     String bookingIdParam = request.getParameter("bookingId");
243 |     boolean isAjax = isAjaxRequest(request);
244 |
245 |     List<Booking> bookings = null;
246 |     Booking singleBooking = null;

247 |
248 |     if (studentIdParam != null && !studentIdParam.trim().isEmpty()) {
249 |         try {
250 |             int studentId = Integer.parseInt(studentIdParam.trim());
251 |             bookings = bookingService.getStudentBookings(studentId);
252 |             request.setAttribute("studentId", studentId);
253 |         } catch (NumberFormatException ignored) {}
254 |     } else if (bookingIdParam != null && !bookingIdParam.trim().isEmpty()) {
255 |         try {
256 |             int bookingId = Integer.parseInt(bookingIdParam.trim());
257 |             singleBooking = bookingService.getBookingById(bookingId);
258 |             request.setAttribute("bookingId", bookingId);
259 |         } catch (NumberFormatException ignored) {}
260 |     }

261 |
262 |     if (isAjax) {

```

```

263 |         response.setContentType("application/json");
264 |         response.setCharacterEncoding("UTF-8");
265 |         PrintWriter out = response.getWriter();
266 |
267 |         if (bookings != null) {
268 |             StringBuilder sb = new StringBuilder("[");
269 |             for (int i = 0; i < bookings.size(); i++) {
270 |                 Booking b = bookings.get(i);
271 |                 sb.append(formatBookingJson(b));
272 |                 if (i < bookings.size() - 1) sb.append(",");
273 |             }
274 |             sb.append("]");
275 |             out.print(sb.toString());
276 |         } else if (singleBooking != null) {
277 |             out.print(formatBookingJson(singleBooking));
278 |         } else {
279 |             out.print("[]");
280 |         }
281 |     } else {
282 |         request.setAttribute("bookings", bookings);
283 |         request.setAttribute("singleBooking", singleBooking);
284 |         request.getRequestDispatcher("/booking/booking-status.jsp").forward(request, response);
285 |     }
286 | }
287 |
288 | private boolean isAjaxRequest(HttpServletRequest request) {
289 |     String requestedWith = request.getHeader("X-Requested-With");
290 |     String acceptHeader = request.getHeader("Accept");
291 |
292 |     String format = request.getParameter("format");
293 |     return "XMLHttpRequest".equalsIgnoreCase(requestedWith)
294 |         || (acceptHeader != null && acceptHeader.contains("application/json"))
295 |         || "json".equalsIgnoreCase(format);
296 | }
297 | private String formatBookingJson(Booking b) {
298 |     return "{" +
299 |         "\"bookingId\":\"" + b.getBookingId() + \",\" +
300 |         "\"studentId\":\"" + b.getStudentId() + \",\" +
301 |         "\"serviceId\":\"" + b.getServiceId() + \",\" +
302 |         "\"bookingDate\":\"" + (b.getBookingDate() != null ? b.getBookingDate().toString() :
303 |         "\"\"") + \",\" +
304 |         "\"bookingTime\":\"" + (b.getBookingTime() != null ? b.getBookingTime().toString() :
305 |         "\"\"") + \",\" +
306 |         "\"status\":\"" + (b.getStatus() != null ? b.getStatus() : "\"\"") + "\"\" +
307 |         "}";
308 | }
309 | private String escapeJson(String str) {
310 |     if (str == null) return "";
311 |     return str.replace("\\", "\\").replace("\"", "\\").replace("\n", "\\n").replace("\r",
312 |     "\\r");

```

src/main/java/com/queue/controller/QueueServlet.java

a

Queue action HTTP adapter. Authenticates staff and delegates actions into ReservationService; retained routes require care.

Read these lines first

Line 16: Registers this servlet URL mapping; explain which browser/API caller uses the route.

Line 39: Builds the response consumed by another layer. Identify which fields the frontend renders.

Line 42: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 49: Builds the response consumed by another layer. Identify which fields the frontend renders.

Line 50: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 51: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 57: Reads client-supplied input; validation and authorization are needed before using it.

Line 60: Reads client-supplied input; validation and authorization are needed before using it.

Numbered source

```

1 | package com.queue.controller;
2 |
3 | import com.queue.model.Queue;
4 | import com.queue.service.StaffService;
5 | import com.queue.service.StudentService;
6 |
7 | import jakarta.servlet.ServletException;
8 | import jakarta.servlet.annotation.WebServlet;
9 | import jakarta.servlet.http.HttpServlet;
10 | import jakarta.servlet.http.HttpServletRequest;
11 | import jakarta.servlet.http.HttpServletResponse;
12 | import jakarta.servlet.http.HttpSession;
13 |
14 | import java.io.IOException;
15 |
16 | @WebServlet("/api/queue/*")
17 | public class QueueServlet extends HttpServlet {
18 |
19 |     private StaffService staffService;
20 |     private StudentService studentService;
21 |
22 |     @Override
23 |     public void init() {
24 |         staffService = new StaffService();
25 |         studentService = new StudentService();
26 |     }
27 |
28 |     // =====
29 |     // POST QUEUE OPERATIONS
30 |     // =====
31 |
32 |     @Override
33 |     protected void doPost(HttpServletRequest request,
34 |                           HttpServletResponse response)
35 |         throws ServletException, IOException {
36 |
37 |         String path = request.getPathInfo();
38 |
39 |         if ("/skip".equals(path)) {response.setStatus(410);response.setContentType("application/json");
40 | response.getWriter().write("{\"success\":false,\"message\":\"Skip is retired. Use confirmed missed call
41 | for absent students.\"}");return;}
42 |         if (java.util.Set.of("/call-next", "/call", "/start", "/complete", "/skip", "/no-
43 | show").contains(java.util.Objects.toString(path, ""))) {
44 |             response.setContentType("application/json");response.setCharacterEncoding("UTF-8");
45 |
46 |             HttpSession actor=request.getSession(false);
47 |             if(actor==null || !(actor.getAttribute("staffId") instanceof Integer)) {
48 |                 response.setStatus(401);writeError(response,"Staff is not logged in");return;
49 |             }
50 |             try {
51 |                 Object result=new com.queue.service.ReservationService().operate((Integer)actor.getAttr
52 | ibute("staffId"),path.substring(1),getQueueId(request));
53 |                 if(result==null){response.setStatus(409);writeError(response,"No students are currently
54 | waiting");return;}
55 |                 response.getWriter().write(com.queue.util.Json.encode(java.util.Map.of("success", true, "
56 | data", result)));return;
57 |             }catch(IllegalArgumentException
58 | error){response.setStatus(409);writeError(response,error.getMessage());return;}
59 |             catch(java.sql.SQLException error){response.setStatus(503);writeError(response,"The
60 | operation could not be saved. Please try again.");return;}
61 |         }
62 |
63 |         if ("/join".equals(path)) {
64 |
65 |             String studentIdParameter =
66 |                 request.getParameter("studentId");
67 |
68 |             String serviceIdParameter =
69 |                 request.getParameter("serviceId");
70 |

```

```

62 |     if (studentIdParameter == null ||
63 |         serviceIdParameter == null) {
64 |
65 |         writeError(
66 |             response,
67 |             "studentId and serviceId are required"
68 |         );
69 |
70 |         return;
71 |     }
72 |
73 |     try {
74 |
75 |         int studentId =
76 |             Integer.parseInt(studentIdParameter);
77 |
78 |         int serviceId =
79 |             Integer.parseInt(serviceIdParameter);
80 |
81 |
82 |         Queue queue = new Queue();
83 |
84 |         queue.setStudentId(studentId);
85 |         queue.setServiceId(serviceId);
86 |
87 |         boolean success =
88 |             studentService.joinQueue(queue);
89 |
90 |         if (success) {
91 |
92 |             response.getWriter().write(
93 |                 "{"
94 |                 + "\"success\":true,"
95 |                 + "\"message\":\"Joined queue successfully\","
96 |                 + "\"data\":{\""
97 |                 + "\"queueId\":\"" + queue.getQueueId() + ","
98 |                 + "\"studentId\":\"" + queue.getStudentId() + ","
99 |                 + "\"serviceId\":\"" + queue.getServiceId() + ","
100 |                 + "\"tokenNumber\":\"" + queue.getTokenNumber() + ","
101 |                 + "\"status\":\"" + queue.getStatus() + "\""
102 |                 + "}"
103 |             );
104 |
105 |         } else {
106 |
107 |             writeError(
108 |                 response,
109 |                 "Unable to join queue"
110 |             );
111 |         }
112 |
113 |     } catch (NumberFormatException e) {
114 |
115 |         writeError(
116 |             response,
117 |             "studentId and serviceId must be valid numbers"
118 |         );
119 |     }
120 |
121 |     return;
122 | }
123 |
124 |     response.setContentType("application/json");
125 |
126 |     response.setCharacterEncoding("UTF-8");
127 |
128 |     // =====
129 |     // CALL SPECIFIC STUDENT
130 |     // POST /api/queue/call
131 |     // =====

```

```

132 | if ("/call".equals(path)) {
133 |
134 |     int queueId = getQueueId(request);
135 |
136 |     HttpSession session = request.getSession(false);
137 |
138 |     if (session == null ||
139 |         session.getAttribute("staffId") == null) {
140 |
141 |         writeError(response, "Staff is not logged in");
142 |         return;
143 |     }
144 |
145 |     int staffId =
146 |         (Integer) session.getAttribute("staffId");
147 |
148 |     if (queueId <= 0) {
149 |
150 |         writeError(
151 |             response,
152 |             "Valid queueId is required"
153 |         );
154 |
155 |         return;
156 |     }
157 |
158 |     Queue queue =
159 |         staffService.callStudent(
160 |             staffId,
161 |             queueId
162 |         );
163 |
164 |     if (queue == null) {
165 |
166 |         writeError(
167 |             response,
168 |             "Unable to call student"
169 |
170 |         );
171 |
172 |         return;
173 |     }
174 |
175 |     String json =
176 |         "{"
177 |         + "\"success\":true,"
178 |         + "\"message\":\"Student called successfully\","
179 |         + "\"data\":{\""
180 |         + "\"queueId\":\"" + queue.getQueueId() + ","
181 |         + "\"studentId\":\"" + queue.getStudentId() + ","
182 |         + "\"serviceId\":\"" + queue.getServiceId() + ","
183 |         + "\"tokenNumber\":\"" + queue.getTokenNumber() + ","
184 |         + "\"status\":\"" + queue.getStatus() + "\""
185 |         + "}"
186 |         + "}";
187 |
188 |     response.getWriter().write(json);
189 |     return;
190 | }
191 |
192 | // =====
193 | // CALL NEXT
194 | // POST /api/queue/call-next
195 | // =====
196 |
197 | if ("/call-next".equals(path)) {
198 |
199 |     HttpSession session = request.getSession(false);
200 |
201 |     if (session == null ||
202 |         session.getAttribute("staffId") == null) {

```

```

203 |         writeError(response, "Staff is not logged in");
204 |         return;
205 |     }
206 |
207 |     int staffId =
208 |         (Integer) session.getAttribute("staffId");
209 |
210 |     com.queue.dao.StaffDAO staffDAO =
211 |         new com.queue.dao.StaffDAO();
212 |
213 |
214 |     com.queue.model.Staff staff =
215 |         staffDAO.getStaffById(staffId);
216 |
217 |     if (staff == null) {
218 |         writeError(response, "Staff profile not found");
219 |         return;
220 |     }
221 |
222 |     Queue queue =
223 |         staffService.callNextStudent(
224 |             staff.getServiceId());
225 |
226 |     if (queue == null) {
227 |
228 |         writeError(
229 |             response,
230 |             "No students are currently waiting"
231 |         );
232 |         return;
233 |     }
234 |
235 |     String json =
236 |         "{"
237 |         + "\"success\":true,"
238 |         + "\"message\":\"Next student called\","
239 |         + "\"data\":{\""
240 |         + "\"queueId\":\"" + queue.getQueueId() + ","
241 |         + "\"studentId\":\"" + queue.getStudentId() + ","
242 |         + "\"serviceId\":\"" + queue.getServiceId() + ","
243 |         + "\"tokenNumber\":\"" + queue.getTokenNumber() + ","
244 |         + "\"status\":\"" + queue.getStatus() + "\",
245 |         + "\"counterId\":\"" + staff.getCounterId()
246 |         + "\""
247 |         + "}"
248 |
249 |     response.getWriter().write(json);
250 |     return;
251 | }
252 |
253 | // =====
254 | // START SERVICE
255 | // POST /api/queue/start
256 | // =====
257 |
258 | if ("/start".equals(path)) {
259 |
260 |     int queueId = getQueueId(request);
261 |     HttpSession session = request.getSession(false);
262 |
263 |     if (session == null ||
264 |         session.getAttribute("staffId") == null) {
265 |         writeError(response, "Staff is not logged in");
266 |         return;
267 |     }
268 |
269 |     int staffId = (Integer) session.getAttribute("staffId");
270 |
271 |     if (queueId <= 0) {
272 |         writeError(response, "Valid queueId is required");

```

```

273 |         return;
274 |     }
275 |
276 |     boolean success =
277 |     staffService.startService(staffId, queueId);
278 |
279 |     if (success) {
280 |
281 |         writeSuccess(
282 |             response,
283 |             "Service started"
284 |         );
285 |
286 |     } else {
287 |
288 |         writeError(
289 |             response,
290 |             "Unable to start service"
291 |         );
292 |     }
293 |
294 |     return;
295 | }
296 |
297 | // =====
298 | // COMPLETE SERVICE
299 | // POST /api/queue/complete
300 | // =====
301 |
302 | if ("/complete".equals(path)) {
303 |
304 |     int queueId = getQueueId(request);
305 |     HttpSession session = request.getSession(false);
306 |
307 | if (session == null ||
308 |     session.getAttribute("staffId") == null) {
309 |     writeError(response, "Staff is not logged in");
310 |     return;
311 | }
312 |
313 | int staffId = (Integer) session.getAttribute("staffId");
314 |
315 |     if (queueId <= 0) {
316 |         writeError(response, "Valid queueId is required");
317 |         return;
318 |     }
319 |
320 |     boolean success =
321 |     staffService.completeService(staffId, queueId);
322 |     if (success) {
323 |
324 |         writeSuccess(
325 |             response,
326 |             "Service completed"
327 |         );
328 |
329 |     } else {
330 |
331 |         writeError(
332 |             response,
333 |             "Unable to complete service"
334 |         );
335 |     }
336 |
337 |     return;
338 | }
339 |
340 | // =====
341 | // SKIP STUDENT
342 | // POST /api/queue/skip
343 | // =====

```

```

344 |
345 |         if ("/skip".equals(path)) {
346 |
347 |             int queueId = getQueueId(request);
348 |             HttpSession session = request.getSession(false);
349 |
350 | if (session == null ||
351 |     session.getAttribute("staffId") == null) {
352 |     writeError(response, "Staff is not logged in");
353 |     return;
354 | }
355 |
356 | int staffId = (Integer) session.getAttribute("staffId");
357 |
358 |         if (queueId <= 0) {
359 |             writeError(response, "Valid queueId is required");
360 |             return;
361 |         }
362 |
363 |         boolean success =
364 | staffService.skipStudent(staffId, queueId);
365 |
366 |         if (success) {
367 |
368 |             writeSuccess(
369 |                 response,
370 |                 "Student skipped"
371 |             );
372 |
373 |         } else {
374 |
375 |             writeError(
376 |                 response,
377 |                 "Unable to skip student"
378 |             );
379 |         }
380 |
381 |         return;
382 |     }
383 |
384 |     writeError(response, "Invalid queue endpoint");
385 | }
386 |
387 | // =====
388 | // GET QUEUE STATUS
389 |
390 | // =====
391 | @Override
392 | protected void doGet(HttpServletRequest request,
393 |                     HttpServletResponse response)
394 |     throws ServletException, IOException {
395 |
396 |     response.setContentType("application/json");
397 |     response.setCharacterEncoding("UTF-8");
398 |
399 |     String path = request.getPathInfo();
400 |
401 |     // GET /api/queue/status
402 |     if ("/status".equals(path)) {
403 |
404 |         String studentIdParameter =
405 |             request.getParameter("studentId");
406 |
407 |         String serviceIdParameter =
408 |             request.getParameter("serviceId");
409 |
410 |         if (studentIdParameter == null ||
411 |             serviceIdParameter == null) {
412 |
413 |             writeError(

```

```

414 |         response,
415 |         "studentId and serviceId are required"
416 |     );
417 |
418 |     return;
419 | }
420 |
421 | try {
422 |
423 |     int studentId =
424 |         Integer.parseInt(studentIdParameter);
425 |
426 |     int serviceId =
427 |         Integer.parseInt(serviceIdParameter);
428 |
429 |     Queue queue =
430 |         studentService.getQueueStatus(
431 |             studentId,
432 |             serviceId
433 |
434 |         );
435 |
436 |     if (queue != null) {
437 |
438 |         response.getWriter().write(
439 |             "{"
440 |             + "\"success\":true,"
441 |             + "\"message\": \"Queue status retrieved successfully\","
442 |             + "\"data\":{"
443 |             + "\"queueId\": " + queue.getQueueId() + ","
444 |             + "\"studentId\": " + queue.getStudentId() + ","
445 |             + "\"serviceId\": " + queue.getServiceId() + ","
446 |             + "\"tokenNumber\": " + queue.getTokenNumber() + ","
447 |             + "\"status\": \" " + queue.getStatus() + "\"
448 |             + "}"
449 |         );
450 |
451 |     } else {
452 |
453 |         writeError(
454 |             response,
455 |             "No queue entry found"
456 |         );
457 |     }
458 |
459 | } catch (NumberFormatException e) {
460 |
461 |     writeError(
462 |         response,
463 |         "studentId and serviceId must be valid numbers"
464 |     );
465 | }
466 |
467 | return;
468 | }
469 |
470 | // GET /api/queue/position
471 | if ("/position".equals(path)) {
472 |
473 |     String positionStudentIdParameter =
474 |         request.getParameter("studentId");
475 |
476 |     String positionServiceIdParameter =
477 |
478 |         request.getParameter("serviceId");
479 |
480 |     if (positionStudentIdParameter == null ||
481 |         positionServiceIdParameter == null) {
482 |
483 |         writeError(
484 |             response,

```

```
484 |         "studentId and serviceId are required"
485 |     );
486 |
487 |     return;
488 | }
489 |
490 | try {
491 |
492 |     int studentId =
493 |         Integer.parseInt(
494 |             positionStudentIdParameter
495 |         );
496 |
497 |     int serviceId =
498 |         Integer.parseInt(
499 |             positionServiceIdParameter
500 |         );
501 |
502 |     int position =
503 |         studentService.getQueuePosition(
504 |             studentId,
505 |             serviceId
506 |         );
507 |
508 |     response.getWriter().write(
509 |         "{"
510 |         + "\"success\":true,"
511 |         + "\"message\":\"Queue position retrieved successfully\","
512 |         + "\"data\":{\""
513 |         + "\"studentId\":\"" + studentId + ","
514 |         + "\"serviceId\":\"" + serviceId + ","
515 |         + "\"position\":\"" + position
516 |         + "\""
517 |         + "}"
518 |     );
519 |
520 | } catch (NumberFormatException e) {
521 |
522 |     writeError(
523 |         response,
524 |         "studentId and serviceId must be valid numbers"
525 |     );
526 | }
527 |
528 | return;
529 | }
530 |
531 | // GET /api/queue/current-token
532 | if ("/current-token".equals(path)) {
533 |
534 |     String currentTokenServiceIdParameter =
535 |         request.getParameter("serviceId");
536 |
537 |     if (currentTokenServiceIdParameter == null) {
538 |
539 |         writeError(
540 |             response,
541 |             "serviceId is required"
542 |         );
543 |
544 |         return;
545 |     }
546 |
547 |     try {
548 |
549 |         int serviceId =
550 |             Integer.parseInt(
551 |                 currentTokenServiceIdParameter
552 |             );
553 |
554 |         int currentToken =
```

```

555 |         studentService.getCurrentToken(
556 |             serviceId
557 |         );
558 |
559 |         response.getWriter().write(
560 |             "{"
561 |             + "\"success\":true,"
562 |             + "\"message\": \"Current token retrieved successfully\","
563 |             + "\"data\":{"
564 |             + "\"serviceId\": " + serviceId + ","
565 |
566 |             + "\"currentToken\": " + currentToken
567 |             + "}"
568 |         );
569 |
570 |     } catch (NumberFormatException e) {
571 |
572 |         writeError(
573 |             response,
574 |             "serviceId must be a valid number"
575 |         );
576 |     }
577 |
578 |     return;
579 | }
580 |
581 | writeError(
582 |     response,
583 |     "GET operation not available for this endpoint"
584 | );
585 | }
586 |
587 | // =====
588 | // GET QUEUE ID
589 | // =====
590 |
591 | private int getQueueId(HttpServletRequest request) {
592 |
593 |     String queueIdParameter =
594 |         request.getParameter("queueId");
595 |
596 |     if (queueIdParameter == null ||
597 |         queueIdParameter.isBlank()) {
598 |
599 |         return -1;
600 |     }
601 |
602 |     try {
603 |
604 |         return Integer.parseInt(queueIdParameter);
605 |
606 |     } catch (NumberFormatException e) {
607 |
608 |         return -1;
609 |
610 |     }
611 | }
612 |
613 | // =====
614 | // SUCCESS RESPONSE
615 | // =====
616 |
617 | private void writeSuccess(
618 |     HttpServletResponse response,
619 |     String message)
620 |     throws IOException {
621 |
622 |     response.getWriter().write(
623 |         "{"
624 |         + "\"success\":true,"

```

```
625 |         + "\"data\":null"
626 |         + "}"
627 |     );
628 | }
629 |
630 | // =====
631 | // ERROR RESPONSE
632 | // =====
633 |
634 | private void writeError(
635 |     HttpServletResponse response,
636 |     String message)
637 |     throws IOException {
638 |
639 |     response.getWriter().write(
640 |         "{"
641 |         + "\"success\":false,"
642 |         + "\"message\":\"" + message + "\", "
643 |         + "\"data\":null"
644 |         + "}"
645 |     );
646 | }
647 | }
```

src/main/java/com/queue/controller/ServiceServlet.java

a

Public catalogue adapter. Returns configured services; catches DAO failure as HTTP 503 and writes safe diagnostics to stderr.

Read these lines first

Line 15: Registers this servlet URL mapping; explain which browser/API caller uses the route.

Line 36: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | package com.queue.controller;
2 |
3 | import com.queue.model.Service;
4 | import com.queue.service.ServiceService;
5 |
6 | import jakarta.servlet.ServletException;
7 | import jakarta.servlet.annotation.WebServlet;
8 | import jakarta.servlet.http.HttpServlet;
9 | import jakarta.servlet.http.HttpServletRequest;
10 | import jakarta.servlet.http.HttpServletResponse;
11 |
12 | import java.io.IOException;
13 | import java.util.List;
14 |
15 | @WebServlet("/api/services")
16 | public class ServiceServlet extends HttpServlet {
17 |
18 |     private ServiceService serviceService;
19 |
20 |     @Override
21 |     public void init() {
22 |         serviceService = new ServiceService();
23 |     }
24 |
25 |     @Override
26 |     protected void doGet(HttpServletRequest request,
27 |                           HttpServletResponse response)
28 |         throws ServletException, IOException {
29 |
30 |         response.setContentType("application/json");
31 |         response.setCharacterEncoding("UTF-8");
32 |
33 |         List<Service> services;
34 |         try {
35 |             services = serviceService.getAllServices();
36 |         } catch (IllegalStateException failure) {
37 |             Throwable cause = failure.getCause();
38 |             String diagnostic = cause == null ? "unknown" : cause.getClass().getSimpleName();
39 |             if (cause instanceof java.sql.SQLException sqlFailure) {
40 |                 diagnostic += " SQLState=" + sqlFailure.getSQLState() + " code=" +
41 | sqlFailure.getErrorCode();
42 |             }
43 |             // Container hosts collect stdout/stderr; Tomcat context logs can go to files.
44 |             System.err.println("Service catalogue database failure: " + diagnostic);
45 |
46 |             response.setStatus(HttpServletResponse.SC_SERVICE_UNAVAILABLE);
47 |             response.getWriter().write("{\"success\":false,\"message\":\"Service catalogue unavailable. " +
48 | Check backend database configuration and server logs.\"}");
49 |             return;
50 |         }
51 |
52 |         StringBuilder json =
53 |             new StringBuilder();

```

```
51 |  
52 |     json.append("{}");  
53 |     json.append("\"success\":true,");  
54 |     json.append("\"message\": \"Services retrieved successfully\",");  
55 |     json.append("\"data\":[");  
56 |  
57 |     for (int i = 0; i < services.size(); i++) {  
58 |  
59 |         Service service = services.get(i);  
60 |  
61 |         json.append("{}");  
62 |         json.append("\"serviceId\":")  
63 |             .append(service.getServiceId())  
64 |             .append(",");  
65 |  
66 |         json.append("\"serviceName\": \"")  
67 |             .append(service.getServiceName())  
68 |             .append("\",");  
69 |  
70 |         json.append("\"description\": \"")  
71 |             .append(service.getDescription())  
72 |             .append("\",");  
73 |  
74 |         json.append("\"averageServiceTime\":")  
75 |             .append(service.getAverageServiceTime());  
76 |  
77 |         json.append("}");  
78 |  
79 |         if (i < services.size() - 1) {  
80 |             json.append(",");  
81 |         }  
82 |     }  
83 |  
84 |     json.append("]");  
85 |     json.append("}");  
86 |  
87 |     response.getWriter().write(  
88 |         json.toString()  
89 |     );  
90 | }  
91 | }
```

src/main/java/com/queue/controller/StaffServlet.java

Staff HTTP adapter for login, profile, service queue, counter availability, summary and activity. Study service scoping and remaining legacy branches.

Read these lines first

Line 20: Registers this servlet URL mapping; explain which browser/API caller uses the route.

Line 49: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 55: Reads client-supplied input; validation and authorization are needed before using it.

Line 56: Reads client-supplied input; validation and authorization are needed before using it.

Line 74: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 77: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 108: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 124: Reads client-supplied input; validation and authorization are needed before using it.

Numbered source

```

1 | package com.queue.controller;
2 |
3 | import com.queue.model.Staff;
4 | import com.queue.service.StaffService;
5 | import com.queue.model.Booking;
6 | import com.queue.service.AnalyticsService;
7 | import com.queue.service.BookingService;
8 |
9 | import java.util.List;
10 |
11 | import jakarta.servlet.ServletException;
12 | import jakarta.servlet.annotation.WebServlet;
13 | import jakarta.servlet.http.HttpServlet;
14 | import jakarta.servlet.http.HttpServletRequest;
15 | import jakarta.servlet.http.HttpServletResponse;
16 | import jakarta.servlet.http.HttpSession;
17 |
18 | import java.io.IOException;
19 |
20 | @WebServlet("/api/staff/*")
21 | public class StaffServlet extends HttpServlet {
22 |
23 |     private StaffService staffService;
24 |     private AnalyticsService analyticsService;
25 |     private BookingService bookingService;
26 |
27 |     @Override
28 |     public void init() {
29 |         staffService = new StaffService();
30 |         analyticsService = new AnalyticsService();
31 |         bookingService = new BookingService();
32 |     }
33 |
34 |     // =====
35 |     // POST /api/staff/login
36 |     // =====
37 |
38 |     @Override
39 |     protected void doPost(HttpServletRequest request,
40 |                           HttpServletResponse response)
41 |         throws ServletException, IOException {
42 |
43 |         String path = request.getPathInfo();
44 |

```

```

45 |         response.setContentType("application/json");
46 |         response.setCharacterEncoding("UTF-8");
47 |
48 |         if ("/logout".equals(path)) {
49 |             HttpSession current=request.getSession(false);
50 |             if(current!=null)current.invalidate();
51 |             response.getWriter().write("{\"success\":true,\"data\":null}");return;
52 |         }
53 |         if ("/login".equals(path)) {
54 |
55 |             String email = request.getParameter("email");
56 |             String password = request.getParameter("password");
57 |
58 |             if (email == null || password == null ||
59 |                 email.isBlank() || password.isBlank()) {
60 |
61 |                 response.getWriter().write(
62 |                     "{\"success\":false,\" +
63 |                     \"message\":\"Email and password are required\",\" +
64 |                     \"data\":null}"
65 |                 );
66 |
67 |                 return;
68 |             }
69 |
70 |             Staff staff = staffService.login(email, password);
71 |
72 |             if (staff != null) {
73 |
74 |                 HttpSession session = request.getSession();
75 |                 request.changeSessionId();
76 |                 session.removeAttribute("studentId");
77 |                 session.setAttribute("staffId", staff.getStaffId());
78 |
79 |                 String json =
80 |                     "{"
81 |                     + "\"success\":true,\" +
82 |                     + "\"message\":\"Staff login successful\",\" +
83 |                     + "\"data\":{\" +
84 |                     + "\"staffId\":\"" + staff.getStaffId() + "\",\" +
85 |                     + "\"staffName\":\"" + staff.getStaffName() + "\",\" +
86 |                     + "\"email\":\"" + staff.getEmail() + "\",\" +
87 |                     + "\"serviceId\":\"" + staff.getServiceId() + "\",\" +
88 |                     + "\"counterId\":\"" + staff.getCounterId()
89 |
90 |                     + "}"
91 |                     + "}";
92 |
93 |                 response.getWriter().write(json);
94 |             } else {
95 |
96 |                 response.getWriter().write(
97 |                     "{\"success\":false,\" +
98 |                     + "\"message\":\"Invalid email or password\",\" +
99 |                     + "\"data\":null}"
100 |                 );
101 |             }
102 |
103 |             return;
104 |         }
105 |
106 |         if ("/counter/status".equals(path)) {
107 |
108 |             HttpSession session = request.getSession(false);
109 |
110 |             if (session == null ||
111 |                 session.getAttribute("staffId") == null) {
112 |                 response.setStatus(401);
113 |
114 |                 response.getWriter().write(
115 |                     "{\"success\":false,\" +

```

```

116 |         "\"message\": \"Staff is not logged in\", \" +
117 |         "\"data\": null}"
118 |     );
119 |
120 |     return;
121 | }
122 |
123 | String status =
124 |     request.getParameter("status");
125 |
126 | if (status == null || status.isBlank()) {
127 |
128 |     response.getWriter().write(
129 |         "\"success\": false, \" +
130 |         "\"message\": \"Status is required\", \" +
131 |         "\"data\": null}"
132 |     );
133 |
134 |     return;
135 | }
136 |
137 | status = status.toUpperCase();
138 |
139 | if (!status.equals("AVAILABLE") &&
140 |     !status.equals("PAUSED") &&
141 |     !status.equals("CHECKED_OUT")) {
142 |
143 |     response.getWriter().write(
144 |         "\"success\": false, \" +
145 |         "\"message\": \"Invalid status. Use AVAILABLE, PAUSED or CHECKED_OUT\", \" +
146 |         "\"data\": null}"
147 |     );
148 |
149 |     return;
150 | }
151 |
152 | int staffId =
153 |     (Integer) session.getAttribute("staffId");
154 |
155 | boolean updated;
156 | try { updated=new com.queue.service.ReservationService().availability(staffId,status); }
157 | catch(Exception error){response.setStatus(409);response.getWriter().write(com.queue.util.Json.encoded(
158 | java.util.Map.of("success",false,"message",error instanceof
159 | IllegalArgumentException?error.getMessage():"Could not update counter availability."));return;} if
160 | (!updated) {
161 |
162 |     response.getWriter().write(
163 |         "\"success\": false, \" +
164 |         "\"message\": \"Failed to update counter status\", \" +
165 |         "\"data\": null}"
166 |     );
167 |
168 |     return;
169 | }
170 |
171 | java.util.Map<String, Object> details =
172 |     staffService.getCounterDetails(staffId);
173 |
174 | String json =
175 |     "{"
176 |     + "\"success\": true, \" +
177 |     + "\"message\": \"Counter status updated\", \" +
178 |     + "\"data\": {"
179 |     + "\"counterId\": \" + details.get("counterId") + \", \" +
180 |     + "\"counterName\": \"\" + details.get("counterName") + "\", \" +
181 |     + "\"serviceId\": \" + details.get("serviceId") + \", \" +
182 |     + "\"serviceName\": \"\" + details.get("serviceName") + "\", \" +
183 |     + "\"status\": \"\" + status + "\""
184 |     + "}"
185 |     + "}";

```

```

183 |
184 |     response.getWriter().write(json);
185 |
186 |     return;
187 | }
188 |
189 |     response.getWriter().write(
190 |         "{\"success\":false,\" +
191 |         \"message\": \"Invalid staff endpoint\",\" +
192 |         \"data\":null}\"
193 |     );
194 | }
195 |
196 | // =====
197 | // GET /api/staff/profile
198 | // =====
199 |
200 | @Override
201 | protected void doGet(HttpServletRequest request,
202 |                     HttpServletResponse response)
203 |     throws ServletException, IOException {
204 |
205 |     String path = request.getPathInfo();
206 |     if ("/queue".equals(path)) {
207 |         response.setContentType("application/json");response.setCharacterEncoding("UTF-8");
208 |         HttpSession actor=request.getSession(false);
209 |         if(actor==null || !(actor.getAttribute("staffId") instanceof
| Integer)){response.setStatus(401);response.getWriter().write("{\"success\":false,\"message\": \"Staff is
| not logged in\"}");return;}
210 |         try
| {response.getWriter().write(com.queue.util.Json.encode(java.util.Map.of("success",true,"data",new
| com.queue.service.ReservationService().staffQueue((Integer)actor.getAttribute("staffId"))));}
211 |         catch(Exception
| error){response.setStatus(503);response.getWriter().write("{\"success\":false,\"message\": \"Queue
| unavailable\"}");}
212 |
213 |     }
214 |
215 |     response.setContentType("application/json");
216 |     response.setCharacterEncoding("UTF-8");
217 |
218 |     if ("/profile".equals(path)) {
219 |
220 |         HttpSession session = request.getSession(false);
221 |
222 |         if (session == null ||
223 |             session.getAttribute("staffId") == null) {
224 |             response.setStatus(401);
225 |
226 |             response.getWriter().write(
227 |                 "{\"success\":false,\" +
228 |                 \"message\": \"Staff is not logged in\",\" +
229 |                 \"data\":null}\"
230 |             );
231 |
232 |             return;
233 |         }
234 |
235 |         int staffId =
236 |             (Integer) session.getAttribute("staffId");
237 |
238 |         Staff staff =
239 |             staffService.getStaffProfile(staffId);
240 |
241 |         if (staff == null) {
242 |
243 |             response.getWriter().write(
244 |                 "{\"success\":false,\" +
245 |                 \"message\": \"Staff profile not found\",\" +
246 |                 \"data\":null}\"
247 |             );

```

```

248 |
249 |         return;
250 |     }
251 |
252 |     String json =
253 |         "{"
254 |         + "\"success\":true,"
255 |         + "\"message\": \"Staff profile retrieved\","
256 |
257 |         + "\"data\":{"
258 |         + "\"staffId\": " + staff.getStaffId() + ","
259 |         + "\"staffName\": \"\" + staff.getStaffName() + "\", "
260 |         + "\"email\": \"\" + staff.getEmail() + "\", "
261 |         + "\"serviceId\": " + staff.getServiceId() + ","
262 |         + "\"counterId\": " + staff.getCounterId()
263 |         + "}"
264 |         + "}";
265 |
266 |     response.getWriter().write(json);
267 |
268 |     return;
269 | }
270 |
271 | if ("/counter/status".equals(path)) {
272 |     HttpSession session = request.getSession(false);
273 |
274 |     if (session == null ||
275 |         session.getAttribute("staffId") == null) {
276 |         response.setStatus(401);
277 |
278 |         response.getWriter().write(
279 |             "{" + "\"success\":false,\"" +
280 |             "\"message\": \"Staff is not logged in\"," +
281 |             "\"data\":null}"
282 |         );
283 |
284 |         return;
285 |     }
286 |
287 |     int staffId =
288 |         (Integer) session.getAttribute("staffId");
289 |
290 |     java.util.Map<String, Object> details =
291 |         staffService.getCounterDetails(staffId);
292 |
293 |     if (details.isEmpty()) {
294 |
295 |         response.getWriter().write(
296 |             "{" + "\"success\":false,\"" +
297 |             "\"message\": \"Counter details not found\"," +
298 |             "\"data\":null}"
299 |         );
300 |
301 |         return;
302 |     }
303 |
304 |     String status;
305 |     try { status=new
306 | com.queue.service.ReservationService().availability(((Number)details.get("counterId")).intValue()); }
307 | catch(Exception
308 | error){response.setStatus(503);response.getWriter().write("{\"success\":false,\"message\": \"Counter
309 | status unavailable\"");return;} String json =
310 |     "{"
311 |     + "\"success\":true,"
312 |     + "\"message\": \"Counter status retrieved\","
313 |     + "\"data\":{"
314 |     + "\"counterId\": " + details.get("counterId") + ","
315 |     + "\"counterName\": \"\" + details.get("counterName") + "\", "
316 |     + "\"serviceId\": " + details.get("serviceId") + ","
317 |     + "\"serviceName\": \"\" + details.get("serviceName") + "\", "

```

```

315 |         + "\"status\":" + status + "\""
316 |         + "}"
317 |         + "};
318 |
319 |     response.getWriter().write(json);
320 |
321 |     return;
322 | }
323 |
324 |     if ("/dashboard-summary".equals(path)) {
325 |
326 |     HttpSession session = request.getSession(false);
327 |
328 |     if (session == null ||
329 |         session.getAttribute("staffId") == null) {
330 |         response.setStatus(401);
331 |
332 |         response.getWriter().write(
333 |             "{\"success\":false,\" +
334 |             \"message\":\"Staff is not logged in\", \" +
335 |             \"data\":null}"
336 |         );
337 |
338 |         return;
339 |     }
340 |
341 |     int staffId =
342 |         (Integer) session.getAttribute("staffId");
343 |
344 |     Staff staff =
345 |         staffService.getStaffProfile(staffId);
346 |
347 |     if (staff == null) {
348 |
349 |         response.getWriter().write(
350 |             "{\"success\":false,\" +
351 |             \"message\":\"Staff profile not found\", \" +
352 |             \"data\":null}"
353 |         );
354 |
355 |         return;
356 |     }
357 |
358 |     // Get active queue for staff's service
359 |     List<com.queue.model.Queue> activeQueue =
360 |         staffService.getActiveQueue(staffId);
361 |
362 |     int studentsWaiting = 0;
363 |
364 |     for (com.queue.model.Queue queue : activeQueue) {
365 |
366 |         if ("WAITING".equals(queue.getStatus())) {
367 |             studentsWaiting++;
368 |         }
369 |     }
370 |
371 |     java.util.Map<String, Object> metrics;
372 |     try {metrics=analyticsService.getServiceDaySummary(staff.getServiceId());}
373 |     catch (java.sql.SQLException
| error){response.setStatus(503);response.getWriter().write("{\"success\":false,\"message\":\"Analytics
| temporarily unavailable\"");return;}
374 |     int servedToday=((Number)metrics.get("servedToday")).intValue();
375 |     Object averageWaitMinutes=metrics.get("averageWaitMinutes");
376 |     Object averageServiceMinutes=metrics.get("averageServiceMinutes");
377 |     int noShows=((Number)metrics.get("noShowsToday")).intValue();
378 |     String peakHour=(String)metrics.get("peakHour");
379 |     // Get upcoming bookings for staff's service
380 |     List<java.util.Map<String, Object>> upcomingBookings =
381 |         bookingService.getUpcomingBookingsWithDetails(
382 |             staff.getServiceId()

```

```

383 |     );
384 |
385 |     String json =
386 |         "{"
387 |         + "\"success\":true,"
388 |         + "\"message\":\"Dashboard summary fetched successfully\","
389 |         + "\"data\":{\""
390 |         + "\"studentsWaiting\":\"" + studentsWaiting + ","
391 |         + "\"servedToday\":\"" + servedToday + ","
392 |         + "\"averageWaitMinutes\":\"" + averageWaitMinutes + ","
393 |         + "\"averageServiceMinutes\":\"" + averageServiceMinutes + ","
394 |         + "\"noShowsToday\":\"" + noShows + ","
395 |         + "\"peakHour\":\"" + peakHour + "\",
396 |         + "\"upcomingBookings\":\"" + studentsWaiting
397 |         + "}"
398 |         + "}";
399 |
400 |     response.getWriter().write(json);
401 |
402 |     return;
403 | }
404 | if ("/recent-activity".equals(path)) {
405 |
406 |     HttpSession session = request.getSession(false);
407 |
408 |     if (session == null ||
409 |         session.getAttribute("staffId") == null) {
410 |         response.setStatus(401);
411 |
412 |         response.getWriter().write(
413 |             "{" + "\"success\":false," +
414 |             "\"message\":\"Staff is not logged in\"," +
415 |             "\"data\":null}"
416 |         );
417 |
418 |         return;
419 |     }
420 |
421 |     int staffId =
422 |         (Integer) session.getAttribute("staffId");
423 |
424 |     Staff staff =
425 |         staffService.getStaffProfile(staffId);
426 |
427 |     if (staff == null) {
428 |
429 |         response.getWriter().write(
430 |             "{" + "\"success\":false," +
431 |             "\"message\":\"Staff profile not found\"," +
432 |             "\"data\":null}"
433 |         );
434 |
435 |         return;
436 |     }
437 |
438 |     List<com.queue.model.Queue> activityList =
439 |         staffService.getRecentActivity(staffId);
440 |
441 |     StringBuilder json =
442 |         new StringBuilder();
443 |
444 |     json.append("{")
445 |         .append("\"success\":true,")
446 |         .append("\"message\":\"Recent activity retrieved\"," +
447 |         .append("\"data\":[");
448 |
449 |     boolean firstActivity = true;
450 |
451 |     for (com.queue.model.Queue queue : activityList) {
452 |
453 |         if (queue.getCalledAt() != null) {

```

```

454 |
455 |         if (!firstActivity) {
456 |             json.append(",");
457 |         }
458 |
459 |         json.append("{")
460 |             .append("\"tokenNumber\":")
461 |             .append(queue.getTokenNumber())
462 |             .append(",")
463 |             .append("\"action\": \"CALLED\",")
464 |             .append("\"timestamp\": \"")
465 |             .append(queue.getCalledAt())
466 |             .append("\"")
467 |             .append("}");
468 |
469 |         firstActivity = false;
470 |     }
471 |
472 |     if (queue.getStartedAt() != null) {
473 |
474 |         if (!firstActivity) {
475 |             json.append(",");
476 |         }
477 |
478 |         json.append("{")
479 |             .append("\"tokenNumber\":")
480 |             .append(queue.getTokenNumber())
481 |             .append(",")
482 |             .append("\"action\": \"SERVICE_STARTED\",")
483 |             .append("\"timestamp\": \"")
484 |             .append(queue.getStartedAt())
485 |             .append("\"")
486 |             .append("}");
487 |
488 |         firstActivity = false;
489 |     }
490 |
491 |     if (queue.getCompletedAt() != null) {
492 |
493 |         if (!firstActivity) {
494 |             json.append(",");
495 |         }
496 |
497 |         String action =
498 |             "SKIPPED".equals(queue.getStatus())
499 |                 ? "SKIPPED"
500 |                 : queue.getStatus();
501 |
502 |         json.append("{")
503 |             .append("\"tokenNumber\":")
504 |             .append(queue.getTokenNumber())
505 |             .append(",")
506 |             .append("\"action\": \"")
507 |             .append(action)
508 |             .append("\",")
509 |             .append("\"timestamp\": \"")
510 |             .append(queue.getCompletedAt())
511 |             .append("\"")
512 |             .append("}");
513 |
514 |         firstActivity = false;
515 |     }
516 | }
517 |
518 | json.append("]");
519 |
520 | response.getWriter().write(
521 |     json.toString()
522 | );
523 |

```

```

524 |     return;
525 | }
526 |
527 |
528 |     if ("/queue".equals(path)) {
529 |
530 |         HttpSession session = request.getSession(false);
531 |
532 |         if (session == null ||
533 |             session.getAttribute("staffId") == null) {
534 |             response.setStatus(401);
535 |
536 |             response.getWriter().write(
537 |                 "{\"success\":false,\" +
538 |                 \"message\":\"Staff is not logged in\", \" +
539 |                 \"data\":null}"
540 |             );
541 |
542 |             return;
543 |         }
544 |
545 |         int staffId =
546 |             (Integer) session.getAttribute("staffId");
547 |
548 |         java.util.List<com.queue.model.Queue> queueList =
549 |             staffService.getActiveQueue(staffId);
550 |
551 |         StringBuilder json =
552 |             new StringBuilder();
553 |
554 |         json.append("{")
555 |             .append("\"success\":true,")
556 |             .append("\"message\":\"Active queue retrieved\",")
557 |             .append("\"data\":[");
558 |
559 |         for (int i = 0; i < queueList.size(); i++) {
560 |
561 |             com.queue.model.Queue queue =
562 |                 queueList.get(i);
563 |
564 |             if (i > 0) {
565 |                 json.append(",");
566 |             }
567 |
568 |             json.append("{")
569 |                 .append("\"queueId\":")
570 |                 .append(queue.getQueueId())
571 |                 .append(",")
572 |                 .append("\"studentId\":")
573 |                 .append(queue.getStudentId())
574 |                 .append(",")
575 |                 .append("\"serviceId\":")
576 |                 .append(queue.getServiceId())
577 |                 .append(",")
578 |                 .append("\"tokenNumber\":")
579 |                 .append(queue.getTokenNumber())
580 |                 .append(",")
581 |                 .append("\"status\":\"")
582 |                 .append(queue.getStatus())
583 |                 .append("\"")
584 |                 .append("}");
585 |         }
586 |
587 |         json.append("]");
588 |
589 |         response.getWriter().write(json.toString());
590 |
591 |         return;
592 |     }
593 |
594 |     response.getWriter().write(

```

```
595 |         "{\"success\":false,\" +
596 |         \"message\": \"Invalid staff endpoint\", \" +
597 |         \"data\":null}"
598 |     );
599 | }
600 | }
```

src/main/java/com/queue/controller/StudentPortalServlet.java

Current student HTTP adapter. Routes login/register/session/overview/preview/reserve/cancel; obtains identity from the server session.

Read these lines first

Line 23: Registers this servlet URL mapping; explain which browser/API caller uses the route.

Line 47: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 51: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 56: try-with-resources closes database/IO resources automatically when the block ends.

Line 65: Reads client-supplied input; validation and authorization are needed before using it.

Line 70: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 72: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 74: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Numbered source

```

1 | package com.queue.controller;
2 |
3 | import com.queue.dao.CounterDAO;
4 | import com.queue.dao.ServiceDAO;
5 | import com.queue.model.Booking;
6 | import com.queue.model.Service;
7 | import com.queue.model.Student;
8 | import com.queue.service.BookingService;
9 | import com.queue.service.NoShowService;
10 | import com.queue.service.StudentService;
11 | import com.queue.service.WaitingTimeService;
12 | import com.queue.util.DBConnection;
13 | import com.queue.util.Json;
14 | import jakarta.servlet.annotation.WebServlet;
15 | import jakarta.servlet.http.*;
16 | import java.io.IOException;
17 | import java.sql.Connection;
18 | import java.sql.Date;
19 | import java.sql.Time;
20 | import java.util.*;
21 |
22 | /** Student session and presentation adapter. Business rules remain in the team's services. */
23 | @WebServlet("/api/student/*")
24 | public class StudentPortalServlet extends HttpServlet {
25 |     private final com.queue.service.ReservationService reservations = new
26 |     | com.queue.service.ReservationService();
27 |     private final StudentService students = new StudentService();
28 |     private final ServiceDAO services = new ServiceDAO();
29 |     private final BookingService bookings = new BookingService();
30 |     private final WaitingTimeService waiting = new WaitingTimeService();
31 |     private final CounterDAO counters = new CounterDAO();
32 |
33 |     protected void doGet(HttpServletRequestRequest req, HttpServletResponse res) throws IOException {
34 |     | handle(req, res); }
35 |     protected void doPost(HttpServletRequestRequest req, HttpServletResponse res) throws IOException {
36 |     | handle(req, res); }
37 |
38 |     private void handle(HttpServletRequestRequest req, HttpServletResponse res) throws IOException {
39 |     |
40 |     |     res.setContentType("application/json");
41 |     |     res.setCharacterEncoding("UTF-8");
42 |     |     res.setHeader("Cache-Control", "no-store");
43 |     |     req.setCharacterEncoding("UTF-8");
44 |     |     String path = Objects.toString(req.getPathInfo(), "");
45 |     |     boolean post = "POST".equals(req.getMethod());
46 |
47 |     |     if (post && !"XMLHttpRequest".equals(req.getHeader("X-Requested-With"))) {
48 |     |         fail(res, 403, "Use the student application to submit this request."); return;
49 |     |     }
50 |     |     try {
51 |     |         if (!post && "/session".equals(path)) {
52 |     |             HttpSession session = req.getSession(false);
53 |     |             ok(res, session == null ? null : session.getAttribute("student")); return;
54 |     |         }
55 |     |         if (post && "/logout".equals(path)) {
56 |     |             HttpSession session = req.getSession(false);
57 |     |             if (session != null) session.invalidate();
58 |     |             ok(res, null); return;
59 |     |         }
60 |     |         // Fail explicitly instead of treating DAO database failures as an empty successful result.
61 |     |         try (Connection connection = DBConnection.getConnection()) {
62 |     |             if (!connection.isValid(3)) throw new java.sql.SQLException("Database unavailable");
63 |     |         }
64 |     |         if (post && ("/login".equals(path) || "/register".equals(path))) {
65 |     |             String email = required(req, "email");
66 |     |             String password = required(req, "password");
67 |     |             if ("/register".equals(path)) {
68 |     |                 Student account = new Student();
69 |     |                 account.setStudentName(required(req, "studentName"));
70 |     |                 account.setEmail(email); account.setPassword(password);
71 |     |                 account.setPhone(req.getParameter("phone"));

```

```

66 |         if (!students.registerStudent(account)) { fail(res, 409, "Registration failed. That
67 | email may already be registered."); return; }
68 |     }
69 |     Student student = students.loginStudent(email, password);
70 |     if (student == null) { fail(res, 401, "Invalid email or password."); return; }
71 |     HttpSession previous = req.getSession(false);
72 |     if (previous != null) previous.invalidate();
73 |     HttpSession session = req.getSession(true);
74 |     Map<String, Object> profile = Map.of("studentId", student.getStudentId(), "name",
75 | student.getStudentName(), "email", student.getEmail());
76 |     session.setAttribute("studentId", student.getStudentId());
77 |     session.setAttribute("student", profile); session.setMaxInactiveInterval(1800);
78 |     ok(res, profile); return;
79 | }
80 | HttpSession session = req.getSession(false);
81 | if (session == null || !(session.getAttribute("studentId") instanceof Integer)) {
82 |     fail(res, 401, "Please sign in to your student account."); return;
83 | }
84 | int studentId = (Integer) session.getAttribute("studentId");
85 |
86 | if (!post && "/overview".equals(path)) {
87 |     List<Map<String, Object>> catalog = new ArrayList<>();
88 |     List<Map<String, Object>> tickets = new ArrayList<>();
89 |     for (Service service : services.getAllServices()) {
90 |         int id = service.getServiceId();
91 |         reservations.closeWaiting(id);
92 |         int active = counters.getCountersByService(id).size();
93 |         catalog.add(Map.of("serviceId", id, "serviceName", service.getServiceName(),
94 | "description", Objects.toString(service.getDescription(), ""),
95 | "averageServiceTime", new
96 | com.queue.service.WaitingTimeService().getAverageServiceTime(service.getServiceId()), "activeCounters",
97 | active));
98 |         com.queue.model.Queue entry = students.getQueueStatus(studentId, id);
99 |         if (entry != null && Set.of("WAITING", "CALLED",
100 | "SERVING").contains(entry.getStatus())) {
101 |             int ahead = Math.max(0, students.getQueuePosition(studentId, id) - 1);
102 |             Map<String, Object> ticket = new LinkedHashMap<>();
103 |             ticket.put("queueId", entry.getQueueId()); ticket.put("serviceId", id);
104 |             ticket.put("serviceName", service.getServiceName()); ticket.put("place",
105 | service.getServiceName());
106 |             ticket.put("token", String.format("%03d", entry.getTokenNumber()));
107 |             ticket.put("status", entry.getStatus()); ticket.put("ahead", ahead);
108 |             Map<String, Object> projection = reservations.preview(studentId, id);
109 |             ticket.put("ahead", projection.get("ahead")); ticket.put("date", projection.get("d
110 | ate")); ticket.put("rescheduled", projection.get("rescheduled"));
111 |             int minutes =
112 | projection.get("wait") == null ? -1 : ((Number) projection.get("wait")).intValue();
113 |             ticket.put("wait", minutes < 0 ? null : minutes);
114 |             ticket.put("currentToken", students.getCurrentToken(id));
115 |             ticket.put("counterId", projection.get("counterId")); ticket.put("counterName", pr
116 | ojection.get("counterName")); ticket.put("missedTurns", projection.get("missedTurns")); ticket.put("estima
117 | te", projection.get("estimate")); ticket.put("hours", projection.get("hours"));
118 |             ticket.put("closingRisk", projection.get("closingRisk")); ticket.put("warning", pr
119 | ojection.get("warning")); ticket.put("projectedServiceTime", projection.get("projectedServiceTime"));
120 |             tickets.add(ticket);
121 |         }
122 |     }
123 |     ok(res, Map.of("services", catalog, "tickets", tickets)); return;
124 | }
125 | if (post && "/queue/join".equals(path)) {
126 |     int serviceId = Integer.parseInt(required(req, "serviceId"));
127 |     requireService(serviceId);
128 |     com.queue.model.Queue previous = students.getQueueStatus(studentId, serviceId);
129 |     if (previous != null && Set.of("WAITING", "CALLED",
130 | "SERVING").contains(previous.getStatus())) {
131 |         fail(res, 409, "You already have an active ticket for this service."); return;
132 |     }
133 |     com.queue.model.Queue entry = new com.queue.model.Queue();
134 |     entry.setStudentId(studentId); entry.setServiceId(serviceId);
135 |     if (!students.joinQueue(entry)) { fail(res, 409, "Unable to join the queue. Please try
136 | again."); return; }

```

```

123 |         ok(res, Map.of("queueId", entry.getQueueId())); return;
124 |     }
125 |     if (!post && "/bookings".equals(path)) { ok(res,reservations.visits(studentId));return; }
126 |     if (!post && "/bookings/preview".equals(path)) { int
127 | id=Integer.parseInt(required(req,"serviceId"));ok(res,reservations.preview(studentId,id));return; }
128 |         if (post && "/bookings/create".equals(path)) {
129 |             if(req.getParameter("bookingDate")!=null && !required(req,"bookingDate").equals(com.que
130 | ue.service.ServiceHours.now().toLocalDate().toString()))throw new IllegalArgumentException("Only
131 | today's reservations are allowed.");
132 |
133 | ok(res,reservations.reserve(studentId,Integer.parseInt(required(req,"serviceId"))));return;
134 |     }
135 |     if (post && "/bookings/cancel".equals(path)) {
136 |         if(!reservations.cancel(studentId,Integer.parseInt(required(req,"bookingId")))){fail(res,409,"Only your
137 | waiting reservation for today can be cancelled.");return;}
138 |         ok(res,null);return;
139 |     }
140 |     if ("/bookings/slots".equals(path)) { fail(res,410,"Time slots have been replaced by
141 | today's queue reservations.");return; } fail(res, 404, "Student operation not supported.");
142 |     } catch (IllegalArgumentException error) { fail(res, 400, error.getMessage()==null?"Please
143 | supply valid required fields." :error.getMessage()); }
144 |     catch (java.sql.SQLException error) { fail(res, 503, "The database is unavailable. Please try
145 | again shortly."); }
146 |     catch (IllegalStateException error) { fail(res, 503, "The operation could not be saved. Please
147 | try again shortly."); }
148 |     }
149 |     private void requireService(int id) {
150 |         if (services.getAllServices().stream().noneMatch(service -> service.getServiceId() == id))
151 |             throw new IllegalArgumentException("Unknown service");
152 |     }
153 |     private String required(HttpServletRequest req, String name) {
154 |         String value = req.getParameter(name);
155 |         if (value == null || value.isBlank()) throw new IllegalArgumentException(name);
156 |         return value;
157 |     }
158 |     private void ok(HttpServletResponse res, Object data) throws IOException {
159 |         Map<String,Object> body = new LinkedHashMap<>(); body.put("success", true); body.put("data",
160 | data);
161 |         res.getWriter().write(Json.encode(body));
162 |     }
163 |     private void fail(HttpServletResponse res, int status, String message) throws IOException {
164 |         res.setStatus(status); res.getWriter().write(Json.encode(Map.of("success", false, "message",
165 | message)));
166 |     }
167 | }

```

src/main/java/com/queue/controller/StudentServlet.java

Older student HTTP surface. Compare with current StudentPortalServlet before claiming this is the active UI route.

Read these lines first

Line 14: Registers this servlet URL mapping; explain which browser/API caller uses the route.

Line 38: Reads client-supplied input; validation and authorization are needed before using it.

Line 41: Reads client-supplied input; validation and authorization are needed before using it.

Line 44: Reads client-supplied input; validation and authorization are needed before using it.

Line 47: Reads client-supplied input; validation and authorization are needed before using it.

Line 95: Reads client-supplied input; validation and authorization are needed before using it.

Line 98: Reads client-supplied input; validation and authorization are needed before using it.

Numbered source

```

1 | package com.queue.controller;
2 |
3 | import com.queue.model.Student;
4 | import com.queue.service.StudentService;
5 |
6 | import jakarta.servlet.ServletException;
7 | import jakarta.servlet.annotation.WebServlet;
8 | import jakarta.servlet.http.HttpServlet;
9 | import jakarta.servlet.http.HttpServletRequest;
10 | import jakarta.servlet.http.HttpServletResponse;
11 |
12 | import java.io.IOException;
13 |
14 | @WebServlet("/api/students/*")
15 | public class StudentServlet extends HttpServlet {
16 |
17 |     private StudentService studentService;
18 |
19 |     @Override
20 |     public void init() {
21 |         studentService = new StudentService();
22 |     }
23 |
24 |     @Override
25 |     protected void doPost(HttpServletRequest request,
26 |                           HttpServletResponse response)
27 |         throws ServletException, IOException {
28 |
29 |         response.setContentType("application/json");
30 |         response.setCharacterEncoding("UTF-8");
31 |
32 |         String path = request.getPathInfo();
33 |
34 |         // POST /api/students/register
35 |         if ("/register".equals(path)) {
36 |
37 |             String studentName =
38 |                 request.getParameter("studentName");
39 |
40 |             String email =
41 |                 request.getParameter("email");
42 |
43 |             String phone =
44 |                 request.getParameter("phone");
45 |
46 |             String password =
47 |                 request.getParameter("password");
48 |
49 |             if (studentName == null ||
50 |                 email == null ||
51 |                 password == null) {
52 |
53 |                 writeError(
54 |                     response,
55 |                     "studentName, email and password are required"
56 |                 );
57 |
58 |                 return;
59 |             }
60 |
61 |             Student student = new Student();
62 |
63 |             student.setStudentName(studentName);
64 |             student.setEmail(email);
65 |             student.setPhone(phone);
66 |             student.setPassword(password);
67 |
68 |             boolean success =
69 |                 studentService.registerStudent(student);

```

```
70 |
71 |         if (success) {
72 |
73 |             response.getWriter().write(
74 |                 "{"
75 |                 + "\"success\":true,"
76 |                 + "\"message\": \"Student registered successfully\""
77 |                 + "}"
78 |             );
79 |
80 |         } else {
81 |
82 |             writeError(
83 |                 response,
84 |                 "Unable to register student. Email may already exist."
85 |             );
86 |         }
87 |
88 |         return;
89 |
90 |
91 |         // POST /api/students/login
92 |         if ("/login".equals(path)) {
93 |
94 |             String email =
95 |                 request.getParameter("email");
96 |
97 |             String password =
98 |                 request.getParameter("password");
99 |
100 |             if (email == null ||
101 |                 password == null) {
102 |
103 |                 writeError(
104 |                     response,
105 |                     "email and password are required"
106 |                 );
107 |
108 |                 return;
109 |             }
110 |
111 |             Student student =
112 |                 studentService.loginStudent(
113 |                     email,
114 |                     password
115 |                 );
116 |
117 |             if (student != null) {
118 |
119 |                 response.getWriter().write(
120 |                     "{"
121 |                     + "\"success\":true,"
122 |                     + "\"message\": \"Student login successful\","
123 |                     + "\"data\":{"
124 |                     + "\"studentId\": " + student.getStudentId() + ","
125 |                     + "\"studentName\": \"\" + student.getStudentName() + "\", "
126 |                     + "\"email\": \"\" + student.getEmail() + "\""
127 |                     + "}"
128 |                     + "}"
129 |                 );
130 |
131 |             } else {
132 |
133 |                 writeError(
134 |                     response,
135 |                     "Invalid email or password"
136 |                 );
137 |             }
138 |
139 |             return;
```

```
140 |     }
141 |
142 |     writeError(
143 |         response,
144 |         "POST operation not available for this endpoint"
145 |     );
146 | }
147 |
148 | private void writeError(HttpServletResponse response,
149 |                         String message)
150 |     throws IOException {
151 |
152 |     response.getWriter().write(
153 |         "{"
154 |         + "\"success\":false,"
155 |         + "\"message\":\"" + message + "\""
156 |         + "}"
157 |     );
158 | }
159 | }
```

src/main/java/com/queue/dao/CounterDAO.java

Counter lookup and state/storage operations. A counter belongs to one service.

Read these lines first

Line 7: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 23: try-with-resources closes database/IO resources automatically when the block ends.

Line 25: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 28: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 30: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Line 56: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 69: try-with-resources closes database/IO resources automatically when the block ends.

Line 71: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Numbered source

```

1 | package com.queue.dao;
2 |
3 | import com.queue.model.Counter;
4 | import com.queue.util.DBConnection;
5 |
6 | import java.sql.Connection;
7 | import java.sql.PreparedStatement;
8 | import java.sql.ResultSet;
9 | import java.util.ArrayList;
10 | import java.util.List;
11 |
12 | public class CounterDAO {
13 |
14 |     public List<Counter> getCountersByService(int serviceId) {
15 |
16 |         List<Counter> counters = new ArrayList<>();
17 |
18 |         String sql = "SELECT * FROM counters " +
19 |             "WHERE service_id = ? " +
20 |             "AND is_active = TRUE " +
21 |             "ORDER BY counter_id";
22 |
23 |         try (
24 |             Connection connection = DBConnection.getConnection();
25 |             PreparedStatement statement = connection.prepareStatement(sql)
26 |         ) {
27 |
28 |             statement.setInt(1, serviceId);
29 |
30 |             try (ResultSet resultSet = statement.executeQuery()) {
31 |
32 |                 while (resultSet.next()) {
33 |
34 |                     Counter counter = new Counter();
35 |
36 |                     counter.setCounterId(
37 |                         resultSet.getInt("counter_id")
38 |                     );
39 |
40 |                     counter.setCounterName(
41 |                         resultSet.getString("counter_name")
42 |                     );
43 |
44 |                     counter.setServiceId(

```

```
45 |             resultSet.getInt("service_id")
46 |         );
47 |
48 |         counter.setActive(
49 |             resultSet.getBoolean("is_active")
50 |         );
51 |
52 |         counters.add(counter);
53 |     }
54 | }
55 |
56 | } catch (Exception e) {
57 |     e.printStackTrace();
58 | }
59 |
60 | return counters;
61 | }
62 |
63 |
64 | public Counter getCounterById(int counterId) {
65 |
66 |     String sql = "SELECT * FROM counters " +
67 |         "WHERE counter_id = ?";
68 |
69 |     try (
70 |         Connection connection = DBConnection.getConnection();
71 |         PreparedStatement statement = connection.prepareStatement(sql)
72 |     ) {
73 |
74 |         statement.setInt(1, counterId);
75 |
76 |         try (ResultSet resultSet = statement.executeQuery()) {
77 |
78 |             if (resultSet.next()) {
79 |
80 |                 Counter counter = new Counter();
81 |
82 |                 counter.setCounterId(
83 |                     resultSet.getInt("counter_id")
84 |                 );
85 |
86 |                 counter.setCounterName(
87 |                     resultSet.getString("counter_name")
88 |                 );
89 |
90 |                 counter.setServiceId(
91 |                     resultSet.getInt("service_id")
92 |                 );
93 |
94 |                 counter.setActive(
95 |                     resultSet.getBoolean("is_active")
96 |                 );
97 |
98 |                 return counter;
99 |             }
100 |         }
101 |
102 |     } catch (Exception e) {
103 |         e.printStackTrace();
104 |     }
105 |
106 |     return null;
107 | }
108 |
109 |
110 | public boolean updateCounterStatus(int counterId,
111 |     boolean active) {
112 |
113 |     String sql = "UPDATE counters " +
114 |         "SET is_active = ? " +
115 |         "WHERE counter_id = ?";
```

```

116 |
117 |         try (
118 |             Connection connection = DBConnection.getConnection();
119 |             PreparedStatement statement = connection.prepareStatement(sql)
120 |         ) {
121 |
122 |             statement.setBoolean(1, active);
123 |             statement.setInt(2, counterId);
124 |
125 |             return statement.executeUpdate() > 0;
126 |
127 |         } catch (Exception e) {
128 |             e.printStackTrace();
129 |         }
130 |
131 |         return false;
132 |     }
133 |
134 |     public java.util.Map<String, Object> getCounterDetails(int counterId) {
135 |
136 |         java.util.Map<String, Object> details =
137 |             new java.util.HashMap<>();
138 |
139 |         String sql =
140 |             "SELECT c.counter_id, c.counter_name, " +
141 |             "c.service_id, c.is_active, " +
142 |             "s.service_name " +
143 |             "FROM counters c " +
144 |             "JOIN services s ON c.service_id = s.service_id " +
145 |             "WHERE c.counter_id = ?";
146 |
147 |         try (
148 |             Connection connection = DBConnection.getConnection();
149 |             PreparedStatement statement =
150 |                 connection.prepareStatement(sql)
151 |         ) {
152 |
153 |             statement.setInt(1, counterId);
154 |
155 |             try (ResultSet resultSet = statement.executeQuery()) {
156 |
157 |                 if (resultSet.next()) {
158 |
159 |                     details.put(
160 |                         "counterId",
161 |                         resultSet.getInt("counter_id")
162 |                     );
163 |
164 |                     details.put(
165 |                         "counterName",
166 |                         resultSet.getString("counter_name")
167 |                     );
168 |
169 |                     details.put(
170 |                         "serviceId",
171 |                         resultSet.getInt("service_id")
172 |                     );
173 |
174 |                     details.put(
175 |                         "serviceName",
176 |                         resultSet.getString("service_name")
177 |
178 |                     );
179 |
180 |                     details.put(
181 |                         "active",
182 |                         resultSet.getBoolean("is_active")
183 |                     );
184 |                 }
185 |             }

```

```
186 |     } catch (Exception e) {  
187 |         e.printStackTrace();  
188 |     }  
189 |  
190 |     return details;  
191 | }  
192 | }
```

src/main/java/com/queue/dao/QueueDAO.java

Retained queue read/DAO operations and student ticket projections. Current transactional writes are mainly in ReservationService.

Read these lines first

Line 7: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 20: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 21: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 30: try-with-resources closes database/IO resources automatically when the block ends.

Line 32: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 36: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 37: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 39: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Numbered source

```

1 | package com.queue.dao;
2 |
3 | import com.queue.model.Queue;
4 | import com.queue.util.DBConnection;
5 |
6 | import java.sql.Connection;
7 | import java.sql.PreparedStatement;
8 | import java.sql.ResultSet;
9 |
10 | public class QueueDAO {
11 |
12 |     // =====
13 |     // STUDENT QUEUE OPERATIONS
14 |     // =====
15 |
16 |     public boolean joinQueue(Queue queue) {
17 |         try {
18 |             java.util.Map<String,Object> result=new
19 | com.queue.service.ReservationService().reserve(queue.getStudentId(),queue.getServiceId());
20 |             queue.setQueueId(((Number)result.get("queueId")).intValue());queue.setTokenNumber(Integer.p
21 | arseInt(result.get("token").toString()));queue.setStatus("WAITING");return true;
22 |         } catch (IllegalArgumentException conflict) { return false; }
23 |         catch (java.sql.SQLException error) { throw new IllegalStateException("Unable to reserve a
24 | place.",error); }
25 |     }
26 |     public Queue getQueueStatus(int studentId, int serviceId) {
27 |
28 |         String sql =
29 |             "SELECT queue.* FROM queue JOIN queue_reservations r ON r.queue_id=queue.queue_id " +
30 |             "WHERE student_id = ? AND service_id = ? AND r.visit_date >=
31 | DATE(CONVERT_TZ(UTC_TIMESTAMP(), '+00:00', '+05:30')) " +
32 |             "ORDER BY queue.queue_id DESC LIMIT 1";
33 |
34 |         try (
35 |             Connection connection = DBConnection.getConnection();
36 |             PreparedStatement statement =
37 |                 connection.prepareStatement(sql)
38 |         ) {
39 |
40 |             statement.setInt(1, studentId);
41 |             statement.setInt(2, serviceId);
42 |
43 |             try (ResultSet resultSet = statement.executeQuery()) {

```

```

41 |         if (resultSet.next()) {
42 |
43 |             return mapQueue(resultSet);
44 |         }
45 |     }
46 |
47 |     } catch (Exception e) {
48 |         e.printStackTrace();
49 |     }
50 |
51 |     return null;
52 | }
53 |
54 |
55 |     public int getQueuePosition(int studentId,int serviceId) {
56 |         try { java.util.Map<String,Object> preview=new
57 | com.queue.service.ReservationService().preview(studentId,serviceId);return
58 | preview.get("queueId")==null?-1:((Number)preview.get("position")).intValue(); }
59 |         catch(Exception error) { throw new IllegalStateException("Unable to read queue
60 | position.",error); }
61 |     }
62 |     public int getCurrentToken(int serviceId) {
63 |
64 |         String sql =
65 |             "SELECT token_number FROM queue JOIN queue_reservations r ON r.queue_id=queue.queue_id
66 | " +
67 |             "WHERE service_id = ? " +
68 |             "AND status IN ('CALLED', 'SERVING') AND r.visit_date=DATE(CONVERT_TZ(UTC_TIMESTAMP(),
69 | '+00:00', '+05:30')) " +
70 |             "ORDER BY token_number DESC LIMIT 1";
71 |
72 |         try (
73 |             Connection connection = DBConnection.getConnection();
74 |             PreparedStatement statement =
75 |                 connection.prepareStatement(sql)
76 |         ) {
77 |
78 |             statement.setInt(1, serviceId);
79 |
80 |             try (ResultSet resultSet = statement.executeQuery()) {
81 |
82 |                 if (resultSet.next()) {
83 |                     return resultSet.getInt("token_number");
84 |                 }
85 |
86 |             }
87 |
88 |         } catch (Exception e) {
89 |             e.printStackTrace();
90 |         }
91 |
92 |         return 0;
93 |     }
94 |
95 |     public java.util.List<Queue> getActiveQueue(int serviceId) {
96 |
97 |         java.util.List<Queue> queueList =
98 |             new java.util.ArrayList<>();
99 |
100 |         String sql =
101 |             "SELECT queue.* FROM queue JOIN queue_reservations r ON r.queue_id=queue.queue_id " +
102 |             "WHERE service_id = ? " +
103 |             "AND status IN ('WAITING', 'CALLED', 'SERVING') AND r.visit_date =
104 | DATE(CONVERT_TZ(UTC_TIMESTAMP(), '+00:00', '+05:30')) " +
105 |             "ORDER BY CASE WHEN status IN ('CALLED','SERVING') THEN 0 ELSE 1
106 | END,r.queue_order,queue.queue_id";
107 |
108 |         try (
109 |             Connection connection = DBConnection.getConnection();
110 |             PreparedStatement statement =
111 |                 connection.prepareStatement(sql)
112 |         ) {

```

```

105 |
106 |         statement.setInt(1, serviceId);
107 |
108 |         try (ResultSet resultSet =
109 |             statement.executeQuery()) {
110 |
111 |             while (resultSet.next()) {
112 |                 queueList.add(mapQueue(resultSet));
113 |             }
114 |         }
115 |
116 |     } catch (Exception e) {
117 |         e.printStackTrace();
118 |     }
119 |
120 |     return queueList;
121 | }

122 |
123 |
124 |     // =====
125 |     // STAFF QUEUE OPERATIONS
126 |     // =====
127 |
128 |     public Queue callStudent(int queueId) {
129 |
130 |         String sql =
131 |             "UPDATE queue " +
132 |             "SET status = 'CALLED', " +
133 |             "called_at = CURRENT_TIMESTAMP " +
134 |             "WHERE queue_id = ? " +
135 |             "AND status = 'WAITING'";
136 |
137 |         try (
138 |             Connection connection = DBConnection.getConnection();
139 |             PreparedStatement statement =
140 |                 connection.prepareStatement(sql)
141 |         ) {
142 |
143 |             statement.setInt(1, queueId);
144 |
145 |             int updated = statement.executeUpdate();
146 |
147 |             if (updated > 0) {
148 |                 return getQueueById(connection, queueId);
149 |             }
150 |
151 |         } catch (Exception e) {
152 |             e.printStackTrace();
153 |         }
154 |
155 |         return null;
156 |     }
157 |
158 |     public Queue callNext(int serviceId) {
159 |
160 |         String findSql =
161 |             "SELECT queue.* FROM queue JOIN queue_reservations r ON r.queue_id=queue.queue_id " +
162 |             "WHERE service_id = ? " +
163 |             "AND status = 'WAITING' " +
164 |             "ORDER BY token_number ASC " +
165 |             "LIMIT 1 FOR UPDATE";
166 |
167 |         String updateSql =
168 |             "UPDATE queue " +
169 |             "SET status = 'CALLED', called_at = CURRENT_TIMESTAMP " +
170 |             "WHERE queue_id = ? AND status = 'WAITING'";
171 |
172 |         try (Connection connection = DBConnection.getConnection()) {
173 |
174 |             connection.setAutoCommit(false);

```

```
175 |
176 |         try (
177 |             PreparedStatement findStatement =
178 |                 connection.prepareStatement(findSql);
179 |             PreparedStatement updateStatement =
180 |                 connection.prepareStatement(updateSql)
181 |         ) {
182 |
183 |             findStatement.setInt(1, serviceId);
184 |
185 |             try (ResultSet resultSet =
186 |                 findStatement.executeQuery()) {
187 |
188 |                 if (!resultSet.next()) {
189 |                     connection.rollback();
190 |                     return null;
191 |                 }
192 |
193 |                 int queueId =
194 |                     resultSet.getInt("queue_id");
195 |
196 |                 updateStatement.setInt(1, queueId);
197 |
198 |                 int updated =
199 |                     updateStatement.executeUpdate();
200 |
201 |                 if (updated == 0) {
202 |                     connection.rollback();
203 |                     return null;
204 |                 }
205 |
206 |                 connection.commit();
207 |
208 |                 return getQueueById(connection, queueId);
209 |             }
210 |
211 |         } catch (Exception e) {
212 |
213 |             connection.rollback();
214 |             throw e;
215 |
216 |         } finally {
217 |
218 |             connection.setAutoCommit(true);
219 |         }
220 |
221 |     } catch (Exception e) {
222 |         e.printStackTrace();
223 |     }
224 |
225 |     return null;
226 | }
227 |
228 |
229 | public boolean startService(int queueId) {
230 |
231 |     String sql =
232 |         "UPDATE queue " +
233 |         "SET status = 'SERVING', " +
234 |         "started_at = CURRENT_TIMESTAMP " +
235 |         "WHERE queue_id = ? " +
236 |         "AND status = 'CALLED'";
237 |
238 |     try (
239 |         Connection connection = DBConnection.getConnection();
240 |         PreparedStatement statement =
241 |             connection.prepareStatement(sql)
242 |     ) {
243 |
244 |         statement.setInt(1, queueId);
245 |
```

```

246 |         return statement.executeUpdate() > 0;
247 |
248 |     } catch (Exception e) {
249 |         e.printStackTrace();
250 |     }
251 |
252 |     return false;
253 | }

254 |
255 |
256 | public boolean completeService(int queueId) {
257 |
258 |     String sql =
259 |         "UPDATE queue " +
260 |         "SET status = 'COMPLETED', " +
261 |         "completed_at = CURRENT_TIMESTAMP " +
262 |         "WHERE queue_id = ? " +
263 |         "AND status = 'SERVING'";
264 |
265 |     try (
266 |         Connection connection = DBConnection.getConnection();
267 |         PreparedStatement statement =
268 |             connection.prepareStatement(sql)
269 |     ) {
270 |
271 |         statement.setInt(1, queueId);
272 |
273 |         return statement.executeUpdate() > 0;
274 |
275 |     } catch (Exception e) {
276 |         e.printStackTrace();
277 |     }
278 |
279 |     return false;
280 | }
281 |
282 |
283 | public boolean skipStudent(int queueId) {
284 |
285 |     String sql =
286 |         "UPDATE queue " +
287 |         "SET status = 'SKIPPED', " +
288 |         "completed_at = CURRENT_TIMESTAMP " +
289 |         "WHERE queue_id = ? " +
290 |         "AND status IN ('CALLED', 'SERVING')";
291 |
292 |     try (
293 |         Connection connection = DBConnection.getConnection();
294 |         PreparedStatement statement =
295 |             connection.prepareStatement(sql)
296 |     ) {
297 |
298 |         statement.setInt(1, queueId);
299 |
300 |         return statement.executeUpdate() > 0;
301 |
302 |     } catch (Exception e) {
303 |         e.printStackTrace();
304 |     }
305 |
306 |     return false;
307 | }
308 |
309 | public java.util.List<Queue> getRecentActivity(int serviceId) {
310 |
311 |     java.util.List<Queue> activityList =
312 |         new java.util.ArrayList<>();
313 |
314 |     String sql =
315 |         "SELECT queue.* FROM queue JOIN queue_reservations r ON r.queue_id=queue.queue_id " +

```

```

316 |         "WHERE service_id = ? " +
317 |         "AND (" +
318 |         "called_at IS NOT NULL " +
319 |         "OR started_at IS NOT NULL " +
320 |         "OR completed_at IS NOT NULL" +
321 |         ") " +
322 |         "ORDER BY " +
323 |         "COALESCE(completed_at, started_at, called_at) DESC " +
324 |         "LIMIT 10";
325 |
326 |     try (
327 |         Connection connection = DBConnection.getConnection();
328 |         PreparedStatement statement =
329 |             connection.prepareStatement(sql)
330 |     ) {
331 |
332 |         statement.setInt(1, serviceId);
333 |
334 |         try (ResultSet resultSet =
335 |             statement.executeQuery()) {
336 |
337 |             while (resultSet.next()) {
338 |
339 |                 activityList.add(
340 |                     mapQueue(resultSet)
341 |                 );
342 |
343 |             }
344 |
345 |         } catch (Exception e) {
346 |             e.printStackTrace();
347 |         }
348 |
349 |         return activityList;
350 |     }
351 |
352 |
353 |     // =====
354 |     // HELPER METHODS
355 |     // =====
356 |
357 |     public Queue getQueueById(int queueId) {
358 |
359 |         try (
360 |             Connection connection = DBConnection.getConnection()
361 |         ) {
362 |             return getQueueById(connection, queueId);
363 |
364 |         } catch (Exception e) {
365 |             e.printStackTrace();
366 |         }
367 |
368 |         return null;
369 |     }
370 |
371 |     private Queue getQueueById(Connection connection,
372 |                                 int queueId) throws Exception {
373 |
374 |         String sql =
375 |             "SELECT queue.* FROM queue JOIN queue_reservations r ON r.queue_id=queue.queue_id " +
376 |             "WHERE queue.queue_id = ?";
377 |
378 |         try (
379 |             PreparedStatement statement =
380 |                 connection.prepareStatement(sql)
381 |         ) {
382 |
383 |             statement.setInt(1, queueId);
384 |
385 |             try (ResultSet resultSet =

```

```
386 |             statement.executeQuery()) {
387 |
388 |                 if (resultSet.next()) {
389 |                     return mapQueue(resultSet);
390 |                 }
391 |             }
392 |         }
393 |
394 |         return null;
395 |     }
396 |
397 |
398 |     private Queue mapQueue(ResultSet resultSet)
399 |         throws Exception {
400 |
401 |         Queue queue = new Queue();
402 |
403 |         queue.setQueueId(
404 |             resultSet.getInt("queue_id")
405 |         );
406 |
407 |         queue.setStudentId(
408 |             resultSet.getInt("student_id")
409 |         );
410 |
411 |         queue.setServiceId(
412 |             resultSet.getInt("service_id")
413 |         );
414 |
415 |         queue.setTokenNumber(
416 |             resultSet.getInt("token_number")
417 |         );
418 |
419 |         queue.setStatus(
420 |             resultSet.getString("status")
421 |         );
422 |
423 |         queue.setJoinedAt(
424 |             resultSet.getTimestamp("joined_at")
425 |         );
426 |
427 |         queue.setCalledAt(
428 |             resultSet.getTimestamp("called_at")
429 |         );
430 |
431 |         queue.setStartedAt(
432 |             resultSet.getTimestamp("started_at")
433 |         );
434 |
435 |         queue.setCompletedAt(
436 |             resultSet.getTimestamp("completed_at")
437 |         );
438 |
439 |         return queue;
440 |     }
441 | }
```

src/main/java/com/queue/dao/ServiceDAO.java

Catalogue SELECT and model mapping. Now throws on failure so the servlet does not fabricate an empty successful result.

Read these lines first

Line 7: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 20: try-with-resources closes database/IO resources automatically when the block ends.

Line 22: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 23: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Line 49: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | package com.queue.dao;
2 |
3 | import com.queue.model.Service;
4 | import com.queue.util.DBConnection;
5 |
6 | import java.sql.Connection;
7 | import java.sql.PreparedStatement;
8 | import java.sql.ResultSet;
9 | import java.util.ArrayList;
10 | import java.util.List;
11 |
12 | public class ServiceDAO {
13 |
14 |     public List<Service> getAllServices() {
15 |
16 |         List<Service> services = new ArrayList<>();
17 |
18 |         String sql = "SELECT * FROM services";
19 |
20 |         try (
21 |             Connection connection = DBConnection.getConnection();
22 |             PreparedStatement statement = connection.prepareStatement(sql);
23 |             ResultSet resultSet = statement.executeQuery()
24 |         ) {
25 |
26 |             while (resultSet.next()) {
27 |
28 |                 Service service = new Service();
29 |
30 |                 service.setServiceId(
31 |                     resultSet.getInt("service_id")
32 |                 );
33 |
34 |                 service.setServiceName(
35 |                     resultSet.getString("service_name")
36 |                 );
37 |
38 |                 service.setDescription(
39 |                     resultSet.getString("description")
40 |                 );
41 |
42 |                 service.setAverageServiceTime(
43 |                     resultSet.getInt("average_service_time")
44 |                 );
45 |
46 |                 services.add(service);
47 |             }
48 |

```

```
49 |         } catch (Exception e) {  
50 |             throw new IllegalStateException("Cannot load service catalogue", e);  
51 |         }  
52 |  
53 |         return services;  
54 |     }  
55 | }
```

src/main/java/com/queue/dao/StaffDAO.java

Staff SQL login/profile queries and password verification. Assigned service and counter originate from stored account data.

Read these lines first

Line 7: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 17: try-with-resources closes database/IO resources automatically when the block ends.

Line 19: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 22: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 24: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Line 30: Password key derivation: random salt plus iteration count slows guessing; this is hashing rather than reversible encryption.

Line 31: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 32: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Numbered source

```

1 | package com.queue.dao;
2 |
3 | import com.queue.model.Staff;
4 | import com.queue.util.DBConnection;
5 |
6 | import java.sql.Connection;
7 | import java.sql.PreparedStatement;
8 | import java.sql.ResultSet;
9 |
10 | public class StaffDAO {
11 |
12 |     public Staff loginStaff(String email, String password) {
13 |
14 |         String sql = "SELECT * FROM staff " +
15 |             "WHERE email = ?";
16 |
17 |         try (
18 |             Connection connection = DBConnection.getConnection();
19 |             PreparedStatement statement = connection.prepareStatement(sql)
20 |         ) {
21 |
22 |             statement.setString(1, email);
23 |
24 |             try (ResultSet resultSet = statement.executeQuery()) {
25 |
26 |                 if (resultSet.next()) {
27 |
28 |                     String stored = resultSet.getString("password");
29 |                     if (!com.queue.util.Passwords.matches(password, stored)) return null;
30 |                     if (!stored.startsWith("pbkdf2$")) {
31 |                         try (PreparedStatement migrate = connection.prepareStatement("UPDATE staff SET
32 | password=? WHERE staff_id=? AND password=?")) {
33 |                             migrate.setString(1, com.queue.util.Passwords.hash(password));
34 |                             migrate.setInt(2, resultSet.getInt("staff_id"));
35 |                             migrate.setString(3, stored);
36 |                             migrate.executeUpdate();
37 |                         }
38 |
39 |                         Staff staff = new Staff();
40 |
41 |                         staff.setStaffId(
42 |                             resultSet.getInt("staff_id")
43 |                         );
44 |
45 |                         staff.setStaffName(
46 |                             resultSet.getString("staff_name")
47 |                         );
48 |
49 |                         staff.setEmail(
50 |                             resultSet.getString("email")
51 |                         );
52 |
53 |                         staff.setPassword(
54 |                             resultSet.getString("password")
55 |                         );
56 |
57 |                         staff.setServiceId(
58 |                             resultSet.getInt("service_id")
59 |                         );
60 |
61 |                         staff.setCounterId(
62 |                             resultSet.getInt("counter_id")
63 |                         );
64 |
65 |                         return staff;
66 |                     }
67 |                 }
68 |

```

```
69 |         } catch (Exception e) {
70 |             e.printStackTrace();
71 |         }
72 |
73 |         return null;
74 |     }
75 |
76 |
77 |     public Staff getStaffById(int staffId) {
78 |
79 |         String sql = "SELECT * FROM staff WHERE staff_id = ?";
80 |
81 |         try (
82 |             Connection connection = DBConnection.getConnection();
83 |             PreparedStatement statement = connection.prepareStatement(sql)
84 |         ) {
85 |
86 |             statement.setInt(1, staffId);
87 |
88 |
89 |             try (ResultSet resultSet = statement.executeQuery()) {
90 |
91 |                 if (resultSet.next()) {
92 |
93 |                     Staff staff = new Staff();
94 |
95 |                     staff.setStaffId(
96 |                         resultSet.getInt("staff_id")
97 |                     );
98 |
99 |                     staff.setStaffName(
100 |                         resultSet.getString("staff_name")
101 |                     );
102 |
103 |                     staff.setEmail(
104 |                         resultSet.getString("email")
105 |                     );
106 |
107 |                     staff.setPassword(
108 |                         resultSet.getString("password")
109 |                     );
110 |
111 |                     staff.setServiceId(
112 |                         resultSet.getInt("service_id")
113 |                     );
114 |
115 |                     staff.setCounterId(
116 |                         resultSet.getInt("counter_id")
117 |                     );
118 |
119 |                     return staff;
120 |                 }
121 |
122 |             } catch (Exception e) {
123 |                 e.printStackTrace();
124 |             }
125 |
126 |             return null;
127 |         }
128 |     }
```

src/main/java/com/queue/dao/StudentDAO.java

Student SQL storage/login operations, password verification and hash migration. Read PreparedStatement bindings and generated IDs.

Read these lines first

Line 8: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 20: try-with-resources closes database/IO resources automatically when the block ends.

Line 22: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 25: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 26: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 27: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 28: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 30: Executes a write and returns affected-row count; it does not alone guarantee the full business transaction succeeded.

Numbered source

```

1 | package com.queue.dao;
2 |
3 | import com.queue.model.Student;
4 | import com.queue.util.Passwords;
5 | import com.queue.util.DBConnection;
6 |
7 | import java.sql.Connection;
8 | import java.sql.PreparedStatement;
9 | import java.sql.ResultSet;
10 |
11 | public class StudentDAO {
12 |
13 |     public boolean registerStudent(Student student) {
14 |         com.queue.util.StudentValidation.registration(student);
15 |
16 |         String sql = "INSERT INTO students " +
17 |             "(student_name, email, phone, password) " +
18 |             "VALUES (?, ?, ?, ?)";
19 |
20 |         try (
21 |             Connection connection = DBConnection.getConnection();
22 |             PreparedStatement statement = connection.prepareStatement(sql)
23 |         ) {
24 |
25 |             statement.setString(1, student.getStudentName());
26 |             statement.setString(2, student.getEmail());
27 |             statement.setString(3, student.getPhone());
28 |             statement.setString(4, Passwords.hash(student.getPassword()));
29 |
30 |             int rowsInserted = statement.executeUpdate();
31 |
32 |             return rowsInserted > 0;
33 |
34 |         } catch (Exception e) {
35 |             e.printStackTrace();
36 |             return false;
37 |         }
38 |     }
39 |
40 |
41 |     public Student loginStudent(String email, String password) {
42 |
43 |         String sql = "SELECT * FROM students " +
44 |             "WHERE email = ?";

```

```
45 |  
46 |     try (  
47 |         Connection connection = DBConnection.getConnection();  
48 |         PreparedStatement statement = connection.prepareStatement(sql)  
49 |     ) {  
50 |  
51 |         statement.setString(1, email);  
52 |  
53 |  
54 |         ResultSet resultSet = statement.executeQuery();  
55 |  
56 |         if (resultSet.next() && Passwords.matches(password, resultSet.getString("password"))) {  
57 |  
58 |             Student student = new Student();  
59 |  
60 |             student.setStudentId(resultSet.getInt("student_id"));  
61 |             student.setStudentName(resultSet.getString("student_name"));  
62 |             student.setEmail(resultSet.getString("email"));  
63 |             student.setPhone(resultSet.getString("phone"));  
64 |             student.setPassword(resultSet.getString("password"));  
65 |  
66 |             if (!student.getPassword().startsWith("pbkdf2$")) {  
67 |                 try (PreparedStatement migrate = connection.prepareStatement("UPDATE students SET  
68 | password = ? WHERE student_id = ? AND password = ?")) {  
69 |                     migrate.setString(1, Passwords.hash(password)); migrate.setInt(2,  
70 | student.getId()); migrate.setString(3, student.getPassword()); migrate.executeUpdate();  
71 |                 }  
72 |                 return student;  
73 |             }  
74 |         } catch (Exception e) {  
75 |             e.printStackTrace();  
76 |         }  
77 |  
78 |         return null;  
79 |     }  
80 | }
```

src/main/java/com/queue/filter/StudentOwnershipFilter.java

Request filter protecting older endpoints using authenticated ownership/service checks. Not a substitute for current service validation.

Read these lines first

Line 20: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 29: Uses server-side login state. Distinguish session identity from untrusted request parameters.

Line 35: Reads client-supplied input; validation and authorization are needed before using it.

Line 39: Reads client-supplied input; validation and authorization are needed before using it.

Line 46: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | package com.queue.filter;
2 |
3 | import com.queue.service.BookingService;
4 | import com.queue.util.Json;
5 | import jakarta.servlet.*;
6 | import jakarta.servlet.annotation.WebFilter;
7 | import jakarta.servlet.http.*;
8 | import java.io.IOException;
9 | import java.util.Map;
10 | import java.util.Set;
11 |
12 | /** Protect the existing personal student routes as well as the new portal. */
13 | @WebFilter(urlPatterns = {"/api/queue/*", "/api/bookings/*", "/create-booking", "/cancel-booking",
14 |   | "/booking-status"})
15 | public class StudentOwnershipFilter implements Filter {
16 |     public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) throws
17 |       | IOException, ServletException {
18 |         HttpServletRequest req = (HttpServletRequest) request;
19 |         HttpServletResponse res = (HttpServletResponse) response;
20 |         String route = req.getServletPath() + (req.getPathInfo() == null ? "" : req.getPathInfo());
21 |         if ("/api/bookings/upcoming".equals(route)) {
22 |             HttpSession staffSession = req.getSession(false);
23 |             if (staffSession == null || staffSession.getAttribute("staffId") == null) {
24 |                 reject(res, 401, "Staff sign-in is required."); return;
25 |             }
26 |         }
27 |         boolean personal = Set.of("/api/queue/join", "/api/queue/status", "/api/queue/position",
28 |           | "/api/bookings/create", "/api/bookings/cancel", "/api/bookings/status",
29 |           | "/create-booking", "/cancel-booking", "/booking-status").contains(route);
30 |         if (!personal) { chain.doFilter(request, response); return; }
31 |         HttpSession session = req.getSession(false);
32 |         if (session == null || !(session.getAttribute("studentId") instanceof Integer)) {
33 |             reject(res, 401, "Student sign-in is required."); return;
34 |         }
35 |         int studentId = (Integer) session.getAttribute("studentId");
36 |         try {
37 |             String claimedId = req.getParameter("studentId");
38 |             if (claimedId == null || Integer.parseInt(claimedId) != studentId) {
39 |                 reject(res, 403, "This request does not belong to your student account."); return;
40 |             }
41 |             String bookingId = req.getParameter("bookingId");
42 |             if (bookingId != null) {
43 |                 var booking = new BookingService().getBookingById(Integer.parseInt(bookingId));
44 |                 if (booking == null || booking.getStudentId() != studentId) {

```

```
45 |         }
46 |     } catch (NumberFormatException error) { reject(res, 400, "Invalid account or booking
| identifier."); return; }
47 |         chain.doFilter(request, response);
48 |     }
49 |     private void reject(HttpServletResponse res, int status, String message) throws IOException {
50 |         res.setStatus(status); res.setContentType("application/json");
| res.setCharacterEncoding("UTF-8");
51 |         res.getWriter().write(Json.encode(Map.of("success", false, "message", message)));
52 |     }
53 | }
```

src/main/java/com/queue/model/Booking.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

Numbered source

```
1 | package com.queue.model;
2 |
3 | import java.sql.Date;
4 | import java.sql.Time;
5 | import java.sql.Timestamp;
6 |
7 | public class Booking {
8 |
9 |     private int bookingId;
10 |    private int studentId;
11 |    private int serviceId;
12 |    private Date bookingDate;
13 |    private Time bookingTime;
14 |    private String status;
15 |    private Timestamp createdAt;
16 |
17 |    public Booking() {
18 |    }
19 |
20 |    public Booking(int bookingId, int studentId, int serviceId,
21 |                  Date bookingDate, Time bookingTime,
22 |                  String status, Timestamp createdAt) {
23 |
24 |        this.bookingId = bookingId;
25 |        this.studentId = studentId;
26 |        this.serviceId = serviceId;
27 |        this.bookingDate = bookingDate;
28 |        this.bookingTime = bookingTime;
29 |        this.status = status;
30 |        this.createdAt = createdAt;
31 |    }
32 |
33 |    public int getBookingId() {
34 |        return bookingId;
35 |    }
36 |
37 |    public void setBookingId(int bookingId) {
38 |        this.bookingId = bookingId;
39 |    }
40 |
41 |    public int getStudentId() {
42 |        return studentId;
43 |    }
44 |
45 |    public void setStudentId(int studentId) {
46 |        this.studentId = studentId;
47 |    }
48 |
49 |    public int getServiceId() {
50 |        return serviceId;
51 |    }
52 |
53 |    public void setServiceId(int serviceId) {
54 |        this.serviceId = serviceId;
55 |    }
56 |
57 |    public Date getBookingDate() {
58 |        return bookingDate;
59 |    }
60 |
61 |    public void setBookingDate(Date bookingDate) {
```

```
62 |         this.bookingDate = bookingDate;
63 |     }
64 |
65 |     public Time getBookingTime() {
66 |         return bookingTime;
67 |     }
68 |
69 |     public void setBookingTime(Time bookingTime) {
70 |         this.bookingTime = bookingTime;
71 |     }
72 |
73 |     public String getStatus() {
74 |         return status;
75 |     }
76 |
77 |     public void setStatus(String status) {
78 |         this.status = status;
79 |     }
80 |
81 |     public Timestamp getCreatedAt() {
82 |         return createdAt;
83 |     }
84 |
85 |     public void setCreatedAt(Timestamp createdAt) {
86 |         this.createdAt = createdAt;
87 |     }
88 | }
```

src/main/java/com/queue/model/Counter.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

Numbered source

```
1 | package com.queue.model;
2 |
3 | public class Counter {
4 |
5 |     private int counterId;
6 |     private String counterName;
7 |     private int serviceId;
8 |     private boolean active;
9 |
10 |    public Counter() {
11 |    }
12 |
13 |    public Counter(int counterId, String counterName,
14 |                  int serviceId, boolean active) {
15 |        this.counterId = counterId;
16 |        this.counterName = counterName;
17 |        this.serviceId = serviceId;
18 |        this.active = active;
19 |    }
20 |
21 |    public int getCounterId() {
22 |        return counterId;
23 |    }
24 |
25 |    public void setCounterId(int counterId) {
26 |        this.counterId = counterId;
27 |    }
28 |
29 |    public String getCounterName() {
30 |        return counterName;
31 |    }
32 |
33 |    public void setCounterName(String counterName) {
34 |        this.counterName = counterName;
35 |    }
36 |
37 |    public int getServiceId() {
38 |        return serviceId;
39 |    }
40 |
41 |    public void setServiceId(int serviceId) {
42 |        this.serviceId = serviceId;
43 |    }
44 |
45 |    public boolean isActive() {
46 |        return active;
47 |    }
48 |
49 |    public void setActive(boolean active) {
50 |        this.active = active;
51 |    }
52 | }
```

src/main/java/com/queue/model/Queue.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

Numbered source

```
1 | package com.queue.model;
2 |
3 | import java.sql.Timestamp;
4 |
5 | public class Queue {
6 |
7 |     private int queueId;
8 |     private int studentId;
9 |     private int serviceId;
10 |    private int tokenNumber;
11 |    private String status;
12 |    private Timestamp joinedAt;
13 |    private Timestamp calledAt;
14 |    private Timestamp startedAt;
15 |    private Timestamp completedAt;
16 |
17 |    public Queue() {
18 |    }
19 |
20 |    public Queue(int queueId, int studentId, int serviceId,
21 |                int tokenNumber, String status,
22 |                Timestamp joinedAt, Timestamp calledAt,
23 |                Timestamp startedAt, Timestamp completedAt) {
24 |
25 |        this.queueId = queueId;
26 |        this.studentId = studentId;
27 |        this.serviceId = serviceId;
28 |        this.tokenNumber = tokenNumber;
29 |        this.status = status;
30 |        this.joinedAt = joinedAt;
31 |        this.calledAt = calledAt;
32 |        this.startedAt = startedAt;
33 |        this.completedAt = completedAt;
34 |    }
35 |
36 |    public int getQueueId() {
37 |        return queueId;
38 |    }
39 |
40 |    public void setQueueId(int queueId) {
41 |        this.queueId = queueId;
42 |    }
43 |
44 |    public int getStudentId() {
45 |
46 |        return studentId;
47 |    }
48 |
49 |    public void setStudentId(int studentId) {
50 |        this.studentId = studentId;
51 |    }
52 |
53 |    public int getServiceId() {
54 |        return serviceId;
55 |    }
56 |
57 |    public void setServiceId(int serviceId) {
58 |        this.serviceId = serviceId;
59 |    }
60 |
61 |    public int getTokenNumber() {
62 |        return tokenNumber;
63 |    }
64 |
65 | }
```

```
62 |     }
63 |
64 |     public void setTokenNumber(int tokenNumber) {
65 |         this.tokenNumber = tokenNumber;
66 |     }
67 |
68 |     public String getStatus() {
69 |         return status;
70 |     }
71 |
72 |     public void setStatus(String status) {
73 |         this.status = status;
74 |     }
75 |
76 |     public Timestamp getJoinedAt() {
77 |         return joinedAt;
78 |     }
79 |
80 |     public void setJoinedAt(Timestamp joinedAt) {
81 |         this.joinedAt = joinedAt;
82 |     }
83 |
84 |     public Timestamp getCalledAt() {
85 |         return calledAt;
86 |     }
87 |
88 |     public void setCalledAt(Timestamp calledAt) {
89 |
90 |         this.calledAt = calledAt;
91 |     }
92 |
93 |     public Timestamp getStartedAt() {
94 |         return startedAt;
95 |     }
96 |
97 |     public void setStartedAt(Timestamp startedAt) {
98 |         this.startedAt = startedAt;
99 |     }
100 |
101 |     public Timestamp getCompletedAt() {
102 |         return completedAt;
103 |     }
104 |
105 |     public void setCompletedAt(Timestamp completedAt) {
106 |         this.completedAt = completedAt;
107 |     }
```

src/main/java/com/queue/model/Service.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

Numbered source

```
1 | package com.queue.model;
2 |
3 | public class Service {
4 |
5 |     private int serviceId;
6 |     private String serviceName;
7 |     private String description;
8 |     private int averageServiceTime;
9 |
10 |    public Service() {
11 |    }
12 |
13 |    public Service(int serviceId, String serviceName,
14 |                  String description, int averageServiceTime) {
15 |        this.serviceId = serviceId;
16 |        this.serviceName = serviceName;
17 |        this.description = description;
18 |        this.averageServiceTime = averageServiceTime;
19 |    }
20 |
21 |    public int getServiceId() {
22 |        return serviceId;
23 |    }
24 |
25 |    public void setServiceId(int serviceId) {
26 |        this.serviceId = serviceId;
27 |    }
28 |
29 |    public String getServiceName() {
30 |        return serviceName;
31 |    }
32 |
33 |    public void setServiceName(String serviceName) {
34 |        this.serviceName = serviceName;
35 |    }
36 |
37 |    public String getDescription() {
38 |        return description;
39 |    }
40 |
41 |    public void setDescription(String description) {
42 |        this.description = description;
43 |    }
44 |
45 |    public int getAverageServiceTime() {
46 |        return averageServiceTime;
47 |    }
48 |
49 |    public void setAverageServiceTime(int averageServiceTime) {
50 |        this.averageServiceTime = averageServiceTime;
51 |    }
52 | }
```

src/main/java/com/queue/model/Staff.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

Numbered source

```
1 | package com.queue.model;
2 |
3 | public class Staff {
4 |
5 |     private int staffId;
6 |     private String staffName;
7 |     private String email;
8 |     private String password;
9 |     private int serviceId;
10 |    private int counterId;
11 |
12 |    public Staff() {
13 |    }
14 |
15 |    public Staff(int staffId, String staffName, String email,
16 |                String password, int serviceId, int counterId) {
17 |        this.staffId = staffId;
18 |        this.staffName = staffName;
19 |        this.email = email;
20 |        this.password = password;
21 |        this.serviceId = serviceId;
22 |        this.counterId = counterId;
23 |    }
24 |
25 |    public int getStaffId() {
26 |        return staffId;
27 |    }
28 |
29 |    public void setStaffId(int staffId) {
30 |        this.staffId = staffId;
31 |    }
32 |
33 |    public String getStaffName() {
34 |        return staffName;
35 |    }
36 |
37 |    public void setStaffName(String staffName) {
38 |        this.staffName = staffName;
39 |    }
40 |
41 |    public String getEmail() {
42 |        return email;
43 |    }
44 |
45 |    public void setEmail(String email) {
46 |        this.email = email;
47 |    }
48 |
49 |    public String getPassword() {
50 |        return password;
51 |    }
52 |
53 |    public void setPassword(String password) {
54 |        this.password = password;
55 |    }
56 |
57 |    public int getServiceId() {
58 |        return serviceId;
59 |    }
60 |
61 |    public void setServiceId(int serviceId) {
```

```
62 |         this.serviceId = serviceId;
63 |     }
64 |
65 |     public int getCounterId() {
66 |         return counterId;
67 |     }
68 |
69 |     public void setCounterId(int counterId) {
70 |         this.counterId = counterId;
71 |     }
72 | }
```

src/main/java/com/queue/model/Student.java

Java data model: fields, getters and setters transport structured values between SQL mapping and application logic. It does not itself enforce all business rules.

Numbered source

```
1 | package com.queue.model;
2 |
3 | public class Student {
4 |
5 |     private int studentId;
6 |     private String studentName;
7 |     private String email;
8 |     private String phone;
9 |     private String password;
10 |
11 |     public Student() {
12 |     }
13 |
14 |     public Student(int studentId, String studentName, String email,
15 |                   String phone, String password) {
16 |         this.studentId = studentId;
17 |         this.studentName = studentName;
18 |         this.email = email;
19 |         this.phone = phone;
20 |         this.password = password;
21 |     }
22 |
23 |     public int getStudentId() {
24 |         return studentId;
25 |     }
26 |
27 |     public void setStudentId(int studentId) {
28 |         this.studentId = studentId;
29 |     }
30 |
31 |     public String getStudentName() {
32 |         return studentName;
33 |     }
34 |
35 |     public void setStudentName(String studentName) {
36 |         this.studentName = studentName;
37 |     }
38 |
39 |     public String getEmail() {
40 |         return email;
41 |     }
42 |
43 |     public void setEmail(String email) {
44 |         this.email = email;
45 |     }
46 |
47 |     public String getPhone() {
48 |         return phone;
49 |     }
50 |
51 |     public void setPhone(String phone) {
52 |         this.phone = phone;
53 |     }
54 |
55 |     public String getPassword() {
56 |         return password;
57 |     }
58 |
59 |     public void setPassword(String password) {
60 |         this.password = password;
61 |     }
```

62 | }

src/main/java/com/queue/service/AnalyticsService.java

a

SQL aggregation for dashboard reporting. Some elapsed-time metrics are different from ETA scheduled-window processing samples.

Read these lines first

Line 6: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 26: try-with-resources closes database/IO resources automatically when the block ends.

Line 27: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 28: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Line 33: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 50: try-with-resources closes database/IO resources automatically when the block ends.

Line 51: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 52: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Numbered source

```

1 | package com.queue.service;
2 |
3 | import com.queue.util.DBConnection;
4 |
5 | import java.sql.Connection;
6 | import java.sql.PreparedStatement;
7 | import java.sql.ResultSet;
8 | import java.util.HashMap;
9 | import java.util.Map;
10 |
11 | /**
12 |  * Service class for computing system analytics and reporting data using SQL aggregation.
13 |  */
14 | public class AnalyticsService {
15 |
16 |     /**
17 |      * Retrieves the total count of students successfully served today.
18 |      * Uses SQL aggregation: COUNT(*) on completed queue records for CURRENT_DATE.
19 |      */
20 |     public int getStudentsServedToday() {
21 |         String sql =
22 |             "SELECT COUNT(*) AS count FROM queue " +
23 |             "WHERE status = 'COMPLETED' " +
24 |             "AND DATE(COALESCE(completed_at, joined_at)) = CURRENT_DATE";
25 |
26 |         try (Connection connection = DBConnection.getConnection();
27 |             PreparedStatement statement = connection.prepareStatement(sql);
28 |             ResultSet resultSet = statement.executeQuery()) {
29 |
30 |             if (resultSet.next()) {
31 |                 return resultSet.getInt("count");
32 |             }
33 |         } catch (Exception e) {
34 |             e.printStackTrace();
35 |         }
36 |         return 0;
37 |     }
38 |
39 |     /**
40 |      * Retrieves the average service time in minutes across all completed queue tickets.
41 |      * Uses SQL aggregation: AVG(TIMESTAMPDIFF(MINUTE, started_at, completed_at)).
42 |      */
43 |     public double getAverageServiceTime() {
44 |         String sql =
45 |
46 |             "SELECT AVG(TIMESTAMPDIFF(MINUTE, started_at, completed_at)) AS avg_service " +
47 |             "FROM queue " +
48 |             "WHERE status = 'COMPLETED' " +
49 |             "AND started_at IS NOT NULL AND completed_at IS NOT NULL";
50 |
51 |         try (Connection connection = DBConnection.getConnection();
52 |             PreparedStatement statement = connection.prepareStatement(sql);
53 |             ResultSet resultSet = statement.executeQuery()) {
54 |
55 |             if (resultSet.next()) {
56 |                 double avg = resultSet.getDouble("avg_service");
57 |                 return resultSet.isNull() ? 0.0 : Math.round(avg * 10.0) / 10.0;
58 |             }
59 |         } catch (Exception e) {
60 |             e.printStackTrace();
61 |         }
62 |         return 0.0;
63 |     }
64 |
65 |     /**
66 |      * Retrieves the average waiting time in minutes for students in the queue.
67 |      * Uses SQL aggregation: AVG(TIMESTAMPDIFF(MINUTE, joined_at, COALESCE(started_at, called_at))).
68 |      */
69 |     public double getAverageWaitingTime() {
70 |         String sql =

```

```

70 |         "SELECT AVG(TIMESTAMPDIFF(MINUTE, joined_at, COALESCE(started_at, called_at,
71 | completed_at))) AS avg_wait " +
72 |         "FROM queue " +
73 |         "WHERE joined_at IS NOT NULL " +
74 |         "AND (started_at IS NOT NULL OR called_at IS NOT NULL OR completed_at IS NOT NULL)";
75 |
76 |     try (Connection connection = DBConnection.getConnection();
77 |         PreparedStatement statement = connection.prepareStatement(sql);
78 |         ResultSet resultSet = statement.executeQuery()) {
79 |
80 |         if (resultSet.next()) {
81 |             double avg = resultSet.getDouble("avg_wait");
82 |             return resultSet.wasNull() ? 0.0 : Math.round(avg * 10.0) / 10.0;
83 |         }
84 |     } catch (Exception e) {
85 |         e.printStackTrace();
86 |     }
87 | }
88 |
89 | /**
90 |  * Calculates the peak traffic hour based on historical queue joins.
91 |  * Uses SQL aggregation: GROUP BY HOUR(joined_at) ORDER BY count DESC LIMIT 1.
92 |  *
93 |  * @return Formatted string representation of peak hour (e.g. "10:00 - 11:00") or "N/A"
94 |  */
95 | public String getPeakHour() {
96 |     String sql =
97 |         "SELECT HOUR(joined_at) AS hr, COUNT(*) AS queue_count " +
98 |         "FROM queue " +
99 |         "WHERE joined_at IS NOT NULL " +
100 |         "GROUP BY HOUR(joined_at) " +
101 |         "ORDER BY queue_count DESC LIMIT 1";
102 |
103 |     try (Connection connection = DBConnection.getConnection();
104 |         PreparedStatement statement = connection.prepareStatement(sql);
105 |         ResultSet resultSet = statement.executeQuery()) {
106 |
107 |         if (resultSet.next()) {
108 |             int hour = resultSet.getInt("hr");
109 |             int nextHour = (hour + 1) % 24;
110 |             return String.format("%02d:00 - %02d:00", hour, nextHour);
111 |         }
112 |     } catch (Exception e) {
113 |         e.printStackTrace();
114 |     }
115 |     return "N/A";
116 | }
117 |
118 | /**
119 |  * Calculates the total count of no-shows from bookings and skipped tickets from queue.
120 |  * Uses SQL aggregation.
121 |  */
122 |
123 | /**
124 |  * Calculates the count of no-shows and skipped tickets strictly for TODAY.
125 |  */
126 | public int getNoShowsToday() {
127 |     String sql =
128 |         "SELECT " +
129 |         "(SELECT COUNT(*) FROM bookings WHERE status = 'NO_SHOW' AND booking_date = CURRENT_DATE) +
130 |         " +
131 |         "(SELECT COUNT(*) FROM queue WHERE status = 'SKIPPED' AND DATE(joined_at) = CURRENT_DATE)
132 |         AS no_shows_today";
133 |
134 |     try (Connection connection = DBConnection.getConnection();
135 |         PreparedStatement statement = connection.prepareStatement(sql);
136 |         ResultSet resultSet = statement.executeQuery()) {
137 |
138 |         if (resultSet.next()) {

```

```

137 |         return resultSet.getInt("no_shows_today");
138 |     }
139 | } catch (Exception e) {
140 |     e.printStackTrace();
141 | }
142 |
143 |     return 0;
144 | }
145 |
146 | /**
147 |  * Calculates the count of no-shows and skipped tickets for a specific service for TODAY.
148 |  */
149 | public int getNoShowsToday(int serviceId) {
150 |     String sql =
151 |         "SELECT " +
152 |         "(SELECT COUNT(*) FROM bookings WHERE status = 'NO_SHOW' AND service_id = ? AND
153 | booking_date = CURRENT_DATE) + " +
154 |         "(SELECT COUNT(*) FROM queue WHERE status = 'SKIPPED' AND service_id = ? AND
155 | DATE(joined_at) = CURRENT_DATE) AS no_shows_today";
156 |
157 |     try (Connection connection = DBConnection.getConnection();
158 |         PreparedStatement statement = connection.prepareStatement(sql)) {
159 |
160 |         statement.setInt(1, serviceId);
161 |         statement.setInt(2, serviceId);
162 |
163 |         try (ResultSet resultSet = statement.executeQuery()) {
164 |             if (resultSet.next()) {
165 |                 return resultSet.getInt("no_shows_today");
166 |             }
167 |         } catch (Exception e) {
168 |             e.printStackTrace();
169 |         }
170 |     }
171 | }
172 |
173 | public int getNoShowCount() {
174 |     String sql =
175 |         "SELECT " +
176 |         "(SELECT COUNT(*) FROM bookings WHERE status = 'NO_SHOW') + " +
177 |         "(SELECT COUNT(*) FROM queue WHERE status = 'SKIPPED') AS total_no_shows";
178 |
179 |     try (Connection connection = DBConnection.getConnection();
180 |         PreparedStatement statement = connection.prepareStatement(sql);
181 |         ResultSet resultSet = statement.executeQuery()) {
182 |
183 |         if (resultSet.next()) {
184 |             return resultSet.getInt("total_no_shows");
185 |         }
186 |     } catch (Exception e) {
187 |         e.printStackTrace();
188 |     }
189 | }
190 |
191 | /**
192 |  * Returns a consolidated map of all analytics metrics for view and API rendering.
193 |  */
194 | public Map<String, Object> getAnalyticsSummary() {
195 |     Map<String, Object> data = new HashMap<>();
196 |     data.put("studentsServedToday", getStudentsServedToday());
197 |     data.put("averageServiceTime", getAverageServiceTime());
198 |     data.put("averageWaitingTime", getAverageWaitingTime());
199 |     data.put("peakHour", getPeakHour());
200 |     data.put("noShowCount", getNoShowCount());
201 |     return data;
202 | }
203 | /** Today's assigned-service metrics; only completed visits contribute durations. */
204 | public Map<String, Object> getServiceDaySummary(int serviceId) throws java.sql.SQLException {
205 |     java.time.LocalDate day = ServiceHours.now().toLocalDate();

```

```

206 |         Map<String,Object> data=new HashMap<>();
207 |         String sql="SELECT SUM(q.status='COMPLETED') served, SUM(q.status='NO_SHOW') missed, " +
208 |             "AVG(CASE WHEN q.status='COMPLETED' AND q.started_at=q.joined_at THEN
| TIMESTAMPDIFF(SECOND,q.joined_at,q.started_at)/60.0 END) wait_minutes, " +
209 |             "AVG(CASE WHEN q.status='COMPLETED' AND q.completed_at>=q.started_at THEN
| TIMESTAMPDIFF(SECOND,q.started_at,q.completed_at)/60.0 END) service_minutes " +
210 |             "FROM queue q JOIN queue_reservations r ON r.queue_id=q.queue_id WHERE q.service_id=? AND
| r.visit_date=?";
211 |         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement(sql)) {
212 |             p.setInt(1,serviceId);p.setObject(2,day);

213 |             try(ResultSet r=p.executeQuery()){r.next();data.put("servedToday",r.getInt("served"));data.
| put("noShowsToday",r.getInt("missed"));
214 |                 Double
| wait=r.getObject("wait_minutes")==null?null:Math.round(r.getDouble("wait_minutes")*10)/10.0;
215 |                 Double
| duration=r.getObject("service_minutes")==null?null:Math.round(r.getDouble("service_minutes")*10)/10.0;
216 |                 data.put("averageWaitMinutes",wait);data.put("averageServiceMinutes",duration);}
217 |             try(PreparedStatement peak=c.prepareStatement("SELECT HOUR(q.joined_at) hr FROM queue q
| JOIN queue_reservations r ON r.queue_id=q.queue_id WHERE q.service_id=? AND r.visit_date=? GROUP BY
| HOUR(q.joined_at) ORDER BY COUNT(*) DESC,hr LIMIT 1")) {
218 |                 peak.setInt(1,serviceId);peak.setObject(2,day);try(ResultSet
| r=peak.executeQuery()){data.put("peakHour",r.next()?String.format("%02d:00 -
| %02d:00",r.getInt(1),(r.getInt(1)+1)%24):"N/A");}
219 |             }
220 |         }
221 |         return data;
222 |     }}

```

src/main/java/com/queue/service/BookingService.java

a

Retained legacy fixed-slot booking logic. Do not present it as the current same-day reservation algorithm.

Read these lines first

Line 8: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 51: try-with-resources closes database/IO resources automatically when the block ends.

Line 52: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 54: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 58: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Line 64: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 82: try-with-resources closes database/IO resources automatically when the block ends.

Line 83: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Numbered source

```

1 | package com.queue.service;
2 |
3 | import com.queue.model.Booking;
4 | import com.queue.util.DBConnection;
5 |
6 | import java.sql.Connection;
7 | import java.sql.Date;
8 | import java.sql.PreparedStatement;
9 | import java.sql.ResultSet;
10 | import java.sql.Time;
11 | import java.time.LocalDate;
12 | import java.time.LocalTime;
13 | import java.util.ArrayList;
14 | import java.util.Arrays;
15 | import java.util.List;
16 |
17 | /**
18 |  * Service class for handling appointment pre-bookings.
19 |  */
20 | public class BookingService {
21 |
22 |     // Predefined service time slots (30-minute intervals from 09:00 to 16:30)
23 |     public static final List<String> DEFAULT_SLOTS = Arrays.asList(
24 |         "09:00:00", "09:30:00", "10:00:00", "10:30:00",
25 |         "11:00:00", "11:30:00", "12:00:00", "12:30:00",
26 |         "13:30:00", "14:00:00", "14:30:00", "15:00:00",
27 |         "15:30:00", "16:00:00", "16:30:00"
28 |     );
29 |
30 |     /**
31 |      * Checks if a particular slot is available for booking.
32 |      * Prevents booking if slot is already occupied or past.
33 |      *
34 |      * @param serviceId ID of the service
35 |      * @param bookingDate Date of the booking
36 |      * @param bookingTime Time of the booking
37 |      * @return true if available, false otherwise
38 |      */
39 |     public boolean checkSlotAvailability(int serviceId, Date bookingDate, Time bookingTime) {
40 |         // Rule: Prevent booking past time slots
41 |         if (isPastSlot(bookingDate, bookingTime)) {
42 |             return false;
43 |         }
44 |
45 |         // Rule: Prevent booking full slots (1 booking per service per slot)
46 |         String countSql =
47 |             "SELECT COUNT(*) AS slot_count FROM bookings " +
48 |             "WHERE service_id = ? AND booking_date = ? AND booking_time = ? " +
49 |             "AND status = 'BOOKED'";
50 |
51 |         try (Connection connection = DBConnection.getConnection();
52 |             PreparedStatement statement = connection.prepareStatement(countSql)) {
53 |
54 |             statement.setInt(1, serviceId);
55 |             statement.setDate(2, bookingDate);
56 |             statement.setTime(3, bookingTime);
57 |
58 |             try (ResultSet resultSet = statement.executeQuery()) {
59 |                 if (resultSet.next()) {
60 |                     int slotCount = resultSet.getInt("slot_count");
61 |                     return slotCount == 0;
62 |                 }
63 |             }
64 |         } catch (Exception e) {
65 |             e.printStackTrace();
66 |         }
67 |
68 |         return false;
69 |     }

```

```

70 |
71 | /**
72 |  * Gets all available time slots for a service on a given date.
73 |  */
74 | public List<String> getAvailableSlots(int serviceId, Date bookingDate) {
75 |     List<String> available = new ArrayList<>();
76 |     List<String> bookedSlots = new ArrayList<>();
77 |
78 |     String bookedSql =
79 |         "SELECT booking_time FROM bookings " +
80 |         "WHERE service_id = ? AND booking_date = ? AND status = 'BOOKED'";
81 |
82 |     try (Connection connection = DBConnection.getConnection();
83 |         PreparedStatement statement = connection.prepareStatement(bookedSql)) {
84 |
85 |         statement.setInt(1, serviceId);
86 |         statement.setDate(2, bookingDate);
87 |
88 |         try (ResultSet resultSet = statement.executeQuery()) {
89 |
90 |             while (resultSet.next()) {
91 |                 bookedSlots.add(resultSet.getTime("booking_time").toString());
92 |             }
93 |         } catch (Exception e) {
94 |             e.printStackTrace();
95 |         }
96 |
97 |         for (String slot : DEFAULT_SLOTS) {
98 |             Time time = Time.valueOf(slot);
99 |             if (!isPastSlot(bookingDate, time) && !bookedSlots.contains(slot)) {
100 |                 available.add(slot);
101 |             }
102 |         }
103 |
104 |         return available;
105 |     }
106 |
107 | /**
108 |  * Creates a new booking with validation rules:
109 |  * 1. Prevent past time slots
110 |  * 2. Prevent duplicate active bookings for the student on the same date & time
111 |  * 3. Prevent booking full slots
112 |  *
113 |  * @param booking Booking object containing studentId, serviceId, bookingDate, bookingTime
114 |  * @return Result message indicating success or failure reason
115 |  */
116 | public String createBooking(Booking booking) {
117 |     if (booking == null || booking.getStudentId() <= 0 || booking.getServiceId() <= 0) return
118 | "Invalid booking request.";
119 |     Date date=booking.getBookingDate(); Time time=booking.getBookingTime();
120 |     if (isPastSlot(date,time)) return "Cannot book a past date or time slot.";
121 |     if (!DEFAULT_SLOTS.contains(time.toString())) return "Invalid time slot.";
122 |     try (Connection connection=DBConnection.getConnection()) {
123 |         connection.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
124 |         connection.setAutoCommit(false);
125 |         try {
126 |             // Always student then service: serializes cross-service duplicates and service
127 |             capacity.
128 |             if (!com.queue.util.TransactionLocks.student(connection,booking.getStudentId()) ||
129 |                 !com.queue.util.TransactionLocks.service(connection,booking.getServiceId())) {
130 |                 connection.rollback();return "Invalid student or service.";
131 |             }
132 |             try (PreparedStatement check=connection.prepareStatement("SELECT booking_id FROM
133 | bookings WHERE student_id=? AND booking_date=? AND booking_time=? AND status='BOOKED' LIMIT 1")) {
134 |                 check.setInt(1,booking.getStudentId());check.setDate(2,date);check.setTime(3,time);
135 |                 try(ResultSet rows=check.executeQuery()) { if(rows.next()) {
136 |                     connection.rollback();return "You already have an active booking at this date and time."; } }
137 |                 try (PreparedStatement check=connection.prepareStatement("SELECT booking_id FROM
138 | bookings WHERE service_id=? AND booking_date=? AND booking_time=? AND status='BOOKED' LIMIT 1")) {

```

```

135 |         check.setInt(1, booking.getServiceId());check.setDate(2, date);check.setTime(3, time);
136 |         try(ResultSet rows=check.executeQuery()) { if(rows.next()) {
| connection.rollback();return "Selected time slot is already fully booked."; } }
137 |     }
138 |     if (isPastSlot(date,time)) { connection.rollback();return "Cannot book a past date or
| time slot."; }
139 |     try(PreparedStatement insert=connection.prepareStatement("INSERT INTO
| bookings(student_id,service_id,booking_date,booking_time,status) VALUES
| (?,?,?, 'BOOKED')", java.sql.Statement.RETURN_GENERATED_KEYS)) {
140 |         insert.setInt(1,booking.getStudentId());insert.setInt(2,booking.getServiceId());ins
| ert.setDate(3,date);insert.setTime(4,time);insert.executeUpdate();
141 |         try(ResultSet keys=insert.getGeneratedKeys()) {
| keys.next();booking.setBookingId(keys.getInt(1)); }
142 |     }
143 |     connection.commit();booking.setStatus("BOOKED");return "SUCCESS";
144 |     } catch(Exception error) { connection.rollback();throw error; }
145 |     } catch(Exception error) { throw new IllegalStateException("Unable to create booking.",error);
| }
146 | }
147 | /**
148 |  * Cancels an existing booking if its status is 'BOOKED'.
149 |  *
150 |  * @param bookingId ID of the booking
151 |  * @param studentId ID of the student owning the booking (0 to skip student check)
152 |  * @return true if successfully cancelled, false otherwise
153 |  */
154 | public boolean cancelBooking(int bookingId, int studentId) {
155 |     String updateSql;
156 |     if (studentId > 0) {
157 |         updateSql = "UPDATE bookings SET status = 'CANCELLED' WHERE booking_id = ? AND student_id =
| ? AND status = 'BOOKED'";
158 |     } else {
159 |         updateSql = "UPDATE bookings SET status = 'CANCELLED' WHERE booking_id = ? AND status =
| 'BOOKED'";
160 |     }
161 |
162 |     try (Connection connection = DBConnection.getConnection();
163 |
164 |         PreparedStatement statement = connection.prepareStatement(updateSql)) {
165 |
166 |         statement.setInt(1, bookingId);
167 |         if (studentId > 0) {
168 |             statement.setInt(2, studentId);
169 |         }
170 |
171 |         int rows = statement.executeUpdate();
172 |         return rows > 0;
173 |     } catch (Exception e) {
174 |         e.printStackTrace();
175 |     }
176 |     return false;
177 | }
178 | /**
179 |  * Cancels an existing booking by booking ID.
180 |  */
181 | public boolean cancelBooking(int bookingId) {
182 |     return cancelBooking(bookingId, 0);
183 | }
184 |
185 | /**
186 |  * Retrieves all bookings for a given student.
187 |  *
188 |  * @param studentId ID of the student
189 |  * @return List of Booking objects
190 |  */
191 | public List<Booking> getStudentBookings(int studentId) {
192 |     List<Booking> bookings = new ArrayList<>();
193 |     String sql = "SELECT * FROM bookings WHERE student_id = ? ORDER BY booking_date DESC,
| booking_time DESC";
194 |
195 |     try (Connection connection = DBConnection.getConnection();

```

```

196 |         PreparedStatement statement = connection.prepareStatement(sql)) {
197 |
198 |         statement.setInt(1, studentId);
199 |         try (ResultSet resultSet = statement.executeQuery()) {
200 |             while (resultSet.next()) {
201 |                 Booking b = new Booking();
202 |                 b.setBookingId(resultSet.getInt("booking_id"));
203 |                 b.setStudentId(resultSet.getInt("student_id"));
204 |                 b.setServiceId(resultSet.getInt("service_id"));
205 |                 b.setBookingDate(resultSet.getDate("booking_date"));
206 |
207 |                 b.setBookingTime(resultSet.getTime("booking_time"));
208 |                 b.setStatus(resultSet.getString("status"));
209 |                 b.setCreatedAt(resultSet.getTimestamp("created_at"));
210 |                 bookings.add(b);
211 |             }
212 |         } catch (Exception e) {
213 |             e.printStackTrace();
214 |         }
215 |
216 |         return bookings;
217 |     }
218 |
219 |     /**
220 |     * Retrieves all upcoming active bookings for a specific service (status = 'BOOKED' for today and
221 |     * future).
222 |     *
223 |     * @param serviceId ID of the service
224 |     * @return List of upcoming Booking objects ordered by date and time
225 |     */
226 |     public List<Booking> getUpcomingBookings(int serviceId) {
227 |         List<Booking> bookings = new ArrayList<>();
228 |         String sql =
229 |             "SELECT * FROM bookings " +
230 |             "WHERE service_id = ? AND status = 'BOOKED' " +
231 |             "AND (booking_date > CURRENT_DATE " +
232 |             "      OR (booking_date = CURRENT_DATE AND booking_time >= CURRENT_TIME)) " +
233 |             "ORDER BY booking_date ASC, booking_time ASC";
234 |
235 |         try (Connection connection = DBConnection.getConnection();
236 |             PreparedStatement statement = connection.prepareStatement(sql)) {
237 |
238 |             statement.setInt(1, serviceId);
239 |             try (ResultSet resultSet = statement.executeQuery()) {
240 |                 while (resultSet.next()) {
241 |                     Booking b = new Booking();
242 |                     b.setBookingId(resultSet.getInt("booking_id"));
243 |                     b.setStudentId(resultSet.getInt("student_id"));
244 |                     b.setServiceId(resultSet.getInt("service_id"));
245 |                     b.setBookingDate(resultSet.getDate("booking_date"));
246 |                     b.setBookingTime(resultSet.getTime("booking_time"));
247 |                     b.setStatus(resultSet.getString("status"));
248 |                     b.setCreatedAt(resultSet.getTimestamp("created_at"));
249 |                     bookings.add(b);
250 |                 }
251 |             } catch (Exception e) {
252 |                 e.printStackTrace();
253 |             }
254 |
255 |             return bookings;
256 |         }
257 |
258 |         /**
259 |         * Retrieves upcoming bookings for a specific service on a specific date.
260 |         *
261 |         * @param serviceId ID of the service
262 |         * @param bookingDate Specific date to filter
263 |         * @return List of Booking objects
264 |         */

```

```

265 |     public List<Booking> getUpcomingBookings(int serviceId, Date bookingDate) {
266 |         List<Booking> bookings = new ArrayList<>();
267 |         String sql =
268 |             "SELECT * FROM bookings " +
269 |             "WHERE service_id = ? AND booking_date = ? AND status = 'BOOKED' " +
270 |             "ORDER BY booking_time ASC";
271 |
272 |         try (Connection connection = DBConnection.getConnection();
273 |             PreparedStatement statement = connection.prepareStatement(sql)) {
274 |
275 |             statement.setInt(1, serviceId);
276 |             statement.setDate(2, bookingDate);
277 |             try (ResultSet resultSet = statement.executeQuery()) {
278 |                 while (resultSet.next()) {
279 |                     Booking b = new Booking();
280 |                     b.setBookingId(resultSet.getInt("booking_id"));
281 |                     b.setStudentId(resultSet.getInt("student_id"));
282 |                     b.setServiceId(resultSet.getInt("service_id"));
283 |                     b.setBookingDate(resultSet.getDate("booking_date"));
284 |                     b.setBookingTime(resultSet.getTime("booking_time"));
285 |                     b.setStatus(resultSet.getString("status"));
286 |                     b.setCreatedAt(resultSet.getTimestamp("created_at"));
287 |                     bookings.add(b);
288 |                 }
289 |             }
290 |         } catch (Exception e) {
291 |             e.printStackTrace();
292 |         }
293 |
294 |         return bookings;
295 |     }
296 |
297 |     /**
298 |     * Retrieves upcoming bookings with student and service details.
299 |     * Uses SQL JOIN to avoid N+1 queries.
300 |     *
301 |     * @param serviceId ID of the service
302 |     * @return List of maps containing booking, student, and service details
303 |     */
304 |     public List<java.util.Map<String, Object>> getUpcomingBookingsWithDetails(int serviceId) {
305 |         List<java.util.Map<String, Object>> list = new ArrayList<>();
306 |
307 |         String sql =
308 |             "SELECT b.booking_id, b.student_id, b.service_id, b.booking_date, b.booking_time, " +
309 |             "        b.status, b.created_at, st.student_name, st.email, st.phone, sv.service_name " +
310 |             "FROM bookings b " +
311 |             "JOIN students st ON b.student_id = st.student_id " +
312 |             "JOIN services sv ON b.service_id = sv.service_id " +
313 |             "WHERE b.service_id = ? AND b.status = 'BOOKED' " +
314 |             "AND (b.booking_date > CURRENT_DATE " +
315 |             "      OR (b.booking_date = CURRENT_DATE AND b.booking_time >= CURRENT_TIME)) " +
316 |             "ORDER BY b.booking_date ASC, b.booking_time ASC";
317 |
318 |         try (Connection connection = DBConnection.getConnection();
319 |             PreparedStatement statement = connection.prepareStatement(sql)) {
320 |
321 |             statement.setInt(1, serviceId);
322 |
323 |             try (ResultSet rs = statement.executeQuery()) {
324 |                 while (rs.next()) {
325 |                     java.util.Map<String, Object> map = new java.util.HashMap<>();
326 |
327 |                     map.put("bookingId", rs.getInt("booking_id"));
328 |                     map.put("studentId", rs.getInt("student_id"));
329 |                     map.put("studentName", rs.getString("student_name"));
330 |                     map.put("email", rs.getString("email"));
331 |                     map.put("phone", rs.getString("phone"));
332 |                     map.put("serviceId", rs.getInt("service_id"));
333 |                     map.put("serviceName", rs.getString("service_name"));
334 |                     map.put("bookingDate", rs.getDate("booking_date"));
335 |                     map.put("bookingTime", rs.getTime("booking_time"));

```

```

336 |         map.put("status", rs.getString("status"));
337 |         map.put("createdAt", rs.getTimestamp("created_at"));
338 |
339 |         list.add(map);
340 |     }
341 | }
342 |
343 | } catch (Exception e) {
344 |     e.printStackTrace();
345 | }
346 |
347 | return list;
348 | }
349 |
350 | /**
351 |  * Retrieves a single booking by booking ID.
352 |  */
353 | public Booking getBookingById(int bookingId) {
354 |     String sql = "SELECT * FROM bookings WHERE booking_id = ?";
355 |
356 |     try (Connection connection = DBConnection.getConnection();
357 |          PreparedStatement statement = connection.prepareStatement(sql)) {
358 |
359 |         statement.setInt(1, bookingId);
360 |         try (ResultSet resultSet = statement.executeQuery()) {
361 |             if (resultSet.next()) {
362 |                 Booking b = new Booking();
363 |                 b.setBookingId(resultSet.getInt("booking_id"));
364 |                 b.setStudentId(resultSet.getInt("student_id"));
365 |                 b.setServiceId(resultSet.getInt("service_id"));
366 |                 b.setBookingDate(resultSet.getDate("booking_date"));
367 |                 b.setBookingTime(resultSet.getTime("booking_time"));
368 |                 b.setStatus(resultSet.getString("status"));
369 |                 b.setCreatedAt(resultSet.getTimestamp("created_at"));
370 |                 return b;
371 |             }
372 |         }
373 |     } catch (Exception e) {
374 |         e.printStackTrace();
375 |     }
376 |
377 |     return null;
378 | }
379 |
380 | /**
381 |  * Checks if a student already has an active booking at the same date and time.
382 |  */
383 | public boolean hasDuplicateBooking(int studentId, Date bookingDate, Time bookingTime) {
384 |     String checkSql =
385 |         "SELECT COUNT(*) AS cnt FROM bookings " +
386 |         "WHERE student_id = ? AND booking_date = ? AND booking_time = ? AND status = 'BOOKED'";
387 |
388 |     try (Connection connection = DBConnection.getConnection();
389 |          PreparedStatement statement = connection.prepareStatement(checkSql)) {
390 |
391 |         statement.setInt(1, studentId);
392 |         statement.setDate(2, bookingDate);
393 |         statement.setTime(3, bookingTime);
394 |
395 |         try (ResultSet resultSet = statement.executeQuery()) {
396 |             if (resultSet.next()) {
397 |                 return resultSet.getInt("cnt") > 0;
398 |             }
399 |         }
400 |     } catch (Exception e) {
401 |         e.printStackTrace();
402 |     }
403 |
404 |     return false;
405 | }

```

```
406 |  
407 | /**  
408 |  * Checks if a date and time slot is in the past.  
409 |  */  
410 | private boolean isPastSlot(Date bookingDate, Time bookingTime) {  
411 |     if (bookingDate == null || bookingTime == null) {  
412 |         return true;  
413 |     }  
414 |  
415 |     LocalDate slotDate = bookingDate.toLocalDate();  
416 |     LocalDate today = LocalDate.now();  
417 |  
418 |     if (slotDate.isBefore(today)) {  
419 |         return true;  
420 |     } else if (slotDate.isEqual(today)) {  
421 |         LocalTime slotTime = bookingTime.toLocalTime();  
422 |         LocalTime now = LocalTime.now();  
423 |         return slotTime.isBefore(now);  
424 |     }  
  
425 |  
426 |     return false;  
427 | }  
428 | }
```

src/main/java/com/queue/service/ClosingTimeListener.java

Servlet lifecycle background reconciliation. A scheduled worker checks closing every ten seconds and is stopped on context destruction.

Read these lines first

Line 8: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 16: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | package com.queue.service;
2 |
3 | import jakarta.servlet.ServletContextEvent;
4 | import jakarta.servlet.ServletContextListener;
5 | import jakarta.servlet.annotation.WebListener;
6 | import java.util.concurrent.*;
7 |
8 | /** Runs without a student or staff needing to keep a page open; catches up after restart. */
9 | @WebListener
10 | public class ClosingTimeListener implements ServletContextListener {
11 |     private ScheduledExecutorService scheduler;
12 |     public void contextInitialized(ServletContextEvent event) {
13 |         scheduler=Executors.newSingleThreadScheduledExecutor(r->{Thread t=new Thread(r,"queue-closing-
14 | time");t.setDaemon(true);return t;});
15 |         scheduler.scheduleWithFixedDelay(()->{
16 |             try{new ReservationService().closeAllDue();}
17 |             catch(Exception error){event.getServletContext().log("Queue closing reconciliation failed;
18 | will retry.",error);}
19 |             },0,10,TimeUnit.SECONDS);
20 |     }
21 |     public void contextDestroyed(ServletContextEvent
22 | event){if(scheduler!=null)scheduler.shutdownNow();}
23 | }

```

src/main/java/com/queue/service/NoShowService.java

a

Retained older appointment no-show handling. Current queue absence rules are in ReservationService.

Read these lines first

Line 8: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 102: try-with-resources closes database/IO resources automatically when the block ends.

Line 103: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 105: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 108: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Line 115: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 117: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 118: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Numbered source

```

1 | package com.queue.service;
2 |
3 | import com.queue.model.Booking;
4 | import com.queue.util.DBConnection;
5 |
6 | import java.sql.Connection;
7 | import java.sql.Date;
8 | import java.sql.PreparedStatement;
9 | import java.sql.ResultSet;
10 | import java.sql.Time;
11 | import java.time.LocalDate;
12 | import java.time.LocalTime;
13 | import java.util.ArrayList;
14 | import java.util.List;
15 |
16 | /**
17 |  * Service class for handling no-show detection and slot release based on a configurable grace period.
18 |  */
19 | public class NoShowService {
20 |
21 |     public static final String STATUS_NO_SHOW = "NO_SHOW";
22 |     public static final int DEFAULT_GRACE_PERIOD_MINUTES = 5;
23 |
24 |     private int gracePeriodMinutes;
25 |
26 |     public NoShowService() {
27 |         this.gracePeriodMinutes = DEFAULT_GRACE_PERIOD_MINUTES;
28 |     }
29 |
30 |     public NoShowService(int gracePeriodMinutes) {
31 |         this.gracePeriodMinutes = gracePeriodMinutes > 0 ? gracePeriodMinutes :
32 |         DEFAULT_GRACE_PERIOD_MINUTES;
33 |     }
34 |
35 |     public int getGracePeriodMinutes() {
36 |         return gracePeriodMinutes;
37 |     }
38 |
39 |     public void setGracePeriodMinutes(int gracePeriodMinutes) {
40 |         this.gracePeriodMinutes = gracePeriodMinutes > 0 ? gracePeriodMinutes :
41 |         DEFAULT_GRACE_PERIOD_MINUTES;
42 |     }
43 |
44 |     /**
45 |      * Evaluates if a given booking has exceeded its appointment time plus the grace period.
46 |      * Example: Booking Time = 10:30, Grace Period = 5 mins, Current Time > 10:35 -> true
47 |      */
48 |     public boolean isNoShow(Booking booking, int graceMinutes) {
49 |         if (booking == null || !"BOOKED".equalsIgnoreCase(booking.getStatus())) {
50 |             return false;
51 |         }
52 |
53 |         Date bookingDate = booking.getBookingDate();
54 |         Time bookingTime = booking.getBookingTime();
55 |
56 |         if (bookingDate == null || bookingTime == null) {
57 |             return false;
58 |         }
59 |
60 |         LocalDate bDate = bookingDate.toLocalDate();
61 |         LocalDate today = LocalDate.now();
62 |
63 |         if (bDate.isBefore(today)) {
64 |             return true;
65 |         } else if (bDate.isEqual(today)) {
66 |             LocalTime bTime = bookingTime.toLocalTime();
67 |             LocalTime cutoffTime = bTime.plusMinutes(graceMinutes);
68 |             return LocalTime.now().isAfter(cutoffTime);
69 |         }

```

```

68 |
69 |         return false;
70 |     }
71 |
72 |     public boolean isNoShow(Booking booking) {
73 |         return isNoShow(booking, this.gracePeriodMinutes);
74 |     }
75 |
76 |     /**
77 |      * Scans all active 'BOOKED' records for today (or past dates) that exceeded the grace period,
78 |      * updates their status to 'NO_SHOW', and releases the slot.
79 |      *
80 |      * @return Number of bookings updated to NO_SHOW
81 |      */
82 |     public int checkAndUpdateNoShows() {
83 |         return checkAndUpdateNoShows(this.gracePeriodMinutes);
84 |     }
85 |
86 |     /**
87 |      * Scans and marks no-show bookings with a specified grace period.
88 |      */
89 |     public int checkAndUpdateNoShows(int graceMinutes) {
90 |         int updatedCount = 0;
91 |
92 |         // SQL finds bookings that are past date, or today and past (booking_time + graceMinutes)
93 |         String selectSql =
94 |             "SELECT booking_id FROM bookings " +
95 |             "WHERE status = 'BOOKED' " +
96 |             "AND (booking_date < CURRENT_DATE " +
97 |             "      OR (booking_date = CURRENT_DATE AND ADDTIME(booking_time, SEC_TO_TIME(? * 60)) <
| CURRENT_TIME))";
98 |
99 |         String updateSql =
100 |             "UPDATE bookings SET status = ? WHERE booking_id = ? AND status = 'BOOKED'";
101 |
102 |         try (Connection connection = DBConnection.getConnection();
103 |             PreparedStatement selectStatement = connection.prepareStatement(selectSql)) {
104 |
105 |             selectStatement.setInt(1, graceMinutes);
106 |             List<Integer> noShowIds = new ArrayList<>();
107 |
108 |             try (ResultSet rs = selectStatement.executeQuery()) {
109 |                 while (rs.next()) {
110 |                     noShowIds.add(rs.getInt("booking_id"));
111 |                 }
112 |             }
113 |
114 |             if (!noShowIds.isEmpty()) {
115 |                 try (PreparedStatement updateStatement = connection.prepareStatement(updateSql)) {
116 |                     for (int bookingId : noShowIds) {
117 |                         updateStatement.setString(1, STATUS_NO_SHOW);
118 |                         updateStatement.setInt(2, bookingId);
119 |                         updateStatement.addBatch();
120 |                     }
121 |                     int[] results = updateStatement.executeBatch();
122 |                     for (int res : results) {
123 |                         if (res > 0 || res == PreparedStatement.SUCCESS_NO_INFO) {
124 |                             updatedCount++;
125 |                         }
126 |                     }
127 |                 }
128 |             }
129 |
130 |         } catch (Exception e) {
131 |             e.printStackTrace();
132 |         }
133 |
134 |         return updatedCount;
135 |     }
136 |

```

```
137 |     /**
138 |      * Marks a specific booking as NO_SHOW by booking ID.
139 |      */
140 |     public boolean markAsNoShow(int bookingId) {
141 |         String updateSql = "UPDATE bookings SET status = ? WHERE booking_id = ? AND status = 'BOOKED'";
142 |
143 |         try (Connection connection = DBConnection.getConnection();
144 |             PreparedStatement statement = connection.prepareStatement(updateSql)) {
145 |
146 |             statement.setString(1, STATUS_NO_SHOW);
147 |             statement.setInt(2, bookingId);
148 |
149 |             int rows = statement.executeUpdate();
150 |             return rows > 0;
151 |         } catch (Exception e) {
152 |             e.printStackTrace();
153 |         }
154 |
155 |         return false;
156 |     }
157 | }
```

src/main/java/com/queue/service/ReservationService.java

Core transaction coordinator. Study preview, reserve, operate, callNext, cancellation, availability and closing rollover. Follow the service lock and commit/rollback boundary.

Read these lines first

Line 17: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 18: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 19: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Line 21: Builds the response consumed by another layer. Identify which fields the frontend renders.

Line 26: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 28: Builds the response consumed by another layer. Identify which fields the frontend renders.

Line 32: try-with-resources closes database/IO resources automatically when the block ends.

Line 72: try-with-resources closes database/IO resources automatically when the block ends.

Numbered source

```

1 | package com.queue.service;
2 |
3 | import com.queue.util.*;
4 | import com.queue.model.Queue;
5 | import java.sql.*;
6 | import java.time.*;
7 | import java.util.*;
8 |
9 | /** Service-day FIFO with stable tokens, transactional counter ownership and missed turns. */
10 | public class ReservationService {
11 |     private final LocalDateTime fixedNow;
12 |     public ReservationService(){fixedNow=null;}
13 |     public ReservationService(LocalDateTime at){fixedNow=at;}
14 |     private LocalDateTime now(){return fixedNow==null?ServiceHours.now():fixedNow;}
15 |     private static final String ACTIVE="('WAITING','CALLED','SERVING')";
16 |     private List<Map<String,Object>> rows(Connection c,String sql,Object...args)throws SQLException {
17 |         try(PreparedStatement p=c.prepareStatement(sql)) {
18 |             for(int i=0;i<args.length;i++)p.setObject(i+1,args[i]);
19 |             try(ResultSet r=p.executeQuery()) {
20 |                 List<Map<String,Object>> list=new ArrayList<>();ResultSetMetaData m=r.getMetaData();
21 |                 while(r.next()){Map<String,Object> row=new LinkedHashMap<>();for(int
22 | i=1;i<=m.getColumnCount();i++)row.put(m.getColumnLabel(i),r.getObject(i));list.add(row);}return list;
23 |             }
24 |         }
25 |     private int update(Connection c,String sql,Object...args)throws SQLException {
26 |         try(PreparedStatement p=c.prepareStatement(sql)){for(int
27 | i=0;i<args.length;i++)p.setObject(i+1,args[i]);return p.executeUpdate();}
28 |     private int number(Map<String,Object> row,String key){return
29 | row.get(key)==null?0:((Number)row.get(key)).intValue();}
30 |     private void event(Connection c,int queue,int staff,String type)throws SQLException
31 | {update(c,"INSERT INTO queue_events(queue_id,staff_id,event_type) VALUES
32 | (?, ?, ?)",queue,staff==0?null:staff,type);}
33 |     public Map<String,Object> preview(int student,int service)throws SQLException {
34 |         closeWaiting(service);
35 |         try(Connection c=DBConnection.getConnection()){return preview(c,student,service);}
36 |     }
37 |     private Map<String,Object> preview(Connection c,int student,int service)throws SQLException {
38 |         LocalDateTime at=now();LocalDate day=at.toLocalDate();
39 |         LocalDateTime requestedAt=at;
40 |         List<Map<String,Object>> future=rows(c,"SELECT r.visit_date FROM queue q JOIN
41 | queue_reservations r ON r.queue_id=q.queue_id WHERE q.student_id=? AND q.service_id=? AND
42 | q.status='WAITING' AND r.visit_date>? ORDER BY r.visit_date LIMIT 1",student,service,day);
43 |
44 |         boolean carried=!future.isEmpty();
45 |         if(carried){day=((java.sql.Date)future.get(0).get("visit_date")).toLocalDate();at=day.atTime(10,0);}
46 |         List<Map<String,Object>> config=rows(c,"SELECT service_name FROM services WHERE
47 | service_id=?",service);
48 |         if(config.isEmpty())throw new IllegalArgumentException("Unknown service.");
49 |         List<Map<String,Object>> active=rows(c,"SELECT q.queue_id,q.student_id,q.status,r.queue_order
50 | FROM queue q JOIN queue_reservations r ON r.queue_id=q.queue_id WHERE q.service_id=? AND r.visit_date=?
51 | AND q.status IN "+ACTIVE+" ORDER BY r.queue_order,q.queue_id",service,day);
52 |         int ahead=active.size();Integer existing=null;String status="WAITING";
53 |         active.sort(Comparator.comparingInt(row -> "WAITING".equals(row.get("status"))?1:0));
54 |         for(int i=0;i<active.size();i++)if(number(active.get(i),"student_id")==student){ahead=i;existin
55 | g=number(active.get(i),"queue_id");status=(String)active.get(i).get("status");break;}
56 |         int available=number(rows(c,"SELECT COUNT(*) n FROM counters WHERE service_id=? AND
57 | is_active=TRUE",service).get(0),"n");
58 |         int planned=number(rows(c,"SELECT COUNT(*) n FROM counters WHERE
59 | service_id=?",service).get(0),"n");
60 |         boolean open=ServiceHours.open(at), closed=ServiceHours.nextOpen(at)==null;
61 |         int capacity=open&&!carried?available:planned;
62 |         LocalDateTime projected=capacity>0?ServiceHours.project(at,(long)Math.ceil(ahead*new
63 | WaitingTimeService().getAverageServiceTime(service)/capacity)):null;
64 |         boolean
65 | closingRisk="WAITING".equals(status)&&projected!=null&&!projected.isBefore(day.atTime(15,0));
66 |         LocalDateTime estimate=closingRisk?null:projected;
67 |         if(Set.of("CALLED","SERVING").contains(status))estimate=at;

```

```

54 |         Map<String,Object> result=new LinkedHashMap<>();result.put("position",ahead+1);result.put("ahead",ahead);result.put("queueId",existing);result.put("serviceName",config.get(0).get("service_name"));
55 |         result.put("testClock",fixedNow==null && Math.abs(Duration.between(LocalDateTime.now(ServiceHours.ZONE),at).toSeconds())>2);result.put("campusTime",at.toLocalTime().withSecond(0).withNano(0).toString());result.put("date",day.toString());result.put("hours","10:00-13:00 / 14:00-15:00");result.put("open",open);result.put("closed",closed);result.put("activeCounters",available);
56 |
57 |         result.put("estimate",estimate==null?null:estimate.toLocalTime().withSecond(0).withNano(0).toString());
58 |         result.put("wait",estimate==null?null:Math.max(0,Duration.between(carried?requestedAt:at,estimate).toMinutes()));
59 |         result.put("rescheduled",carried);
60 |         result.put("closingRisk",closingRisk);result.put("projectedServiceTime",projected==null?null:projected.toLocalTime().withSecond(0).withNano(0).toString());
61 |         result.put("minutesToClose",Math.max(0,Duration.between(at,day.atTime(15,0)).toMinutes()));
62 |         result.put("warning",carried?"Your ticket has moved to "+day+" . Counters open at 10 AM; your queue order is preserved.":"closingRisk?"You're unlikely to be served today. Your pending ticket will move to the next day's 10 AM queue if unserved at closing.":null);
63 |         result.put("canReserve",!closed && existing==null && (estimate!=null || (open && available==0)));
64 |
65 |         result.put("message",existing!=null?"You already have a place in this queue.":"closed?"Reservations are closed for today.":"open&&available==0?"The counter has not started or is paused. Your place can be reserved; the service time is unavailable.":"estimate==null?"The queue is expected to reach closing time. No further places are available.":"open?"Live estimate; your confirmed position may change before you submit.":"Prebooking is open. Service starts at the next opening; estimates assume scheduled counters are staffed.");
66 |         if(closingRisk&&existing==null)result.put("message","You're unlikely to be served today. Your estimated turn falls at or after the 3 PM closing time. Please reserve again tomorrow.");
67 |         if(existing!=null) {
68 |             Map<String,Object> assignment=rows(c,"SELECT r.missed_turns,c.counter_name,r.counter_id FROM queue_reservations r LEFT JOIN counters c ON c.counter_id=r.counter_id WHERE r.queue_id=?",existing).get(0);
69 |             result.put("missedTurns",assignment.get("missed_turns"));result.put("counterId",assignment.get("counter_id"));result.put("counterName",assignment.get("counter_name"));
70 |         }
71 |         return result;
72 |     }
73 |     public Map<String,Object> reserve(int student,int service)throws SQLException {
74 |         try(Connection c=DBCConnection.getConnection()) {
75 |             c.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);c.setAutoCommit(false);
76 |             try {
77 |                 if(!TransactionLocks.service(c,service))throw new IllegalArgumentException("Unknown service.");
78 |                 Map<String,Object> preview=preview(c,student,service);
79 |                 if(!Boolean.TRUE.equals(preview.get("canReserve"))){throw new IllegalArgumentException((String)preview.get("message"));
80 |                 LocalDate day=now().toLocalDate();
81 |                 int token=number(rows(c,"SELECT COALESCE(MAX(token_number),0)+1 n FROM queue WHERE service_id=?",service).get(0),"n");
82 |                 int order=number(rows(c,"SELECT COALESCE(MAX(r.queue_order),0)+1 n FROM queue_reservations r JOIN queue q ON q.queue_id=r.queue_id WHERE q.service_id=? AND r.visit_date=?",service,day).get(0),"n");
83 |                 int id;try(PreparedStatement p=c.prepareStatement("INSERT INTO queue(student_id,service_id,token_number,status) VALUES (?,?,'WAITING')",Statement.RETURN_GENERATED_KEYS)){p.setInt(1,student);p.setInt(2,service);p.setInt(3,token);p.executeUpdate();try(ResultSet keys=p.getGeneratedKeys()){keys.next();id=keys.getInt(1);}}
84 |                 update(c,"INSERT INTO queue_reservations(queue_id,visit_date,queue_order) VALUES (?,?,'?'),id,day,order);event(c,id,0,"RESERVED");
85 |                 preview.put("queueId",id);preview.put("token",String.format("%03d",token));c.commit();return preview;
86 |             }catch(Exception error){c.rollback();throw error;}
87 |         }
88 |     }
89 |     public List<Map<String,Object>> visits(int student)throws SQLException {
90 |         closeWaitingForStudent(student);
91 |         try(Connection c=DBCConnection.getConnection()) {
92 |             return rows(c,"SELECT q.queue_id AS id,q.service_id AS serviceId,r.visit_date AS date,q.token_number AS token,q.status,r.missed_turns AS missedTurns FROM queue q JOIN queue_reservations r ON r.queue_id=q.queue_id WHERE q.student_id=? ORDER BY r.visit_date DESC,q.queue_id DESC",student);
93 |         }

```

```

92 |     }
93 |     public List<Map<String,Object>> staffQueue(int staff)throws SQLException {
94 |         try(Connection c=DBConnection.getConnection()){List<Map<String,Object>>
| assignment=rows(c,"SELECT service_id FROM staff WHERE
| staff_id=?",staff);if(!assignment.isEmpty())closeWaiting(number(assignment.get(0),"service_id"));}
95 |         try(Connection c=DBConnection.getConnection()) {
96 |             return rows(c,"SELECT q.queue_id AS queueId,q.student_id AS studentId,student.student_name
| AS studentName,student.email AS studentEmail,student.phone AS studentPhone,q.service_id AS
| serviceId,q.token_number AS tokenNumber,q.status,r.counter_id AS counterId,r.staff_id AS
| assignedStaffId,r.missed_turns AS missedTurns FROM queue q JOIN queue_reservations r ON
| r.queue_id=q.queue_id JOIN students student ON student.student_id=q.student_id JOIN staff s ON
| s.service_id=q.service_id WHERE s.staff_id=? AND r.visit_date=? AND q.status IN "+ACTIVE+" ORDER BY
| CASE WHEN q.status='WAITING' THEN 1 ELSE 0 END,r.queue_order,q.queue_id",staff,now().toLocalDate());
97 |         }
98 |     }
99 |     public Map<String,Object> operate(int staff,String action,int queue)throws SQLException {
100 |         try(Connection c=DBConnection.getConnection()) {
101 |             c.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);c.setAutoCommit(false);
102 |             try {
103 |                 List<Map<String,Object>> profiles=rows(c,"SELECT * FROM staff WHERE staff_id=?",staff);
104 |                 if(profiles.isEmpty())throw new IllegalArgumentException("Staff account not found.");
105 |                 int service=number(profiles.get(0),"service_id"),
| counter=number(profiles.get(0),"counter_id");
106 |                 if(!TransactionLocks.service(c,service))throw new IllegalArgumentException("Service not
| found.");
107 |                 List<Map<String,Object>> counters=rows(c,"SELECT * FROM counters WHERE counter_id=? AND
| service_id=? FOR UPDATE",counter,service);
108 |                 if(counters.isEmpty())throw new IllegalArgumentException("Your counter assignment is
| invalid.");
109 |                 if(action.equals("call")||action.equals("call-next")) {
110 |                     if(!ServiceHours.open(now()))throw new IllegalArgumentException("Service hours are
| 10 AM-1 PM and 2 PM-3 PM.");
111 |                     if(!Boolean.TRUE.equals(counters.get(0).get("is_active"))){throw new
| IllegalArgumentException("Make your counter available first.");
112 |                 }
113 |                 LocalDate day=now().toLocalDate();
114 |                 List<Map<String,Object>> current=rows(c,"SELECT q.queue_id FROM queue q JOIN
| queue_reservations r ON r.queue_id=q.queue_id WHERE r.counter_id=? AND r.visit_date=? AND q.status IN
| ('CALLED','SERVING')",counter,day);
115 |                 if(action.equals("call-next") || action.equals("call")) {
116 |                     if(!current.isEmpty())throw new IllegalArgumentException("Finish your current
| student before calling another.");
117 |                     Map<String,Object>
| called=callNext(c,service,counter,staff,day,action.equals("call"?queue:0);c.commit();return called;
118 |                 }
119 |                 List<Map<String,Object>> records=rows(c,"SELECT q.*,r.missed_turns,r.queue_order FROM
| queue q JOIN queue_reservations r ON r.queue_id=q.queue_id WHERE q.queue_id=? AND q.service_id=? AND
| r.counter_id=? AND r.staff_id=? AND r.visit_date=? FOR UPDATE",queue,service,counter,staff,day);
120 |                 if(records.isEmpty())throw new IllegalArgumentException("That ticket is not assigned to
| you today.");
121 |                 String status=(String)records.get(0).get("status");
122 |                 if(action.equals("no-show")) {
123 |                     if(!status.equals("CALLED"))throw new IllegalArgumentException("Only a called
| student can miss a turn.");
124 |                     int misses=number(records.get(0),"missed_turns")+1;
125 |                     if(misses>=2) {update(c,"UPDATE queue SET
| status='NO_SHOW',completed_at=CURRENT_TIMESTAMP WHERE queue_id=?",queue);update(c,"UPDATE
| queue_reservations SET missed_turns=? WHERE
| queue_id=?",misses,queue);event(c,queue,staff,"REMOVED_SECOND_MISS");}
126 |                 else {
127 |                     List<Map<String,Object>> waiting=rows(c,"SELECT q.queue_id FROM queue q JOIN
| queue_reservations r ON r.queue_id=q.queue_id WHERE q.service_id=? AND r.visit_date=? AND
| q.status='WAITING' ORDER BY r.queue_order,q.queue_id",service,day);
128 |                     int
| insertion=Math.min(10,waiting.size());waiting.add(insertion,Map.of("queue_id",queue));
129 |                     // Re-index all waiting entries; called/serving entries are ahead
| independently.
130 |                     int order=1;for(Map<String,Object> row:waiting)update(c,"UPDATE
| queue_reservations SET queue_order=? WHERE queue_id=?",order++,number(row,"queue_id"));
131 |                     update(c,"UPDATE queue SET status='WAITING',called_at=NULL WHERE
| queue_id=?",queue);

```

```

132 |         update(c,"UPDATE queue_reservations SET
| missed_turns=?,counter_id=NULL,staff_id=NULL WHERE
| queue_id=?",misses,queue);event(c,queue,staff,"MOVED_BACK_FIRST_MISS");
133 |     }
134 |     // Do not immediately recall the absent student if nobody else is waiting.
135 |     Map<String,Object> next=ServiceHours.open(now())&&sameOperatingWindow(c,queue)&&Boo
| lean.TRUE.equals(counters.get(0).get("is_active"))?callNext(c,service,counter,staff,day,0,queue):null;
136 |     Map<String,Object> result=new LinkedHashMap<>();result.put("missedTurns",misses);re
| sult.put("removed",misses>=2);result.put("next",next);c.commit();return result;
137 | }

138 |     String target=action.equals("start")&&status.equals("CALLED")?"SERVING":action.equals("
| complete")&&status.equals("SERVING")?"COMPLETED":action.equals("skip")&&Set.of("CALLED","SERVING").cont
| ains(status)?"SKIPPED":null;
139 |     if(target==null)throw new IllegalArgumentException("The ticket state changed. Refresh
| and try again.");
140 |     if(target.equals("SERVING"))update(c,"UPDATE queue SET
| processing_clock_offset_seconds=? WHERE
| queue_id=?",Duration.between(LocalDateTime.now(ServiceHours.ZONE),now()).toSeconds(),queue);
141 |     if(target.equals("COMPLETED")) {
142 |         Timestamp start=(Timestamp)records.get(0).get("started_at");
143 |         Object offset=records.get(0).get("processing_clock_offset_seconds");
144 |         LocalDateTime realEnd=LocalDateTime.now(ServiceHours.ZONE);
145 |         double minutes=start==null?0:ServiceHours.processingMinutes(start.toLocalDateTime())
| .plusSeconds(offset==null?0:((Number)offset).longValue()),realEnd.plusSeconds(offset==null?0:((Number)o
| fffset).longValue());
146 |         update(c,"UPDATE queue SET processing_seconds=? WHERE
| queue_id=?",minutes*60,queue);
147 |     }
148 |     update(c,"UPDATE queue SET
| status=?,"+(target.equals("SERVING")?"started_at":"completed_at")+ "=CURRENT_TIMESTAMP WHERE
| queue_id=?",target,queue);event(c,queue,staff,target);
149 |     Map<String,Object> result=new
| LinkedHashMap<>();result.put("queueId",queue);result.put("status",target);
150 |     if(target.equals("COMPLETED") && ServiceHours.open(now()) &&
| Boolean.TRUE.equals(counters.get(0).get("is_active"))) {
151 |         // A service that crosses lunch must be followed by an explicit afternoon summon.
152 |         if(sameOperatingWindow(c,queue))
153 |             result.put("next",callNext(c,service,counter,staff,day,0));
154 |     }
155 |     c.commit();return result;
156 | }catch(Exception error){c.rollback();throw error;}
157 | }
158 | }
159 | private boolean sameOperatingWindow(Connection c,int queue)throws SQLException {
160 |     String
| expected=now().toLocalTime().isBefore(LocalTime.of(13,0))?"SUMMON_MORNING":"SUMMON_AFTERNOON";
161 |     List<Map<String,Object>> markers=rows(c,"SELECT event_type FROM queue_events WHERE queue_id=?
| AND event_type IN ('SUMMON_MORNING','SUMMON_AFTERNOON') ORDER BY event_id DESC LIMIT 1",queue);
162 |     return !markers.isEmpty() && expected.equals(markers.get(0).get("event_type"));
163 | }
164 | private Map<String,Object> callNext(Connection c,int service,int counter,int staff,LocalDate
| day,int requested)throws SQLException{return callNext(c,service,counter,staff,day,request,0);}
165 | private Map<String,Object> callNext(Connection c,int service,int counter,int staff,LocalDate
| day,int requested,int exclude)throws SQLException {

166 |     List<Map<String,Object>> waiting=rows(c,"SELECT q.queue_id AS queueId,q.student_id AS
| studentId,q.service_id AS serviceId,q.token_number AS tokenNumber FROM queue q JOIN queue_reservations
| r ON r.queue_id=q.queue_id WHERE q.service_id=? AND r.visit_date=? AND q.status='WAITING' AND
| q.queue_id<>? ORDER BY r.queue_order,q.queue_id LIMIT 1 FOR UPDATE",service,day,exclude);
167 |     if(waiting.isEmpty()){if(requested>0)throw new IllegalArgumentException("No waiting
| student.");return null;}
168 |     Map<String,Object> next=waiting.get(0);int id=number(next,"queueId");
169 |     if(requested>0&&requested!=id)throw new IllegalArgumentException("Call the first waiting
| student to preserve booking order.");
170 |     update(c,"UPDATE queue SET status='CALLED',called_at=CURRENT_TIMESTAMP WHERE queue_id=?",id);
171 |     update(c,"UPDATE queue_reservations SET counter_id=?,staff_id=? WHERE queue_id=?",counter,staff
| ,id);event(c,id,staff,"CALLED");event(c,id,staff,now().toLocalTime().isBefore(LocalTime.of(13,0))?"SUMM
| ON_MORNING":"SUMMON_AFTERNOON");next.put("counterId",counter);next.put("status","CALLED");return next;
172 | }
173 | public boolean cancel(int student,int queue)throws SQLException {
174 |     try(Connection c=DBConnection.getConnection()) {return update(c,"UPDATE queue q JOIN

```

```

    | queue_reservations r ON r.queue_id=q.queue_id SET q.status='CANCELLED',q.completed_at=CURRENT_TIMESTAMP
    | WHERE q.queue_id=? AND q.student_id=? AND q.status='WAITING' AND
    | r.visit_date=?,queue,student,now().toLocalDate()==1;}
175 |     }
176 |     public boolean availability(int staff,String status)throws SQLException {
177 |         if(!Set.of("AVAILABLE","PAUSED","CHECKED_OUT").contains(status))throw new
    | IllegalArgumentException("Invalid counter status.");
178 |         try(Connection c=DBConnection.getConnection()) {
179 |             c.setAutoCommit(false);
180 |             try {
181 |                 List<Map<String,Object>> profile=rows(c,"SELECT service_id,counter_id FROM staff WHERE
    | staff_id=?",staff);
182 |                 if(profile.isEmpty())throw new IllegalArgumentException("Staff not found.");
183 |                 int
    | service=number(profile.get(0),"service_id"),counter=number(profile.get(0),"counter_id");
184 |                 TransactionLocks.service(c,service);
185 |                 if(status.equals("CHECKED_OUT")&&!rows(c,"SELECT q.queue_id FROM queue q JOIN
    | queue_reservations r ON r.queue_id=q.queue_id WHERE r.counter_id=? AND r.visit_date=? AND q.status IN
    | ('CALLED','SERVING')",counter,now().toLocalDate()).isEmpty())throw new IllegalArgumentException("Finish
    | the current student before checking out.");
186 |                 update(c,"UPDATE counters SET is_active=? WHERE counter_id=? AND
    | service_id=?",status.equals("AVAILABLE"),counter,service);
187 |                 update(c,"INSERT INTO counter_states(counter_id,status) VALUES (?,?) ON DUPLICATE KEY
    | UPDATE status=VALUES(status)",counter,status);c.commit();return true;
188 |             }catch(Exception error){c.rollback();throw error;}
189 |         }
190 |     }
191 |     public String availability(int counter)throws SQLException {

192 |         try(Connection c=DBConnection.getConnection()) {
193 |             List<Map<String,Object>> result=rows(c,"SELECT
    | COALESCE(s.status,IF(c.is_active,'AVAILABLE','CHECKED_OUT')) status FROM counters c LEFT JOIN
    | counter_states s ON s.counter_id=c.counter_id WHERE c.counter_id=?",counter);
194 |             return result.isEmpty()?"CHECKED_OUT":(String)result.get(0).get("status");
195 |         }
196 |     }
197 |     /** No absence penalty: only WAITING tickets expire, under the same service lock as calls. */
198 |     public int closeWaiting(int service)throws SQLException {
199 |         LocalDateTime at=now();LocalDate day=at.toLocalDate();boolean
    | closed=!at.toLocalTime().isBefore(LocalTime.of(15,0));
200 |         try(Connection c=DBConnection.getConnection()) {
201 |             c.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);c.setAutoCommit(false);
202 |             try {
203 |                 if(!TransactionLocks.service(c,service)){c.rollback();return 0;}
204 |                 List<Map<String,Object>> due=rows(c,"SELECT q.queue_id FROM queue q JOIN
    | queue_reservations r ON r.queue_id=q.queue_id WHERE q.service_id=? AND q.status='WAITING' AND
    | (r.visit_date<? OR (r.visit_date=? AND ?)) ORDER BY r.visit_date,r.queue_order,q.queue_id FOR
    | UPDATE",service,day,day,closed);
205 |                 LocalDate destination=closed?day.plusDays(1):day;
206 |                 int order=0;
207 |                 // Carryovers take priority over new reservations already made for the destination.
208 |                 if(!due.isEmpty())update(c,"UPDATE queue_reservations r JOIN queue q ON
    | q.queue_id=r.queue_id SET r.queue_order=r.queue_order+? WHERE q.service_id=? AND
    | r.visit_date=?",due.size(),service,destination);
209 |                 for(Map<String,Object> row:due){int id=number(row,"queue_id");update(c,"UPDATE
    | queue_reservations SET visit_date=?,queue_order=?,counter_id=NULL,staff_id=NULL WHERE
    | queue_id=?",destination,++order,id);event(c,id,0,"ROLLED_TO_NEXT_DAY");}
210 |                 c.commit();return due.size();
211 |             }catch(Exception error){c.rollback();throw error;}
212 |         }
213 |     }
214 |     private void closeWaitingForStudent(int student)throws SQLException {
215 |         try(Connection c=DBConnection.getConnection()){for(Map<String,Object> row:rows(c,"SELECT
    | DISTINCT service_id FROM queue WHERE student_id=? AND
    | status='WAITING'",student))closeWaiting(number(row,"service_id"));}
216 |     }
217 |     public void closeAllDue()throws SQLException {
218 |         try(Connection c=DBConnection.getConnection()){for(Map<String,Object> row:rows(c,"SELECT
    | DISTINCT service_id FROM queue WHERE status='WAITING'"))closeWaiting(number(row,"service_id"));}
219 |     }
220 | }

```

src/main/java/com/queue/service/ServiceHours.java

Campus-time policy and processing-window arithmetic. Study real/test time, open boundaries, next opening, lunch projection and interval overlap.

Read these lines first

Line 16: try-with-resources closes database/IO resources automatically when the block ends.

Line 20: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | package com.queue.service;
2 |
3 | import java.time.*;
4 |
5 | /** Campus schedule in Asia/Kolkata. End boundaries are exclusive. */
6 | public final class ServiceHours {
7 |     public static final ZoneId ZONE=ZoneId.of("Asia/Kolkata");
8 |     /** Optional ignored local clock file beside the DB config; never bundled in the WAR. */
9 |     public static LocalDateTime now() {
10 |         LocalDateTime real=LocalDateTime.now(ZONE);
11 |         String config=System.getProperty("queue.config", System.getenv("QUEUE_DB_CONFIG"));
12 |         if(config==null)return real;
13 |         java.nio.file.Path file=java.nio.file.Path.of(config).toAbsolutePath().resolveSibling("test-
| clock.local.properties");
14 |         if(!java.nio.file.Files.isRegularFile(file))return real;
15 |         java.util.Properties properties=new java.util.Properties();
16 |         try(java.io.Reader reader=java.nio.file.Files.newBufferedReader(file)) {
17 |             properties.load(reader);
18 |             if(!"true".equalsIgnoreCase(properties.getProperty("enabled")))return real;
19 |             return real.toLocalDate().atTime(LocalTime.parse(properties.getProperty("time")));
20 |         }catch(Exception error){throw new IllegalStateException("Invalid local test clock
| configuration.",error);}
21 |     }
22 |     public static boolean open(LocalDateTime at) {
23 |         LocalTime t=at.toLocalTime();
24 |         return (!t.isBefore(LocalTime.of(10,0)) && t.isBefore(LocalTime.of(13,0))) ||
25 |             (!t.isBefore(LocalTime.of(14,0)) && t.isBefore(LocalTime.of(15,0)));
26 |     }
27 |     public static LocalDateTime nextOpen(LocalDateTime at) {
28 |         LocalTime t=at.toLocalTime();
29 |         if(t.isBefore(LocalTime.of(10,0)))return at.toLocalDate().atTime(10,0);
30 |         if(!t.isBefore(LocalTime.of(15,0)))return null;
31 |         if(!t.isBefore(LocalTime.of(13,0)) && t.isBefore(LocalTime.of(14,0)))return
| at.toLocalDate().atTime(14,0);
32 |         return at;
33 |     }
34 |     public static LocalDateTime estimate(LocalDateTime at,long serviceMinutes) {
35 |         LocalDateTime predicted=project(at,serviceMinutes);
36 |         return predicted!=null && predicted.isBefore(at.toLocalDate().atTime(15,0))?predicted:null;
37 |     }
38 |     /** Preserve the projection beyond closing so the UI can explain the risk. */
39 |     public static LocalDateTime project(LocalDateTime at,long serviceMinutes) {
40 |         LocalDateTime start=nextOpen(at);
41 |         if(start==null)return null;
42 |
43 |         LocalDateTime predicted=start.plusMinutes(Math.max(0,serviceMinutes));
44 |         LocalDateTime lunch=start.toLocalDate().atTime(13,0);
45 |         if(start.isBefore(lunch) && !predicted.isBefore(lunch))predicted=predicted.plusHours(1);
46 |         return predicted;
47 |     }
48 |     /** Count only scheduled processing windows, excluding pre-opening, lunch and overnight. */
49 |     public static double processingMinutes(LocalDateTime start,LocalDateTime end) {
50 |         if(start==null||end==null||!end.isAfter(start))return 0;
51 |         long seconds=0;
52 |         for(LocalDate day=start.toLocalDate();!day.isAfter(end.toLocalDate());day=day.plusDays(1)) {

```

```
52 |         for(int[] range:new int[][]{{10,13},{14,15}}) {
53 |             LocalDateTime a=day.atTime(range[0],0),b=day.atTime(range[1],0);
54 |             if(start.isAfter(a))a=start;if(end.isBefore(b))b=end;
55 |             if(b.isAfter(a))seconds+=Duration.between(a,b).getSeconds();
56 |         }
57 |     }
58 |     return seconds/60.0;
59 | }
```

src/main/java/com/queue/service/ServiceService.java

Thin catalogue adapter that delegates to ServiceDAO.

Numbered source

```
1 | package com.queue.service;
2 |
3 | import com.queue.dao.ServiceDAO;
4 | import com.queue.model.Service;
5 |
6 | import java.util.List;
7 |
8 | public class ServiceService {
9 |
10 |     private ServiceDAO serviceDAO;
11 |
12 |     public ServiceService() {
13 |         serviceDAO = new ServiceDAO();
14 |     }
15 |
16 |     public List<Service> getAllServices() {
17 |         return serviceDAO.getAllServices();
18 |     }
19 | }
```

src/main/java/com/queue/service/StaffService.java

Staff business adapter. Resolves login/profile and validates account/counter service assignments.

Read these lines first

Line 127: Builds the response consumed by another layer. Identify which fields the frontend renders.

Numbered source

```

1 | package com.queue.service;
2 |
3 | import com.queue.dao.StaffDAO;
4 | import com.queue.dao.QueueDAO;
5 | import com.queue.dao.CounterDAO;
6 | import com.queue.model.Staff;
7 | import com.queue.model.Queue;
8 |
9 | public class StaffService {
10 |
11 |     private StaffDAO staffDAO;
12 |     private QueueDAO queueDAO;
13 |     private CounterDAO counterDAO;
14 |
15 |     public StaffService() {
16 |         staffDAO = new StaffDAO();
17 |         queueDAO = new QueueDAO();
18 |         counterDAO = new CounterDAO();
19 |     }
20 |
21 |     public Staff login(String email, String password) {
22 |         return staffDAO.loginStaff(email, password);
23 |     }
24 |
25 |     public Staff getStaffProfile(int staffId) {
26 |         return staffDAO.getStaffById(staffId);
27 |     }
28 |
29 |     public Queue callNextStudent(int serviceId) {
30 |         return queueDAO.callNext(serviceId);
31 |     }
32 |
33 |     public Queue callStudent(int staffId, int queueId) {
34 |
35 |         Staff staff = staffDAO.getStaffById(staffId);
36 |         Queue queue = queueDAO.getQueueById(queueId);
37 |
38 |         if (staff == null || queue == null) {
39 |             return null;
40 |         }
41 |
42 |         if (staff.getServiceId() != queue.getServiceId()) {
43 |             return null;
44 |         }
45 |
46 |         return queueDAO.callStudent(queueId);
47 |     }
48 |
49 |     public boolean startService(int staffId, int queueId) {
50 |
51 |         Staff staff = staffDAO.getStaffById(staffId);
52 |         Queue queue = queueDAO.getQueueById(queueId);
53 |
54 |         if (staff == null || queue == null) {
55 |             return false;
56 |         }
57 |
58 |         if (staff.getServiceId() != queue.getServiceId()) {

```

```
59 |         return false;
60 |     }
61 |
62 |     return queueDAO.startService(queueId);
63 | }
64 |
65 | public boolean completeService(int staffId, int queueId) {
66 |
67 |     Staff staff = staffDAO.getStaffById(staffId);
68 |     Queue queue = queueDAO.getQueueById(queueId);
69 |
70 |     if (staff == null || queue == null) {
71 |         return false;
72 |     }
73 |
74 |     if (staff.getServiceId() != queue.getServiceId()) {
75 |         return false;
76 |     }
77 |
78 |     return queueDAO.completeService(queueId);
79 | }
80 |
81 | public boolean skipStudent(int staffId, int queueId) {
82 |
83 |     Staff staff = staffDAO.getStaffById(staffId);
84 |     Queue queue = queueDAO.getQueueById(queueId);
85 |
86 |     if (staff == null || queue == null) {
87 |         return false;
88 |     }
89 |
90 |     if (staff.getServiceId() != queue.getServiceId()) {
91 |         return false;
92 |     }
93 |
94 |     return queueDAO.skipStudent(queueId);
95 | }
96 |
97 | public java.util.List<Queue> getActiveQueue(int staffId) {
98 |
99 |     Staff staff = staffDAO.getStaffById(staffId);
100 |
101 |     if (staff == null) {
102 |         return new java.util.ArrayList<>();
103 |     }
104 |
105 |     return queueDAO.getActiveQueue(staff.getServiceId());
106 | }
107 |
108 | public boolean updateCounterStatus(int staffId, boolean active) {
109 |
110 |     Staff staff = staffDAO.getStaffById(staffId);
111 |
112 |     if (staff == null) {
113 |         return false;
114 |     }
115 |
116 |     return counterDAO.updateCounterStatus(
117 |         staff.getCounterId(),
118 |         active
119 |     );
120 | }
121 |
122 | public java.util.Map<String, Object> getCounterDetails(int staffId) {
123 |
124 |     Staff staff = staffDAO.getStaffById(staffId);
125 |
126 |     if (staff == null) {
127 |         return new java.util.HashMap<>();
128 |     }
129 | }
```

```
130 |     return counterDAO.getCounterDetails(  
131 |         staff.getCounterId()  
132 |     );  
  
133 | }  
134 |  
135 | public java.util.List<Queue> getRecentActivity(int staffId) {  
136 |  
137 |     Staff staff = staffDAO.getStaffById(staffId);  
138 |  
139 |     if (staff == null) {  
140 |         return new java.util.ArrayList<>();  
141 |     }  
142 |  
143 |     return queueDAO.getRecentActivity(  
144 |         staff.getServiceId()  
145 |     );  
146 | }  
147 | }
```

src/main/java/com/queue/service/StudentService.java

Student business adapter around registration/login DAO operations and server input validation.

Numbered source

```
1 | package com.queue.service;
2 |
3 | import com.queue.dao.QueueDAO;
4 | import com.queue.dao.StudentDAO;
5 | import com.queue.model.Queue;
6 | import com.queue.model.Student;
7 |
8 | public class StudentService {
9 |
10 |     private QueueDAO queueDAO;
11 |     private StudentDAO studentDAO;
12 |
13 |     public StudentService() {
14 |         queueDAO = new QueueDAO();
15 |         studentDAO = new StudentDAO();
16 |     }
17 |
18 |     // Student registration
19 |     public boolean registerStudent(Student student) {
20 |         return studentDAO.registerStudent(student);
21 |     }
22 |
23 |     // Student login
24 |     public Student loginStudent(String email, String password) {
25 |         return studentDAO.loginStudent(email, password);
26 |     }
27 |
28 |     // Join queue
29 |     public boolean joinQueue(Queue queue) {
30 |         return queueDAO.joinQueue(queue);
31 |     }
32 |
33 |     // Get queue status
34 |     public Queue getQueueStatus(int studentId, int serviceId) {
35 |         return queueDAO.getQueueStatus(studentId, serviceId);
36 |     }
37 |
38 |     // Get queue position
39 |     public int getQueuePosition(int studentId, int serviceId) {
40 |         return queueDAO.getQueuePosition(studentId, serviceId);
41 |     }
42 |
43 |     // Get current token
44 |     public int getCurrentToken(int serviceId) {
45 |         return queueDAO.getCurrentToken(serviceId);
46 |     }
47 | }
```

src/main/java/com/queue/service/WaitingTimeService.java

Recent completed processing samples and estimate helpers. Distinguish measured average from configuration fallback and preview rounding.

Read these lines first

Line 6: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 71: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 72: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 77: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 82: try-with-resources closes database/IO resources automatically when the block ends.

Line 83: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 85: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 86: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Numbered source

```

1 | package com.queue.service;
2 |
3 | import com.queue.util.DBConnection;
4 |
5 | import java.sql.Connection;
6 | import java.sql.PreparedStatement;
7 | import java.sql.ResultSet;
8 |
9 | /**
10 |  * Service to calculate estimated waiting time based on historical service performance,
11 |  * queue length, and active counter staff.
12 |  */
13 | public class WaitingTimeService {
14 |
15 |     public static final String SERVICE_UNAVAILABLE = "SERVICE CURRENTLY UNAVAILABLE";
16 |
17 |     /**
18 |      * Calculates estimated waiting time formatted as a string.
19 |      * Formula: estimatedWait = (studentsAhead * averageServiceTime) / activeStaff
20 |      *
21 |      * @param serviceId      ID of the service
22 |      * @param studentsAhead  Number of students ahead in queue
23 |      * @param activeStaff    Number of active staff currently serving
24 |      * @return String description of estimated wait time or SERVICE CURRENTLY UNAVAILABLE
25 |      */
26 |     public String calculateEstimatedWaitingTime(int serviceId, int studentsAhead, int activeStaff) {
27 |         if (activeStaff <= 0) {
28 |             return SERVICE_UNAVAILABLE;
29 |         }
30 |
31 |         if (studentsAhead < 0) {
32 |             studentsAhead = 0;
33 |         }
34 |
35 |         double averageServiceTime = getAverageServiceTime(serviceId);
36 |         int estimatedMinutes = (int) Math.round((studentsAhead * averageServiceTime) / activeStaff);
37 |
38 |         if (estimatedMinutes <= 0) {
39 |             return "Less than a minute";
40 |         } else if (estimatedMinutes == 1) {
41 |             return "1 minute";
42 |         } else {
43 |             return estimatedMinutes + " minutes";
44 |         }
45 |     }
46 |
47 |     /**
48 |      * Calculates estimated waiting time in minutes as an integer.
49 |      * Returns -1 if activeStaff is 0 or negative.
50 |      */
51 |     public int calculateEstimatedWaitMinutes(int serviceId, int studentsAhead, int activeStaff) {
52 |         if (activeStaff <= 0) {
53 |             return -1;
54 |         }
55 |
56 |         if (studentsAhead < 0) {
57 |             studentsAhead = 0;
58 |         }
59 |
60 |         double averageServiceTime = getAverageServiceTime(serviceId);
61 |         return (int) Math.round((studentsAhead * averageServiceTime) / activeStaff);
62 |     }
63 |
64 |     /**
65 |      * Retrieves average service time in minutes.
66 |      * First checks historical completed records from the queue table.
67 |      * If no completed records exist, falls back to the configured service table average.
68 |      */
69 |     public double getAverageServiceTime(int serviceId) {

```

```

70 |         String historicalSql="SELECT started_at,completed_at,processing_seconds FROM queue WHERE
    | service_id=? AND status='COMPLETED' AND started_at IS NOT NULL AND completed_at>started_at ORDER BY
    | completed_at DESC,queue_id DESC LIMIT 20";
71 |         try(Connection c=DBConnection.getConnection();PreparedStatement
    | p=c.prepareStatement(historicalSql)) {
72 |             p.setInt(1,serviceId);try(ResultSet r=p.executeQuery()) {
73 |                 double total=0;int count=0;
74 |                 while(r.next()){double minutes=r.getObject(3)!=null?r.getDouble(3)/60.0:ServiceHours.pr
    | ocessingMinutes(r.getTimestamp(1).toLocalDateTime(),r.getTimestamp(2).toLocalDateTime());if(minutes>0){
    | total+=minutes;count++;}}
75 |                 if(count>0)return Math.max(1.0/60.0,total/count);
76 |             }
77 |         }catch(Exception error){throw new IllegalStateException("Service estimate unavailable",error);}
78 |         // Fallback to average_service_time defined in services table
79 |         String serviceConfigSql =
80 |             "SELECT average_service_time FROM services WHERE service_id = ?";
81 |
82 |         try (Connection connection = DBConnection.getConnection();
83 |             PreparedStatement statement = connection.prepareStatement(serviceConfigSql)) {
84 |
85 |             statement.setInt(1, serviceId);
86 |             try (ResultSet resultSet = statement.executeQuery()) {
87 |                 if (resultSet.next()) {
88 |                     int configAvg = resultSet.getInt("average_service_time");
89 |                     if (configAvg > 0) {
90 |                         return configAvg;
91 |                     }
92 |                 }
93 |             }
94 |         } catch (Exception e) {
95 |             e.printStackTrace();
96 |         }
97 |
98 |         // Default standard fallback if service not found or 0
99 |         return 10.0;
100 |     }
101 | }

```

src/main/java/com/queue/util/DBConnection.java

Connection factory. Environment variables override an explicitly loaded local properties file. DriverManager opens the MySQL connection.

Read these lines first

Line 17: try-with-resources closes database/IO resources automatically when the block ends.

Line 18: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 24: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | package com.queue.util;
2 |
3 | import java.sql.Connection;
4 | import java.sql.DriverManager;
5 | import java.sql.SQLException;
6 | import java.nio.file.Files;
7 | import java.nio.file.Path;
8 | import java.io.InputStream;
9 | import java.util.Properties;
10 |
11 | /** Local credentials stay outside the WAR and Git. Environment variables take precedence. */
12 | public class DBConnection {
13 |     public static Connection getConnection() throws SQLException {
14 |         Properties config = new Properties();
15 |         String configPath = System.getProperty("queue.config", System.getenv("QUEUE_DB_CONFIG"));
16 |         if (configPath != null && !configPath.isBlank()) {
17 |             try (InputStream stream = Files.newInputStream(Path.of(configPath))) { config.load(stream);
18 |             catch (Exception e) { throw new SQLException("Cannot read the local database
19 | configuration.", e); }
20 |         }
21 |         String url = value("QUEUE_DB_URL", config, "url",
22 | "jdbc:mysql://127.0.0.1:3306/campus_queue?connectionTimeZone=Asia/Kolkata");
23 |         String user = value("QUEUE_DB_USER", config, "user", "root");
24 |         String password = value("QUEUE_DB_PASSWORD", config, "password", "");
25 |         try { Class.forName("com.mysql.cj.jdbc.Driver"); }
26 |         catch (ClassNotFoundException e) { throw new SQLException("MySQL JDBC driver not found.", e); }
27 |         return DriverManager.getConnection(url, user, password);
28 |     }
29 |     private static String value(String env, Properties config, String key, String fallback) {
30 |         String value = System.getenv(env);
31 |         return value != null ? value : config.getProperty(key, fallback);
32 |     }
33 | }
```

src/main/java/com/queue/util/DatabaseSetup.java

Explicit schema/migration runner for local provisioning. A deployment does not automatically execute these scripts merely by pushing Git.

Read these lines first

Line 13: try-with-resources closes database/IO resources automatically when the block ends.

Line 19: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | package com.queue.util;
2 |
3 | import java.nio.file.Files;
4 | import java.nio.file.Path;
5 | import java.sql.Connection;
6 | import java.sql.Statement;
7 |
8 | /** Explicit local setup command; never runs automatically when the web application starts. */
9 | public final class DatabaseSetup {
10 |     public static void main(String[] args) throws Exception {
11 |         if (args.length != 1) throw new IllegalArgumentException("Supply the schema file path.");
12 |         String schema = Files.readString(Path.of(args[0])).replaceAll("(?m)^\s*--.*$", "");
13 |         try (Connection connection = DBConnection.getConnection(); Statement statement =
14 |             connection.createStatement()) {
15 |             for (String sql : schema.split(";")) {
16 |                 if (!sql.isBlank()) statement.execute(sql.trim());
17 |             }
18 |             System.out.println("Database connection verified. Supplied SQL script executed
19 | successfully.");
20 |             System.out.println("Existing records were retained. Staff accounts and counter availability
21 | were not fabricated.");
22 |         } catch (java.sql.SQLException error) {
23 |             System.err.println("Database setup failed (SQL state " + error.getSQLState()
24 |                 + ", code " + error.getErrorCode() + "). Check your local credentials and MySQL
25 | service.");
26 |             System.exit(1);
27 |         }
28 |     }
29 | }

```

src/main/java/com/queue/util/Json.java

Recursive JSON serializer for null, number, boolean, maps, lists and escaped string values.

Numbered source

```

1 | package com.queue.util;
2 |
3 | import java.util.Map;
4 |
5 | public final class Json {
6 |     private Json() {}
7 |     public static String encode(Object value) {
8 |         if (value == null) return "null";
9 |         if (value instanceof Number || value instanceof Boolean) return value.toString();
10 |         if (value instanceof Map<?, ?> map) {
11 |             StringBuilder out = new StringBuilder("{}");
12 |             for (var entry : map.entrySet()) { if (out.length() > 1) out.append(',');
13 | out.append(encode(entry.getKey().toString())).append(':').append(encode(entry.getValue())); }
14 |             return out.append('}').toString();
15 |         }
16 |         if (value instanceof Iterable<?> list) {
17 |             StringBuilder out = new StringBuilder("[");
18 |             for (var item : list) { if (out.length() > 1) out.append(','); out.append(encode(item)); }
19 |             return out.append(']').toString();
20 |         }
21 |         StringBuilder out = new StringBuilder("");
22 |         for (char c : value.toString().toCharArray()) {
23 |             switch (c) {
24 |                 case '"': out.append("\\\""); break;
25 |                 case '\\': out.append("\\\\"); break;
26 |                 case '\n': out.append("\\n"); break;
27 |                 case '\r': out.append("\\r"); break;
28 |                 case '\t': out.append("\\t"); break;
29 |                 default: if (c < 32) out.append(String.format("\\u%04x", (int)c)); else out.append(c);
30 |             }
31 |         }
32 |         return out.append('').toString();
33 |     }

```

src/main/java/com/queue/util/Passwords.java

Random-salt PBKDF2 hash/verification utility. Salt and derived key are Base64 encoded, not encrypted. Legacy migration support remains.

Read these lines first

Line 4: Password key derivation: random salt plus iteration count slows guessing; this is hashing rather than reversible encryption.

Line 14: Password key derivation: random salt plus iteration count slows guessing; this is hashing rather than reversible encryption.

Line 19: Password key derivation: random salt plus iteration count slows guessing; this is hashing rather than reversible encryption.

Line 23: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Line 26: Password key derivation: random salt plus iteration count slows guessing; this is hashing rather than reversible encryption.

Line 27: Password key derivation: random salt plus iteration count slows guessing; this is hashing rather than reversible encryption.

Line 28: Failure path: explain whether it propagates, rolls back, logs safely, or risks hiding the error.

Numbered source

```

1 | package com.queue.util;
2 |
3 | import javax.crypto.SecretKeyFactory;
4 | import javax.crypto.spec.PBEKeySpec;
5 | import java.security.SecureRandom;
6 | import java.security.MessageDigest;
7 | import java.util.Base64;
8 |
9 | public final class Passwords {
10 |     private static final int ITERATIONS = 210000;
11 |     private Passwords() {}
12 |     public static String hash(String password) {
13 |         byte[] salt = new byte[16]; new SecureRandom().nextBytes(salt);
14 |         return "pbkdf2$" + ITERATIONS + "$" + Base64.getEncoder().encodeToString(salt) + "$" +
15 |         Base64.getEncoder().encodeToString(derive(password, salt, ITERATIONS));
16 |     }
17 |     public static boolean matches(String password, String stored) {
18 |         if (password == null || stored == null) return false;
19 |         // Existing team seed accounts are migrated after their first successful login.
20 |         if (!stored.startsWith("pbkdf2$")) return
21 |         MessageDigest.isEqual(password.getBytes(java.nio.charset.StandardCharsets.UTF_8),
22 |         stored.getBytes(java.nio.charset.StandardCharsets.UTF_8));
23 |         try {
24 |             String[] parts = stored.split("\\$");
25 |             return MessageDigest.isEqual(derive(password, Base64.getDecoder().decode(parts[2]),
26 |             Integer.parseInt(parts[1])), Base64.getDecoder().decode(parts[3]));
27 |         } catch (Exception e) { return false; }
28 |     }
29 |     private static byte[] derive(String password, byte[] salt, int iterations) {
30 |         PBEKeySpec spec = new PBEKeySpec(password.toCharArray(), salt, iterations, 256);
31 |         try { return
32 |         SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(spec).getEncoded(); }
33 |         catch (Exception e) { throw new IllegalStateException("Password hashing unavailable.", e); }
34 |         finally { spec.clearPassword(); }
35 |     }
36 | }
```

src/main/java/com/queue/util/StudentValidation.java

Registration syntax and length rules. Validates name, email, Indian phone and user-requested password policy; normalizes input.

Numbered source

```

1 | package com.queue.util;
2 | public final class StudentValidation {
3 |     public static void registration(com.queue.model.Student s) {
4 |         String email=s.getEmail()==null?"":s.getEmail().trim();
5 |         if(email.length()>100||!email.matches("[A-Za-z0-9.!#$%&'*/=?^`{|}~-]+@[A-Za-z0-9](?:[A-Za-z0-9-
| ]*[A-Za-z0-9])?(?:\\.[A-Za-z0-9](?:[A-Za-z0-9-
| 9]))?+"))||email.startsWith(".")||email.contains(".")||email.contains("@"))throw new
| IllegalArgumentException("Enter a valid email address.");
6 |         String phone=s.getPhone()==null?"":s.getPhone().trim();
7 |         if(!phone.matches("(?:\\+91)?[6-9][0-9]{9}"))throw new IllegalArgumentException("Enter a valid
| 10-digit Indian mobile number, optionally prefixed with +91.");
8 |         String password=s.getPassword();
9 |         if(password==null||password.length()<6||password.length()>128||password.isBlank()||!password.matches(
| "(?s).*[A-Za-z].*")||!password.matches("(?s).*[0-9].*"))throw new IllegalArgumentException("Use 6 to
| 128 characters with at least one letter and one number.");
10 |         String name=s.getStudentName()==null?"":s.getStudentName().trim();
11 |         if(name.isEmpty()||name.length()>100)throw new IllegalArgumentException("Enter your name (up to 100
| characters).");
12 |         s.setEmail(email);s.setStudentName(name);s.setPhone(phone.startsWith("+91"?phone:"+91"+phone));
13 |     }
14 | }

```

src/main/java/com/queue/util/TransactionLocks.java

Database parent-row locks. SELECT FOR UPDATE serializes operations per service; locks depend on the surrounding transaction.

Read these lines first

Line 4: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 12: This SQL requests row locks. Explain which row is locked and where commit or rollback releases it.

Line 15: This SQL requests row locks. Explain which row is locked and where commit or rollback releases it.

Line 18: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 19: Creates or binds a parameterized SQL statement. Placeholder numbering starts at one.

Line 20: Executes a row-returning query and obtains a ResultSet that must be iterated and closed.

Numbered source

```

1 | package com.queue.util;
2 |
3 | import java.sql.Connection;
4 | import java.sql.PreparedStatement;
5 | import java.sql.ResultSet;
6 | import java.sql.SQLException;
7 |
8 | /** Stable parent-row locks serialize validation and writes across server instances. */
9 | public final class TransactionLocks {
10 |     private TransactionLocks() {}
11 |     public static boolean service(Connection connection, int id) throws SQLException {
12 |         return lock(connection, "SELECT service_id FROM services WHERE service_id=? FOR UPDATE", id);
13 |     }
14 |     public static boolean student(Connection connection, int id) throws SQLException {
15 |         return lock(connection, "SELECT student_id FROM students WHERE student_id=? FOR UPDATE", id);
16 |     }
17 |     private static boolean lock(Connection connection, String sql, int id) throws SQLException {
18 |         try (PreparedStatement statement=connection.prepareStatement(sql)) {
19 |             statement.setInt(1,id);
20 |             try (ResultSet rows=statement.executeQuery()) { return rows.next(); }
21 |         }
22 |     }
23 | }

```

Dockerfile

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

Numbered source

```
1 | FROM maven:3.9-eclipse-temurin-17 AS build
2 | WORKDIR /build
3 | COPY pom.xml ./
4 | COPY src ./src
5 | RUN mvn --batch-mode --no-transfer-progress package
6 |
7 | FROM tomcat:10.1-jdk17-temurin
8 | RUN rm -rf /usr/local/tomcat/webapps/*
9 | RUN sed -i 's/<Server port="8005"/<Server port="-1"/' /usr/local/tomcat/conf/server.xml \
10 |    && grep -q '<Server port="-1"' /usr/local/tomcat/conf/server.xml
11 | COPY --from=build /build/target/queue-system.war /usr/local/tomcat/webapps/ROOT.war
12 | ENV TZ=Asia/Kolkata
13 | EXPOSE 8080
14 | CMD ["catalina.sh", "run"]
```

37. HTML, CSS and SQL source practice

QueueSystemFrontend/index.html

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

```
1 | <!doctype html>
2 | <html lang="en"><head><meta charset="UTF-8"/><meta name="viewport" content="width=device-width,
  | initial-scale=1.0"/><link rel="icon" href="/favicon.svg"/><meta name="theme-color"
  | content="#f5f3ed"/><meta name="description" content="CampusQueue - less wait, more campus. Your student
  | services, tickets, and appointments in one place."/><title>CampusQueue - Less wait. More
  | campus.</title></head><body><div id="root"></div><script type="module"
  | src="/src/main.jsx"></script></body></html>
```

frontend/index.html

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

```
1 | <!doctype html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-
  | width,initial-scale=1"><link rel="icon" href="/favicon.svg"><title>Campus Queue -
  | Staff</title></head><body><div id="root"></div><script type="module"
  | src="/src/main.jsx"></script></body></html>
```

database/schema.sql

Database structure/migration/seed script. Read constraints, dependencies, repeatability and schema selection before executing.

```
1 | CREATE DATABASE IF NOT EXISTS campus_queue;
2 |
3 | USE campus_queue;
4 |
5 | -- =====
6 | -- STUDENTS
7 | -- =====
8 |
9 | CREATE TABLE IF NOT EXISTS students (
10 |     student_id INT PRIMARY KEY AUTO_INCREMENT,
11 |     student_name VARCHAR(100) NOT NULL,
12 |     email VARCHAR(100) UNIQUE NOT NULL,
13 |     phone VARCHAR(15),
14 |     password VARCHAR(255) NOT NULL
15 | );
16 |
17 | -- =====
18 | -- SERVICES
19 | -- =====
20 |
21 | CREATE TABLE IF NOT EXISTS services (
22 |     service_id INT PRIMARY KEY AUTO_INCREMENT,
23 |     service_name VARCHAR(100) NOT NULL,
24 |     description VARCHAR(255),
25 |     average_service_time INT NOT NULL
26 | );
27 |
28 | -- =====
29 | -- COUNTERS
30 | -- Each counter belongs to one service
31 | -- =====
32 |
33 | CREATE TABLE IF NOT EXISTS counters (
34 |     counter_id INT PRIMARY KEY AUTO_INCREMENT,
35 |     counter_name VARCHAR(100) NOT NULL,
36 |     service_id INT NOT NULL,
37 |     is_active BOOLEAN NOT NULL DEFAULT TRUE,
38 |     FOREIGN KEY (service_id) REFERENCES services(service_id)
39 | );
40 |
41 | -- =====
42 | -- STAFF
```

```

43 | -- Each staff member is assigned to one counter
44 | -- Counter determines the service

45 | -- =====
46 |
47 | CREATE TABLE IF NOT EXISTS staff (
48 |     staff_id INT PRIMARY KEY AUTO_INCREMENT,
49 |     staff_name VARCHAR(100) NOT NULL,
50 |     email VARCHAR(100) UNIQUE NOT NULL,
51 |     password VARCHAR(255) NOT NULL,
52 |     service_id INT,
53 |     counter_id INT,
54 |     FOREIGN KEY (service_id) REFERENCES services(service_id),
55 |     FOREIGN KEY (counter_id) REFERENCES counters(counter_id)
56 | );
57 |
58 | -- =====
59 | -- QUEUE
60 | -- Queue is SERVICE-LEVEL.
61 | -- No counter_id is stored here.
62 | -- Multiple counters can serve the same queue.
63 | -- =====
64 |
65 | CREATE TABLE IF NOT EXISTS queue (
66 |     queue_id INT PRIMARY KEY AUTO_INCREMENT,
67 |     student_id INT NOT NULL,
68 |     service_id INT NOT NULL,
69 |     token_number INT NOT NULL,
70 |     status VARCHAR(20) NOT NULL DEFAULT 'WAITING',
71 |     joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
72 |     called_at TIMESTAMP NULL,
73 |     started_at TIMESTAMP NULL,
74 |     completed_at TIMESTAMP NULL,
75 |     FOREIGN KEY (student_id) REFERENCES students(student_id),
76 |     FOREIGN KEY (service_id) REFERENCES services(service_id)
77 | );
78 |
79 | -- =====
80 | -- BOOKINGS
81 | -- =====
82 |
83 | CREATE TABLE IF NOT EXISTS bookings (
84 |     booking_id INT PRIMARY KEY AUTO_INCREMENT,
85 |     student_id INT NOT NULL,
86 |     service_id INT NOT NULL,
87 |     booking_date DATE NOT NULL,
88 |     booking_time TIME NOT NULL,

89 |     status VARCHAR(20) NOT NULL DEFAULT 'BOOKED',
90 |     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
91 |     FOREIGN KEY (student_id) REFERENCES students(student_id),
92 |     FOREIGN KEY (service_id) REFERENCES services(service_id)
93 | );

```

database/migrations/001_daily_reservations.sql

Database structure/migration/seed script. Read constraints, dependencies, repeatability and schema selection before executing.

```

1 | USE campus_queue;
2 | CREATE TABLE IF NOT EXISTS counter_states (
3 |     counter_id INT PRIMARY KEY,
4 |     status VARCHAR(20) NOT NULL,
5 |     FOREIGN KEY(counter_id) REFERENCES counters(counter_id) ON DELETE CASCADE
6 | );
7 | CREATE TABLE IF NOT EXISTS queue_reservations (
8 |     queue_id INT PRIMARY KEY,
9 |     visit_date DATE NOT NULL,
10 | queue_order INT NOT NULL,
11 | missed_turns INT NOT NULL DEFAULT 0,
12 | counter_id INT NULL,
13 | staff_id INT NULL,
14 | FOREIGN KEY(queue_id) REFERENCES queue(queue_id) ON DELETE CASCADE,

```

```

15 | FOREIGN KEY(counter_id) REFERENCES counters(counter_id),
16 | FOREIGN KEY(staff_id) REFERENCES staff(staff_id),
17 | INDEX daily_order(visit_date,queue_order)
18 | );
19 | CREATE TABLE IF NOT EXISTS queue_events (
20 | event_id BIGINT PRIMARY KEY AUTO_INCREMENT,
21 | queue_id INT NOT NULL,
22 | staff_id INT NULL,
23 | event_type VARCHAR(30) NOT NULL,
24 | occurred_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
25 | FOREIGN KEY(queue_id) REFERENCES queue(queue_id) ON DELETE CASCADE
26 | );
27 | INSERT INTO queue_reservations(queue_id,visit_date,queue_order)
28 | SELECT q.queue_id,DATE(q.joined_at),q.token_number FROM queue q
29 | LEFT JOIN queue_reservations r ON r.queue_id=q.queue_id WHERE r.queue_id IS NULL;

```

database/migrations/002_processing_duration.sql

Database structure/migration/seed script. Read constraints, dependencies, repeatability and schema selection before executing.

```

1 | USE campus_queue;
2 | SET @ddl = IF((SELECT COUNT(*) FROM information_schema.columns WHERE table_schema=DATABASE() AND
| table_name='queue' AND column_name='processing_seconds')=0, 'ALTER TABLE queue ADD COLUMN
| processing_seconds DOUBLE NULL', 'SELECT 1');
3 | PREPARE migration_stmt FROM @ddl;
4 | EXECUTE migration_stmt;
5 | DEALLOCATE PREPARE migration_stmt;
6 | SET @ddl = IF((SELECT COUNT(*) FROM information_schema.columns WHERE table_schema=DATABASE() AND
| table_name='queue' AND column_name='processing_clock_offset_seconds')=0, 'ALTER TABLE queue ADD COLUMN
| processing_clock_offset_seconds BIGINT NULL', 'SELECT 1');
7 | PREPARE migration_stmt FROM @ddl;
8 | EXECUTE migration_stmt;
9 | DEALLOCATE PREPARE migration_stmt;

```

database/seed_services.sql

Database structure/migration/seed script. Read constraints, dependencies, repeatability and schema selection before executing.

```

1 | USE campus_queue;
2 |
3 | -- User-approved local catalogue. Re-running does not create duplicate names.
4 | INSERT INTO services (service_name, description, average_service_time)
5 | SELECT 'General Inquiries', 'General questions and campus information', 10
6 | WHERE NOT EXISTS (SELECT 1 FROM services WHERE service_name = 'General Inquiries');
7 |
8 | INSERT INTO services (service_name, description, average_service_time)
9 | SELECT 'Fee Payment', 'Tuition, fees, and financial transactions', 15
10 | WHERE NOT EXISTS (SELECT 1 FROM services WHERE service_name = 'Fee Payment');
11 |
12 | INSERT INTO services (service_name, description, average_service_time)
13 | SELECT 'Document Verification', 'ID checks, transcript verification, and forms', 20
14 | WHERE NOT EXISTS (SELECT 1 FROM services WHERE service_name = 'Document Verification');

```

pom.xml

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

```

1 | <?xml version="1.0" encoding="UTF-8"?>
2 |
3 | <project xmlns="http://maven.apache.org/POM/4.0.0"
4 |         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5 |         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
6 |         https://maven.apache.org/xsd/maven-4.0.0.xsd">
7 |
8 |     <modelVersion>4.0.0</modelVersion>
9 |
10 |    <groupId>com.queue</groupId>
11 |    <artifactId>queue-system</artifactId>
12 |    <version>1.0-SNAPSHOT</version>
13 |
14 |    <packaging>war</packaging>

```

```

15 |
16 |     <properties>
17 |         <maven.compiler.source>17</maven.compiler.source>
18 |         <maven.compiler.target>17</maven.compiler.target>
19 |         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
20 |     </properties>
21 |
22 |     <dependencies>
23 |
24 |         <!-- Jakarta Servlet API for Tomcat 10.1 -->
25 |         <dependency>
26 |             <groupId>jakarta.servlet</groupId>
27 |             <artifactId>jakarta.servlet-api</artifactId>
28 |             <version>6.0.0</version>
29 |             <scope>provided</scope>
30 |         </dependency>
31 |
32 |         <!-- MySQL JDBC Driver -->
33 |         <dependency>
34 |             <groupId>com.mysql</groupId>
35 |             <artifactId>mysql-connector-j</artifactId>
36 |             <version>9.4.0</version>
37 |         </dependency>
38 |
39 |     </dependencies>
40 |
41 |     <build>
42 |         <finalName>queue-system</finalName>
43 |
44 |         <plugins>
45 |
46 |             <plugin>
47 |                 <groupId>org.apache.maven.plugins</groupId>
48 |                 <artifactId>maven-compiler-plugin</artifactId>
49 |                 <version>3.13.0</version>
50 |             </plugin>
51 |
52 |             <plugin>
53 |                 <groupId>org.apache.maven.plugins</groupId>
54 |                 <artifactId>maven-war-plugin</artifactId>
55 |                 <version>3.4.0</version>
56 |                 <configuration>
57 |                     <!-- React is served by Vite or a static host; Tomcat serves the team backend and
58 | JSP modules. -->
59 |                     <warSourceExcludes>ui/**, student/*.html</warSourceExcludes>
60 |                     <packagingExcludes>ui/**, student/*.html</packagingExcludes>
61 |                 </configuration>
62 |             </plugin>
63 |         </plugins>
64 |     </build>
65 | </project>

```

QueueSystemFrontend/netlify.toml

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

```

1 | [build]
2 | command = "npm run build && node ../scripts/netlify-redirects.mjs"
3 | publish = "dist"
4 |
5 | [build.environment]
6 | NODE_VERSION = "22"

```

frontend/netlify.toml

Supporting source or configuration. Check the entry point or caller, inputs, outputs and failure handling when explaining this file.

```

1 | [build]
2 | command = "npm run build && node ../scripts/netlify-redirects.mjs"
3 | publish = "dist"

```

```

4 |
5 | [build.environment]
6 | NODE_VERSION = "22"

```

QueueSystemFrontend/src/styles/original-identity.css - CSS orientation excerpt

The full stylesheet stays in the repository. Explain selectors, cascade, variables, box sizing, flex/grid, media queries and focus styles using this opening excerpt, then navigate to the actual selected rule.

```

1 | /* Evolve the existing warm visual identity rather than replacing it. */
2 | :root {
3 |   --bg: #f5ebe6;
4 |   --surface: rgba(255, 255, 255, 0.94);
5 |   --ink: #1f0f0a;
6 |   --muted: #644b42;
7 |   --line: rgba(229, 92, 38, 0.22);
8 |   --accent: #e74c28;
9 |   --soft: rgba(255, 149, 0, 0.09);
10 |  --green: #318455;
11 |  --orange: #ff9500;
12 | }
13 | [data-theme="dark"] {
14 |   --bg: #1a0c08;
15 |   --surface: rgba(61, 30, 21, 0.88);
16 |   --ink: #fff8f0;
17 |   --muted: #e8d0c5;
18 |   --line: rgba(255, 149, 0, 0.28);
19 |   --accent: #ff704c;
20 |   --soft: rgba(255, 149, 0, 0.1);
21 |   --green: #94ce9e;
22 |   --orange: #ff9500;
23 | }
24 | body {
25 |   background: var(--bg);
26 |   isolation: isolate;
27 | }
28 | body::before {
29 |   content: "";
30 |   position: fixed;
31 |   inset: 0;
32 |   pointer-events: none;
33 |   z-index: -1;
34 |   background:
35 |     radial-gradient(
36 |       ellipse 620px 620px at 12% 16%,
37 |       rgba(255, 149, 0, 0.48),
38 |       transparent 74%
39 |     ),
40 |     radial-gradient(
41 |       ellipse 520px 520px at 94% 90%,
42 |       rgba(255, 59, 48, 0.35),
43 |       transparent 75%
44 |     );
45 | }
46 | [data-theme="dark"] body::before {
47 |   background:
48 |     radial-gradient(
49 |       ellipse 680px 650px at 10% 18%,
50 |       rgba(255, 149, 0, 0.46),
51 |       transparent 74%
52 |     ),
53 |     radial-gradient(
54 |       ellipse 600px 560px at 95% 90%,
55 |       rgba(255, 59, 48, 0.38),
56 |       transparent 75%
57 |     );
58 | }
59 | h1,

```

```

60 | h2,
61 | h3 {
62 |   font-weight: 700;
63 | }
64 | .serif {
65 |   font-family: var(--serif);
66 |   font-style: italic;
67 |   font-weight: 400;
68 |   color: var(--accent);
69 | }
70 | .brand {
71 |   font-size: 22px;
72 |   color: var(--accent);
73 | }
74 | .brand-mark {
75 |   background: linear-gradient(135deg, #ff3b30, #ff9500);
76 |   color: #fff;
77 |   letter-spacing: -4px;
78 | }
79 | .brand-mark span {
80 |   color: #fff;
81 | }
82 | .brand-caption {
83 |   color: var(--muted);
84 | }
85 | .sidebar {
86 |   background: rgba(255, 255, 255, 0.025);
87 |   backdrop-filter: blur(20px);
88 | }

89 | .sidebar nav a {
90 |   border-radius: 100px;

```

frontend/src/style.css - CSS orientation excerpt

The full stylesheet stays in the repository. Explain selectors, cascade, variables, box sizing, flex/grid, media queries and focus styles using this opening excerpt, then navigate to the actual selected rule.

```

1 | :root{font-family:Arial,sans-serif;color:#172c3d;background:#f4f5f7;font-size:14px}*{box-sizing:border-
| box}body{margin:0}button,input,select{font:inherit}button,select{cursor:pointer}button{border:1px solid
| #d5dce4;background:white;color:#172c3d;border-radius:5px;padding:9px
| 13px}button:disabled{opacity:.5;cursor:default}button:focus-visible,input:focus-visible,select:focus-
| visible{outline:3px solid #73a9e2;outline-offset:2px}.app-shell{display:flex;min-
| height:100vh}.sidebar{width:240px;flex-shrink:0;background:linear-
| gradient(170deg,#101725,#234864);color:#edf3fa;padding:30px 20px;display:flex;flex-
| direction:column;position:sticky;top:0;height:100vh}.side-brand{display:flex;align-
| items:center;gap:12px;font-weight:bold;margin-bottom:45px}.brand-mark{display:grid;place-
| items:center;width:46px;height:46px;border-radius:9px;background:#21374f;color:white;font-
| weight:bold}.brand-mark.small{width:28px;height:28px;background:none;font-size:11px}.nav-
| section,.session-label,.eyebrow{font-size:10px;text-transform:uppercase;letter-
| spacing:1.3px;color:#7b8998}.nav-section{margin:0 10px 15px}.nav-item{width:100%;text-
| align:left;border:0;background:none;color:#dce4ed;display:flex;gap:13px;margin:5px 0;padding:13px}.nav-
| item.active{background:#ffffff12;color:white}.side-bottom{margin-top:auto;border-top:1px solid
| #ffffff20;padding-top:22px;display:grid;gap:8px}.side-bottom>span{font-size:12px;color:#acbdca}.logout-
| btn{margin-top:8px;color:white;background:none;border-color:#ffffff30}.main-
| content{padding:40px;flex:1;min-width:0}.topbar{display:flex;justify-content:space-between;align-
| items:center;margin-bottom:28px}h1{font-size:27px;margin:8px 0}h2{font-size:16px;margin:0 0 6px}p{font-
| size:12px;color:#7b8998;margin:0}.topbar-right{display:flex;gap:18px;align-items:center}.live-
| pill{font-size:12px}.live-pill span{display:inline-
| block;width:7px;height:7px;background:#4ba47a;border-radius:50%;margin-
| right:7px}.clock{color:#7b8998;font-size:12px}.context-bar{display:grid;grid-template-
| columns:repeat(4,1fr);gap:20px;padding:20px 0;border-top:1px solid #dee3e8;border-bottom:1px solid
| #dee3e8;margin-bottom:28px}.context-bar>div{display:flex;flex-direction:column;gap:8px}.context-bar
| span{font-size:9px;letter-spacing:1px;text-transform:uppercase;color:#7b8998}.context-bar strong{font-
| size:13px}.context-bar select{border:0;background:none;color:inherit;padding:0}.stats-
| grid{display:flex;flex-wrap:wrap;gap:12px;margin-bottom:28px}.stat-card{flex:1;min-
| width:140px;display:flex;gap:15px;padding:18px;background:#fff;border:1px solid #e8ecf0;border-
| radius:5px}.stat-card strong{display:block;font-size:23px;margin-top:9px}.stat-card small{font-
| size:11px;font-weight:normal}.stat-icon{color:#81919f;margin-top:5px}.page-grid,.dashboard-
| grid{display:grid;grid-template-columns:2fr 1fr;gap:24px}.panel{background:#fff;border:1px solid
| #e4e9ee;border-radius:6px;margin-bottom:22px;overflow:hidden}.panel-

```

```

| head{padding:21px;display:flex;align-items:center;justify-content:space-between;gap:14px;border-
| bottom:1px solid #edf0f3}.info-panel{padding:22px}.info-panel .panel-head{padding:0 0 20px}.primary-
| btn,.login-btn{background:#213b55;color:#fff;border-color:#213b55}.full{width:100%;margin-
| top:10px}.current-card{padding:22px;display:flex;gap:18px}.current-card
| span{display:block;color:#7b8998;font-size:11px;margin-bottom:8px}.current-token{font-size:27px;font-
| weight:bold}.service-actions,.current-actions{padding:0 20px 20px}.empty-state{padding:32px
| 20px;color:#8794a1;font-size:12px;text-align:center}.large-empty{padding:55px
| 20px;display:grid;gap:14px}.table-wrap{overflow:auto}table{width:100%;border-collapse:collapse;font-
| size:12px}th{text-align:left;font-size:9px;letter-spacing:1px;text-
| transform:uppercase;color:#7b8998}td,th{padding:17px;border-bottom:1px solid #edf0f3}td
| button{padding:6px 9px;margin-right:5px}.activity-item{padding:18px 22px;border-bottom:1px solid
| #edf0f3;display:grid;gap:6px}.activity-item span,.activity-item small{font-
| size:11px;color:#7b8998}.analytics-rows>div{display:flex;justify-content:space-between;padding:18px
| 0;border-bottom:1px solid #edf0f3}.workflow-list>div{display:flex;gap:16px;padding:18px 0}.metric-
| badge{font-size:25px}.login-page{min-height:100vh;display:grid;place-
| items:center;padding:24px;background:linear-gradient(135deg,#152338,#2f5066)}.login-
| card{background:white;border-radius:12px;padding:40px;width:420px;max-width:100%}.login-card .brand-
| mark{margin-bottom:23px}.login-card h1{margin:12px 0}.login-subtitle{margin-bottom:25px}.login-card
| form{display:grid;gap:12px}.login-card label{font-size:12px;margin-top:8px}.login-card
| input{padding:12px;border:1px solid #d9e0e7;border-radius:5px;width:100%;min-width:0}.password-
| wrap{display:flex;gap:7px}.login-btn{margin-top:15px;padding:13px}.login-footer{margin-top:24px;font-
| size:11px;color:#7b8998;text-align:center}.error-banner,.error-
| box{padding:14px;background:#fff1ee;color:#a43b2c;margin-bottom:16px;border-radius:6px}@media(max-
| width:900px){.sidebar{width:190px}.main-content{padding:24px}.page-grid,.dashboard-grid{grid-template-
| columns:1fr}.context-bar{grid-template-columns:repeat(2,1fr)}}@media(max-width:600px){.app-
| shell{display:block}.sidebar{position:static;width:auto;height:auto;padding:18px}.side-brand{margin-
| bottom:15px}.sidebar nav{display:flex;flex-wrap:wrap}.nav-section{display:none}.nav-
| item{width:auto;padding:10px;font-size:12px}.side-bottom{display:none}.main-
| content{padding:20px}.topbar-right{display:none}.stats-grid{gap:8px}.stat-card{min-
| width:120px}.context-bar{gap:16px}}
2 | .workspace-grid{display:grid;grid-template-columns:2fr 1fr;gap:24px}.bottom-grid{display:grid;grid-
| template-columns:repeat(3,1fr);gap:20px}.mini-panel{padding:22px;background:white;border:1px solid
| #e4e9ee;border-radius:6px;display:grid;gap:10px}.mini-panel span,.mini-panel small{font-
| size:11px;color:#7b8998}.status{font-size:10px;color:#7b8998}@media(max-width:900px){.workspace-
| grid{grid-template-columns:1fr}}@media(max-width:600px){.bottom-grid{grid-template-columns:1fr}}
3 | .side-bottom{min-width:0}.side-bottom strong,.side-bottom>span{overflow-wrap:anywhere}.context-bar
| strong{overflow-wrap:anywhere}
4 | .cq-brand{display:flex;align-items:center;gap:11px;text-decoration:none;color:inherit}.cq-brand .cq-
| symbol{width:44px;height:44px;flex-shrink:0;display:block}.cq-brand .cq-wordmark{display:flex;flex-
| direction:column;gap:1px;font-family:Outfit,Arial,sans-serif;line-height:1.05;letter-
| spacing:-.3px;font-size:19px;font-weight:500}.cq-brand .cq-wordmark strong{font-weight:800}.cq-brand
| .cq-wordmark>span{color:inherit}.login-card>.cq-brand{margin-bottom:24px}
5 | @font-face{font-family:Outfit;src:url("/fonts/Outfit.ttf");font-weight:100 900;font-display:swap;}
6 | .student-phone{display:inline-flex;align-items:center;gap:10px;flex-wrap:wrap}.phone-
| action{display:inline-flex;align-items:center;justify-content:center;width:34px;height:34px;border:1px
| solid #cbd5e1;border-radius:10px;color:#183b56;background:#f0f7ff}.phone-
| action:hover{background:#dbeeaf}.phone-action:focus-visible{outline:2px solid #2563eb;outline-
| offset:3px}.current-student{padding:0 22px 18px;line-height:1.8}

```